THÈSE

Pour obtenir le grade de

DOCTEUR DE L'UNIVERSITÉ GRENOBLE ALPES

École doctorale : EEATS - Electronique, Electrotechnique, Automatique, Traitement du Signal
Spécialité : Nano électronique et Nano technologies
Unité de recherche : CEA /LIST - Laboratoire d'intégration de systèmes et de technologies

Conception d'un Système Mémoire pour Traitement de Données Creuses

Design of a Memory System for Sparse Data Processing

Présentée par :

Valentin ISAAC--CHASSANDE

Direction de thèse :

Frédéric ROUSSEAU PROFESSEUR DES UNIVERSITES, Grenoble INP - UGA	Directeur de thèse
Adrian EVANS CEA	Co-encadrant de thèse

Rapporteurs :

Thèse soutenue publiquement le , devant le jury composé de :

Valentin ISAAC--CHASSANDE
Université Grenoble Alpes
CEA Grenoble (DRT/LIST/DSCIN/LSTA)
TIMA Laboratory (SLS)
ORCID : 0009-0007-9842-5777
valentin.isaacchassande@univ-grenoble-alpes.fr
valentin.isaacchassande@cea.fr

THESIS

DESIGN OF A MEMORY SUB-SYSTEM FOR SPARSE DATA PROCESSING

January 16, 2026

Thesis Adviser: Pr. Frédéric ROUSSEAU
Thesis Co-Supervisors: Dr. Adrian EVANS
Dr. Yves DURAND

EEATS Doctoral College, NENT Speciality
2022 – 2025 Academic Years
From 10.25.2022 to 10.24.2025

Acknowledgements

TODO

"Work. Or school. This is for the money. But what is anything else, for that matter, but required means of enhancing your bare living – better sleep and better food – like spending decades just to turn a little knob higher on a pathetic radio called existence."

Karl Kristian Flores, The Goodbye Song.

Résumé

TODO

Abstract

TODO

List of Publications

This thesis is based on the following publications:

I Valentin Isaac--Chassande, Adrian Evans, Yves Durand, and Frédéric Rousseau. 2024. Dedicated Hardware Accelerators for Processing of Sparse Matrices and Vectors: A Survey. *ACM Trans. Archit. Code Optim.* 21, 2, Article 27 (Feb. 2024), 26 pages. Reference [54]

II Valentin Isaac--Chassande, Adrian Evans, Yves Durand, and Frédéric Rousseau. 2024. SpDCache: Region-Based Reduction Cache for Outer-Product Sparse Matrix Kernels. In *2024 IEEE 35th International Conference on Application-specific Systems, Architectures and Processors (ASAP)*. IEEE Computer Society, Los Alamitos, CA, USA, 3–7. Reference [55]

III **TODO**

IV **add patent**

The papers will be referred to in the thesis using their Roman numerals.

Summary

Notations

$$\forall (a,b) \in \mathbb{Z}^2, [\![a,b]\!] = [a,b] \cap \mathbb{Z}$$

nnz, M, A, b and c, $\overline{nnz_r}$, d_r ...

TODO

Glossary

band Refers to an area of a matrix that is concentrated around the diagonal. 10, 11, 18, 19, 59, 60, 63, 64, 65, 68, 69, 70, 75, 76, 77, 78, 79, 80, 81, 82, 83, 84, 96, 97, 98, 105, 113, 115, 117, 118, 119, 128, 134, 135

banded Refers to a matrix structure where the non-zero elements are concentrated around the diagonal. 18, 19, 20, 29, 32, 54, 55, 59, 60, 62, 63, 68, 69, 70, 72, 75, 77, 80, 82, 84, 96, 97, 99, 105, 106, 115, 134

bandwidth With this typography, refers to the size of the band of a matrix. 18

common rank Refers to a rank that is common between two data-entries, when computing a kernel. 21, 22, 26, 29

dense format Is the uncompressed format where all zeros and non-zeros are stored, without any use of metadata. 19, 26, 28

eviction Is a cache event where a valid cache entry is flushed and freed. 54, 56, 57, 58, 59, 60, 61, 62, 63, 66, 68, 71, 72, 74, 75, 76, 77, 78, 80, 81, 84, 88, 90, 91, 93, 94, 95, 102, 103, 105, 107, 114, 115, 117, 134

fiber Is the generic term to describe a one-dimensional stream of data or metadata. It corresponds to a row or column in the case of a two-dimensional matrix. A stream of indices is a *fiber* of metadata. 25, 29

gather Refers to reading data from memory. We specifically use this term to categorize data transfers that are irregular. 27, 32, 36

hit Is a cache event where a given address does match one of the valid cache entries. 56, 57, 59, 61, 63, 74, 80, 88, 90, 91, 93, 94, 95, 99, 100, 102, 103, 105, 107, 114, 115, 117, 118

ineffectual operation Is an operation that leads to a neutral result, 0 for an addition or 1 for a multiplication. 25

intersection Refers to the process of finding non-zero partial sums, when computing a matrix multiplication. 21, 22, 24, 25, 26, 27, 29, 31, 32, 137

irregular access See “random” access. 11, 16, 17, 32, 55, 56, 60, 70, 71, 100, 115

irregular data Refers to data that has low spatial and temporal locality, thus being more likely to generate irregular accesses. 32, 57

metadata Is data that provides information about other data, such as index arrays of sparse formats. 10, 11, 19, 20, 25, 27, 125

miss Is a cache event where a given address does not match any of the valid cache entries. 26, 31, 56, 57, 59, 61, 63, 74, 88, 90, 93, 94, 95, 99, 114

partial sum Refers to the intermediary result before updating the output, when computing a matrix multiplication. 10, 22, 25, 26, 27

'perfectly' banded Refers to a matrix structure where all the non-zero elements are concentrated within a delimitated area around the diagonal (the band). 18, 70, 73, 75, 77, 81, 105, 106

pipeline Is a sequence of processing stages where instructions flow through each stage in order. 86, 88, 90, 91, 92, 93, 94, 95, 108, 109, 112, 113, 135

"random" access Is a memory access that occurs on an arbitrary memory address (not necessarily conscutive in memory with previous and next accesses). 10, 11, 17

"random" update Is a "random" access where the access is a read, modify and write on a given address. It is similar to a "random" *reduction*. 26

rank Refers to a dimension of the kernel. 10, 19, 21, 22, 23, 24, 27

reduction Refers to the process of updating an entry. 11, 22, 26, 27, 28, 31, 32, 54, 55, 56, 57, 58, 59, 60, 61, 63, 64, 65, 66, 67, 68, 79, 86, 87, 88, 91, 92, 95, 99, 103, 106, 107, 108, 109, 112, 113, 114, 115, 116, 117, 118, 134, 137

reduction cache Is a data-cache supporting *reduction* operations. 57, 58, 59, 60, 63, 64, 65, 66, 68, 69, 70, 71, 72, 74, 75, 77, 78, 79, 82, 84, 88, 90, 92, 95, 108

scatter Refers to updating data in memory. We specifically use this term to categorize data transfers that are irregular. 27, 32, 36

sequential accesses Are a group of memory accesses (from a data array, for example) that occur on conscutive memory addresses, in order through time. 19, 56

set Refers to a fixed number of cache-lines, a subdivision related to set-associative caches. 11, 56, 59, 60, 63, 65, 66, 70, 71, 73, 74, 80, 90, 91, 96, 107, 112

sparse format Is a compression format for sparse matrices, which avoids storing zeros by using metadata. 11, 19, 20, 23, 24, 25, 26, 32, 58, 125

tag Refers to cache metadata that allows identification of cache-line memory addresses. 56, 63

tile Refers to a sub-region of a *tiling* process. 23, 24, 31

tiling Is a method consisting in splitting data into several independent sub-regions to be processed separately. 11, 23, 24, 27, 29, 31, 32, 125, 134, 137

way Refers to the number of cache-lines that a *set* contains. 56, 59, 63, 70, 80

Acronyms

AI Artificial Intelligence. 114

BaCSC *Banded Compressed Sparse Column*. 20, 32, 96, 97, 98, 99, 100, 102, 103, 104, 105, 117, 135, 138

BCSR *Blocked Compressed Sparse Row*. 20

CAM Content Addressable Memory. 90, 91, 93, 94

CMO Cache Management Operation. 88, 91

COO *COOrdinate*. 19, 20, 28, 32, 125, 134

CSC *Compressed Sparse Column*. 19, 20, 25, 26, 28, 29, 32, 58, 71, 96, 97, 98, 99, 100, 102, 103, 104, 113, 116, 118, 125, 134, 137

CSR *Compressed Sparse Row*. 19, 20, 24, 25, 26, 28, 29, 32, 125, 134, 135, 137

DIA *DIAgonal*. 20

DRC *Dense Region Cache*. 60, 61, 62, 63, 64, 65, 66, 67, 68, 75, 77, 80, 81, 82, 84, 86, 90, 91, 92, 93, 94, 95, 96, 97, 99, 100, 103, 105, 107, 108, 113, 114, 115, 117, 118, 119, 134

ELL *ELLPACK*. 20

FPU Floating-Point Unit. 112

GC Generic Cache. 63, 64, 65, 88, 95, 96, 97, 98, 99, 100, 101, 102, 103, 104, 105, 106, 107, 112, 135

GRC *Generic Region Cache*. 60, 61, 62, 63, 65, 66, 67, 68, 71, 84, 86, 90, 92, 95, 96, 107, 108, 114, 117, 134

`Gust` *Gustavson*. 21, 22, 23, 24, 27, 29, 30, 31, 32, 36, 114, 115, 135, 136, 137

HYB *HYBbrid*. 20

IBR *In-Band Region*. 60, 61, 63, 68, 92, 93, 94, 97, 113, 135

`IP` *Inner-Product*. 21, 22, 23, 24, 25, 26, 27, 29, 30, 31, 32, 36, 73, 136, 137

L1 First-Level Cache. 116, 117, 118

L2 Second-Level Cache. 116, 117, 118

LLC Last-Level Cache. 116, 117, 118

LRU *Least Recently Used*. 56, 63, 71, 81, 82

MM Matrix-Matrix. 20, 21, 22, 23, 29, 32, 36, 137

MV Matrix-Vector. 20, 21, 22, 23, 24, 29, 32, 36, 137

nop *no-operation*. 88

OBR *Out-of-Band Region*. 60, 61, 63, 68, 92, 94, 95, 97, 113, 135

OP *Outer-Product*. 21, 22, 23, 24, 26, 27, 28, 29, 30, 31, 32, 36, 54, 55, 58, 59, 60, 63, 65, 66, 68, 69, 71, 72, 73, 75, 77, 82, 84, 97, 98, 99, 100, 103, 113, 114, 115, 134, 135, 136, 137, 138

PO *partial output*. 22, 23, 24, 26, 27, 29, 32, 66, 117

RAM Random-Access Memory. 88, 90, 91, 92, 93, 94, 95

RED *Reduction*. 57, 58, 59, 60, 61, 63, 65, 66, 88, 90, 91, 92, 93, 94, 95, 97, 98, 99, 102, 105, 107, 114, 138

RedC Reduction Cache. 63, 64, 65, 66, 88, 95, 96, 97, 98, 99, 100, 102, 103, 104, 105, 106, 107, 112, 113, 135

REG *Region*. 91, 92, 97, 98, 113

RMW *Read-Modify-Write*. 61, 62, 63, 64, 65, 79, 80, 91, 95, 112, 117

RMWQ *Read-Modify-Write Queue*. 61, 62, 91, 95

RTL Register Transfer Language. 68, 86, 88, 91, 95, 100, 101, 102, 105, 106, 107, 108, 109, 110, 112, 119, 128, 129, 130, 135, 136

SpDC SpDCache. 55, 60, 61, 62, 63, 64, 65, 66, 67, 68, 70, 71, 75, 77, 79, 81, 82, 84, 86, 87, 88, 89, 90, 91, 92, 95, 96, 97, 98, 99, 100, 102, 103, 104, 105, 106, 107, 108, 109, 110, 111, 112, 113, 114, 115, 116, 117, 118, 119, 134, 135

SpGEMM General Sparse Matrix-Matrix multiplication. 20

SpGEMV General Sparse Matrix-Vector multiplication. 20

SpMM Sparse Matrix-Dense Matrix multiplication. 20, 115

SpMSpM Sparse Matrix-Sparse Matrix multiplication. 20, 21, 23, 25, 26, 27, 28, 29, 30, 31, 32, 110, 114, 115, 135, 136, 137

SpMSpV Sparse Matrix-Sparse Vector multiplication. 20, 115, 135

SpMV Sparse Matrix-Vector multiplication. 20, 21, 24, 31, 32, 54, 55, 58, 59, 60, 63, 65, 66, 68, 69, 71, 72, 73, 75, 77, 82, 84, 97, 98, 99, 100, 103, 110, 112, 113, 114, 115, 119, 134, 137, 138

SRT *Sparse Region Table*. 60, 61, 62, 63, 64, 65, 66, 67, 68, 82, 84, 86, 90, 91, 92, 93, 94, 95, 96, 99, 100, 105, 107, 108, 114, 117, 134

Introduction

THE rapid growth of data-intensive applications has made sparse and irregular data structures increasingly prevalent in scientific computing, machine learning, and graph analytics. However, efficiently processing sparse matrices remains a fundamental challenge due to their inherent irregularity, which leads to unpredictable memory access patterns, limited opportunities for data reuse, and suboptimal utilization of hardware resources. Existing hardware accelerators have demonstrated partial solutions, yet they often rely on restrictive assumptions, exhibit limited scalability, or fail to fully exploit the memory bandwidth.

This thesis addresses the following central question:

PROBLEM STATEMENT

How can hardware and architectural mechanisms be designed to efficiently support computation with irregular sparse data, in particular random reductions, while ensuring scalability, adaptability, and high performance across diverse workloads?

To tackle this problem, we investigate the limitations of existing algorithms and accelerators, propose a novel cache architecture with support for reduction operations, tailored to sparse data. We then evaluate the behaviour of this specialized cache through modeling, simulation, and hardware prototyping.

This manuscript is structured around several key contributions. We begin with an in-depth discussion of irregular data, with a particular emphasis on sparse matrices (Chapter 1). This includes a study of fundamental sparse matrix multiplication kernels and their associated algorithms, highlighting the main hardware challenges that arise when dealing with sparse data. A high-level analysis of algorithms for sparse data computing is provided, with a focus on basic storage schemes and opportunities for data reuse. Building on this foundation, we review the state-of-the-art in hardware accelerators designed for efficient sparse matrix computing (Chapter 2). This review identifies common characteristics, proposes a classification of existing accelerators and design tendencies, offers a comparative analysis, and identifies the major tendencies across designs. This analysis of the state of the art guided our subsequent architectural choices.

Following this survey, we introduce a novel cache architecture specifically designed to address the challenge of "random" reductions (Chapter 3). The proposed cache integrates reduction support and organizes its memory into multiple regions optimized for denser and

sparser data regions. A Python model was developed to simulate memory transactions in this cache and to compare its behavior with that of a generic cache. To further investigate its performance, we complement this analysis with a probabilistic model, implemented in C, that enables rapid simulation and sizing parameters exploration depending on matrices structure (Chapter 4). At the micro-architectural level, we detail the virtual partitioning of cache memory into multiple configurable regions, which ensures both flexibility and adaptability to diverse workloads (Chapter **??**).

The proposed cache is then evaluated through RTL simulation, which reveals two distinct classes of matrices that exhibit different performance profiles (Chapter **??**). Finally, the manuscript outlines potential future enhancements to improve both performance and scalability, including a multicore extension of the cache architecture and optimizations targeting bandwidth efficiency (Chapter 6).

Contributions of the Thesis

The main contributions of this thesis can be summarized as follows:

1. **Analysis of irregular and sparse data computing** (Chapter 1)
   - Study of fundamental sparse matrix multiplication kernels and algorithms.
   - Identification of the key hardware challenges associated with sparse data processing.
   - High-level analysis of algorithms for sparse data computing, with emphasis on storage schemes and data reuse opportunities.
2. **Survey and classification of state-of-the-art accelerators** (Chapter 2)
   - Comprehensive review of hardware accelerators for sparse matrix computing.
   - Identification of common architectural features and design tendencies.
   - Comparative analysis to explain design choices under varying objectives and constraints.
3. **Design of a cache architecture for random reductions** (Chapter 3)
   - Proposal of a cache with native reduction support, capable of handling both dense and sparse data via multiple cache regions.
   - Development of a Python-based simulation model to analyze memory transactions and benchmark against a generic cache.
4. **Probabilistic modeling and exploration of cache behavior** (Chapter 4)
   - Implementation of a lightweight C-based probabilistic model for rapid simulation.
   - Exploration of cache dimensioning and behavior across diverse workloads.
5. **Micro-architecture of the reduction cache** (Chapter **??**)
   - Introduction of a virtual memory partitioning scheme enabling multiple configurable regions.
   - Emphasis on flexibility and adaptability in cache design.
6. **Evaluation through RTL simulation** (Chapter **??**)
   - Assessment of the proposed cache architecture using RTL-level simulation.
   - Identification of two distinct groups of sparse matrices leading to different performance profiles.
7. **Future directions for performance and scalability** (Chapter 6)
   - Proposal of a multicore version of the cache to extend scalability.
   - Exploration of bandwidth optimization strategies.

Chapter 1

Background

Contents

1.1 Introduction

MODERN computing systems are designed to deliver high performance by exploiting memory hierarchies, parallelism, and increasingly sophisticated hardware features. However, the efficiency of these systems is strongly tied to the regularity of memory access patterns. Applications with predictable, sequential data access can fully benefit from cache locality, prefetching mechanisms, and vectorization. In contrast, applications characterized by irregular accesses, where memory locations are not known in advance or follow non-contiguous patterns, are not able to achieve peak performance.

In domains such as graph analytics, sparse linear algebra, unstructured mesh computations, and database queries, the memory access patterns are often irregular. In these applications, pointer chasing, indirect indexing, and dynamically changing data structures disrupt spatial and temporal locality. As a result, the hardware struggles to hide the latency of memory accesses, leading to poor cache utilization and underutilization of compute ressources.

The impact of irregular accesses extends beyond raw performance. They also complicate algorithm design, hinder scalability on many-core architectures, and increase energy consumption due to repeated off-chip memory requests. Understanding and addressing these

challenges is therefore a central concern in both computer design and application optimization. This chapter provides the necessary background for analyzing irregular accesses, highlighting their sources, effects, and the architectural considerations they expose, focusing on the case of indirect indexing in sparse matrix computation. The content of this chapter is partially based on the content from Papers I and II.

DEFINITION

Irregular accesses, or **"random" accesses**, are sequences of accesses where the sequence can not be easily predicted.

1.2 Sparse Matrices

SPARSE matrices are two dimensional arrays filled with many zeros and stored in compressed formats in memory. These sparse matrices are commonly used in diverse fields such as linear algebra, where sparse matrix multiplication is used to solve linear systems of equations for scientific purposes or simulation, as well as graph analytics where the vertices of a graph are represented as a sparse matrix [19], or finally transformers or other similar AI features where weights and activations can be represented and processed as sparse matrices [31, 36]. In these contexts, matrices are large, with dimensions up to billions of elements, and highly sparse, with non-zero densities down to 10^{-7} [63].

We note matrices with capital letters A, B, C, conversely to vectors a, b, c. Their dimension are denoted using M, N and K. For example, we can define a square matrix A of dimension M, or a matrix B of dimension (K, N). We note nnz the number of non-zero elements of a matrix, and d the associated density. With the example of a matrix $A(M, N)$:

$$d_A = \frac{nnz_A}{M \times N}$$

DEFINITION

A **sparse matrix** is a matrix filled with many zeros.

NOTE

In general, we consider a matrix to be highly sparse when its number of non-zero elements has the same order of magnitude than its dimension, $nnz \sim M$. But there is no intrinsic definition of the frontier between a sparse and a dense matrix. In this manuscript, we consider a matrix as sparse when it needs to be compressed in memory (see Section 1.3).

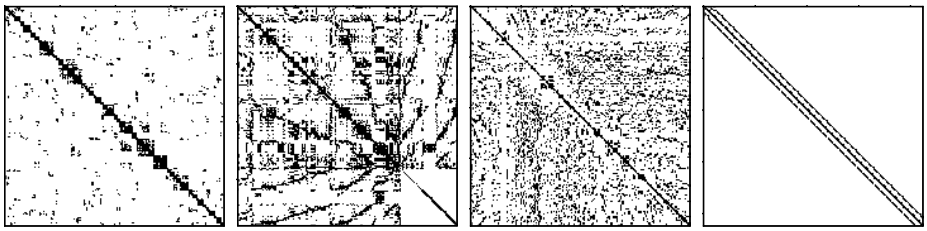
1.2.1 Banded Matrices

Matrices used in scientific computation originate from physical modelling where a system of partial differential equations is discretized, resulting in a large sparse matrix. In these systems, the forces between elements decreases quickly with their distance, creating banded matrices. Moreover, numerical techniques exist to transform matrices into banded matrices [75, 78, 113].

A banded matrix is a matrix which has a dense region around its main diagonal and which is sparser away from the diagonal. To give a few visual examples, Figure 1.1 shows the structure plots for four typical matrices found in the SuiteSparse Matrix Collection [63], having a high-density band along the diagonal. In Figure 1.2, we illustrate a banded, square matrix of dimension M. We define w_B as the width of the banded region, or *bandwidth*, as shown in *red* in the figure. Sometimes it is easier to refer to the half-*bandwidth*, w_{hB}, which respects the relation $w_B = 2 \times w_{hB} - 1$. The average density in the band and out of the band is defined as d_{IB} and d_{OB}, respectively. As in Figure 1.1d, in a 'perfectly' banded matrix, there are no elements outside the band, thus $d_{OB} = 0$.

DEFINITION

A **banded matrix** is a sparse matrix which has a higher density of non-zero elements around the diagonal. A **'perfectly' banded matrix** concentrates all its non-zero elements within a



(a) *roadNet-CA* **(b)** *scircuit* **(c)** *msc10848* **(d)** *raefsky3*

Figure 1.1 – Examples of Banded Matrix Structure

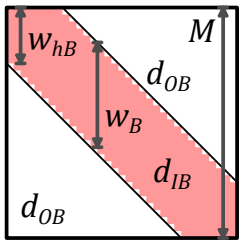


Figure 1.2 – Banded Matrix Definition

delimitated region around the diagonal. This region is called the band.

NOTE

The SuiteSparse Matrix Collection [63] is a large, open source data base of sparse matrices from multiple real-world applications, many of which were provided by companies such as Boeing. It gathers around 3,000 matrices of various sizes, densities and structure, which are extensively used for benchmarking (see Chapter 2).

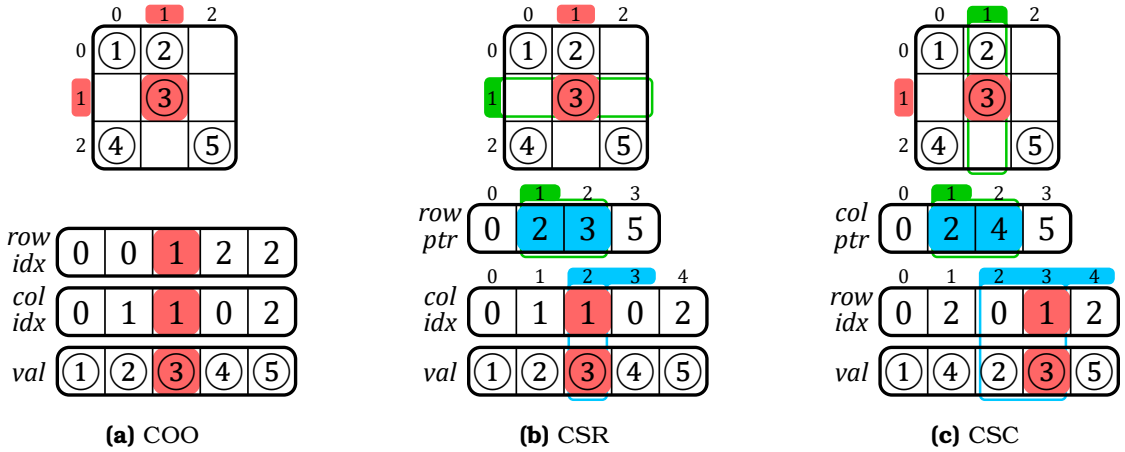
1.3 Sparse Formats

In the absence of a compressed format, matrices are stored in fixed-size two-dimensional arrays. With this dense format, elements can be randomly accessed ($\mathcal{O}(1)$), and sequential accesses are highly efficient. The two dimensions are called ranks which correspond to the rows and columns. For matrices, there are thus two variants of the dense format, depending on which rank is stored sequentially in memory. In a *row-wise* (or *row-major*) storage, the elements within the rows are sequential, and conversely for *column-wise* (or *column-major*).

Sparse matrices contain a very large number of zeros, and to avoid storing these repeated zeros, there exist many compressed formats [12, 14, 22, 60, 93, 110], where certain formats are best-suited for matrices with a specific structure. The main principle of these sparse formats is to avoid storing the zero elements, but this requires additional metadata which gives the position of the non-zero elements.

For example, let us consider the commonly used *COOrdinate* (COO) format that is relatively simple. It consists of three tables of size nnz with only the non-zero elements (val table in Figure 1.3a) and their row and column indices (row and $col\ idx$ tables). The tuples do not necessarily need to be sorted, which makes it easy to append new entries. However, if the tuples are not sorted, this format is not well suited for sequential traversal or locating a specific matrix entry. This format can be sorted *row-wise* as in the example in Figure 1.3a or *column-wise* if needed, but in this case, it is less efficient than the two other widely used sparse formats *Compressed Sparse Row* (CSR) (Figure 1.3b) and its *column-major* counterpart *Compressed Sparse Column* (CSC) (Figure 1.3c), which are sorted by construction. With CSR, the val table of size nnz stores the non-zero elements sorted *row-wise* and the idx table of size nnz stores the corresponding column indices, similarly to COO. The third table ptr of size $M + 1$ stores the positions of the rows. For example, in Figure 1.3b, the second row of index 1 (*green*) has one non-zero element between positions 2 and 3 (*blue*, 3 excluded) of tables idx and val. The CSC format works similarly but in a *column-wise* manner: the val table of size nnz is sorted *column-wise* and the idx table of size nnz stores column indices. The ptr table

of size $N+1$ stores the positions of the columns. In the example in Figure 1.3c, the second column of index 1 (*green*) has non-zero elements between positions 2 and 4 (*blue*, 4 excluded) of tables idx and val. The ptr table allows CSR and CSC to have a lower memory footprint than COO.



(a) COO **(b)** CSR **(c)** CSC

Figure 1.3 – Example of a Sparse Matrix in COO, CSR and CSC Formats

Many others classes of sparse formats exists such as *Blocked Compressed Sparse Row* (BCSR), *DIAgonal* (DIA), *ELLPACK* (ELL) and *HYBbrid* (HYB), which all have their own advantages depending on the matrix structure or the hardware setup. Additional information about these other formats is provided in Appendix A. In general, there are many derivates from the above main categories of sparse formats. Ultimately, we can create any format we want depending on the software and hardware constraints. Later in this manuscript, we propose our own custom sparse format called *Banded Compressed Sparse Column* (BaCSC) that is an extension of CSC for banded matrices.

DEFINITION

A **sparse format** is a memory representation of a sparse matrix which avoids storing zeros by using metadata.

NOTE

The most commonly used sparse formats are COO, CSR and CSC.

CONCLUSION

With the use of sparse formats, it is possible to greatly **decrease the memory footprint** of a sparse matrix while increasing the data dimensions. These formats **can be tailored** based on the needs of the software and hardware.

1.4 Algorithms for Dense and Sparse Matrix Multiplication

THE main matrix multiplication kernels that are of interest are Matrix-Vector (MV) and Matrix-Matrix (MM), which include SpMV and SpMSpV ($A \times b = c$), SpGEMV ($A \times b + d = c$), SpMM and SpMSpM ($A \times B = C$), and SpGEMM ($A \times B + D = C$), with 'Sp' for *sparse* and 'GE' for *general*. SpMV, as well as its variant SpGEMV, are by far the dominant operations in modern algebraic kernels (linear solvers, eigensolvers) which have been explicitly based on them [29,

40, 45, 56, 62, 89, 121]. Since these kernels are ubiquitous, their efficient implementation is primordial. In contrast, the SpMSpM multiplication is less frequent in standard algebraic kernels, but it is extensively used in algebraic graph manipulation [5, 17, 21, 27, 39, 59, 90], as well as in multigrid solvers [4, 16, 66, 92]. The growing importance of large graphs has motivated new research on optimizing SpMSpM applications.

In the following sections, we focus on SpMV and SpMSpM, with the notation $A(M,K) \times b(K) = c(M)$ and $A(M,K) \times B(K,N) = C(M,N)$, respectively. The rank K is the common rank between the two inputs A and B (or b). The existence of different ranks allows several algorithmic possibilities to compute matrix multiplication: the *Inner-Product* (**IP**) algorithm, the *Outer-Product* (**OP**) algorithm and *Gustavson*'s (**Gust**) algorithm [30, 43]. In all cases, the matrix A is traversed sequentially. The three algorithms, however, impose different access patterns for B (or b) and C (or c). Indeed, in many cases, they have to be accessed multiple times, as discussed in the following sections.

NOTE

In this manuscript, we focus on only two kernels, SpMV and SpMSpM. Note that, among MV kernels, SpMV is the most used and we specifically work on this kernel in Chapter **??**. SpMSpM is also widely used and studying this MM kernel is of interest as the memory access patterns in both matrices are irregular.
The *general* ('GE') version of these kernels is not fundamentally different, as the majority of the compute operations are in the matrix multiplication, so going forward, we will ignore the 'GE' variants.

1.4.0.1 Inner-Product Algorithm

The *Inner-Product* algorithm [30] is an intuitive way to calculate a matrix multiplication because it follows the loop order of the mathematical formulas (1.1) and (1.2) for MV and MM, respectively. The common rank K is the innermost loop of the algorithm (see Algorithms 1.1 and 1.2, lines 2 and 3, respectively). The output C (or c) is always updated sequentially, but the B-matrix (or b-vector) will be read entirely M times. In the context of sparse matrices, the reads of B (or b) are conditioned by the non-zero elements of A, which requires indirect accesses. If B (or b) is also sparse, the algorithm needs to perform *intersections* between the non-zero elements in the rows of A and the columns of B (or in the b-vector) (see Section 1.5). In Algorithm 1.2, exchanging M and N (lines 1 and 2) simply causes the output to be updated *column-wise*.

$$\forall i \in [\![0, M-1]\!],\ c_i = \sum_{k=0}^{K-1} A_{i,k} \times b_k \tag{1.1}$$

$$\forall (i,j) \in [\![0, M-1]\!] \times [\![0, N-1]\!],\ C_{i,j} = \sum_{k=0}^{K-1} A_{i,k} \times B_{k,j} \tag{1.2}$$

1.4.0.2 Outer-Product Algorithm

The *Outer-Product* algorithm [30] has the opposite rank loop order than the *Inner-Product*. Indeed, Algorithms 1.3 and 1.4 show that the common rank K is the outermost loop (line 1). It changes nothing mathematically except that the B-matrix (or b-vector) is always read

Algorithm 1.1: Dense Matrix-Vector *Inner-Product* Algorithm

```
1 for i ∈ [[0, M − 1]] do                    // row first
2 |   for k ∈ [[0, K − 1]] do
3 |   |   c_i += A_{i,k} × b_k
```

Algorithm 1.2: Dense Matrix-Matrix *Inner-Product* Algorithm

```
1 for i ∈ [[0, M − 1]] do                    // A-row first
2 |   for j ∈ [[0, N − 1]] do
3 |   |   for k ∈ [[0, K − 1]] do
4 |   |   |   C_{i,j} += A_{i,k} × B_{k,j}
```

sequentially. In the context of sparse matrices, the output C (or c) is updated with indirect accesses. The algorithm needs to perform *reductions* of the partial sums $A_{j,k} \times B_{k,j}$ (or $A_{j,k} \times b_k$), as we show in Section 1.5. To avoid irregularity when updating C (or c), the **OP** algorithm is often separated into two phases. First, during the *multiply* phase, we compute several *partial outputs* (*POs*) consisting of intermediate matrices (or vectors) of partial sums $A_{j,k} \times B_{k,j}$ (or $A_{j,k} \times b_k$), then during the *merge* phase, we accumulate all the *POs* to form the final C-matrix (or c-vector). With this method, **OP** necessitates the generation of at most K *POs* of densities conditioned by the non-zeros elements of both inputs A and B (or b).

Algorithm 1.3: Dense Matrix-Vector *Outer-Product* Algorithm

```
1 for k ∈ [[0, K − 1]] do                    // column first
2 |   for i ∈ [[0, M − 1]] do
3 |   |   c_i += A_{i,k} × b_k
```

Algorithm 1.4: Dense Matrix-Matrix *Outer-Product* Algorithm

```
1 for k ∈ [[0, K − 1]] do // common rank first
2 |   for i ∈ [[0, M − 1]] do
3 |   |   for j ∈ [[0, N − 1]] do
4 |   |   |   C_{i,j} += A_{i,k} × B_{k,j}
```

1.4.0.3 Gustavson's Algorithm

As the MM algorithms have three ranks instead of two for MV, there is a third loop ordering, known as *Gustavson*'s algorithm [43] where the common rank K is the middle-loop (see line 2 in Algorithm 1.5). With this approach, the output is computed row-by-row with *PO*-rows generated and accumulated for each output C-row. With this algorithm, updates of the C-rows are sequential, but elements within the rows are updated with indirect accesses. In the dense case, the B-matrix is entirely read M times and K *PO*-rows, of size N, are generated. In the context of sparse matrices, conversely to the C-rows, the selection of the rows in B is indirect, but the accesses are sequential within a row. Thus, the *intersection* search is only needed for the B-rows and not each element within rows. The *PO*-rows can be accumulated before computing the next C-row. Overall, this algorithm can be seen as a trade-off between **IP** and **OP** for MM kernels, since the indirect accesses are shared between B and C. Again, for this algorithm, exchanging M and N causes the three matrices to be accessed *column-wise* instead of *row-wise*.

Algorithm 1.5: Dense Matrix-Matrix *Gustavson*'s Algorithm

```
1 for i ∈ [[0, M − 1]] do
2 |   for k ∈ [[0, K − 1]] do    // common rank
3 |   |   for j ∈ [[0, N − 1]] do
4 |   |   |   C_{i,j} += A_{i,k} × B_{k,j}
```

1.4.0.4 Comparison of the Algorithms

In Table 1.1, we summarize the main features of **IP**, **OP** and **Gust** in terms of accesses to the matrix A, B (or b) and C (or c). From this high level analysis, we can easily see that the data that is accessed with indirections shifts depending on the algorithm, from the input B (or b) to the output C (or c), with **Gust** being a trade-off between **IP** and **OP** for MM multiplication. To have a better understanding of the consequences on the actual number of memory accesses, we propose a more detailed analysis in Section 1.5.4, for the case of SpMSpM.

NOTE

Note that these algorithms are available in dense as well as in sparse MV and MM multiplications. In the sparse variants, the above algorithms (from 1.1 to 1.5) have the same behavior from a mathematical point-of-view, but need adjustments to traverse a sparse format instead of a dense matrix. Sparse versions of these algorithms are presented in Section 1.5.

Table 1.1 – Comparison of Matrix Multiplication Algorithms

	***Inner-Product* (IP)**[1]	***Outer-Product* (OP)**[1]	***Gustavson* (Gust)**[2]
A(M,K) **Accesses**	• *row-wise* • read N times[3,4] • sequential	• *column-wise* • read once • sequential	• *row-wise* • read once • sequential
B(K,N) **Accesses**	• *column-wise* • read M times[5] • not sequential	• *row-wise*[6] • read $M \times d(A)$ times[4,7] • sequential	• *row-wise* • read $M \times d(A)$ times[7] • sequential within rows
C(M,N) **Accesses**	• generated *row-wise*[6] • one sequential traversal	• generated without specific structure • K *POs* produced[8] (each *PO* has $\overline{nnz_c}(A) \times \overline{nnz_r}(B)$ non-zero elements[9])	• generated *row-wise* • $K \times d(A)$ *PO*-rows produced[10] for each row of A (M) ($\overline{nnz_r}(B)$ non-zero elements per *PO*-row)

[1] Considering $N = 1$ in the case of MV multiplication.
[2] Only for MM multiplication.
[3] Or less than N times if B has empty columns.
[4] Read once in the case of MV multiplication.
[5] If B is stored with a sorted sparse format, else it would be read $\overline{nnz_r}(A) \times M = nnz(A)$ times.
[6] *Element-wise* in the case of MV multiplication (there is no row).
[7] Which is equal to $\overline{nnz_c}(A)$, the average number of non-zero elements in the columns of A.
[8] Or less than K times if A has empty columns or B has empty rows.
[9] $\overline{nnz_c}(A)$ in the case of MV multiplication.
[10] Which is equal to $\overline{nnz_r}(A)$, the average number of non-zero elements in the rows of A.

CONCLUSION

There are *three main algorithms* to compute sparse MV and MM kernels, ***Inner-Product* (IP)**, ***Outer-Product* (OP)** and ***Gustavson* (Gust)**. Each presents different characteristics, the main one being the **irregularity of the accesses**: on the **input** matrix B (or vector b) with **IP**; on the **output** matrix C (or vector c) with **OP**; on both with **Gust** but at **row scale**.

1.4.1 Tiling

The *tiling* process [41] consists in partitioning a matrix: a matrix may be divided into *tiles*, non-overlapping sub-matrices. In other words, this method consists of adding ranks to delimit the *tiles*, and thus adding loop iterations in the kernel algorithm.

Tiling methods make it possible to process smaller portions of the matrix and thus to adapt the working set to the size of the local memory [64, 116], particularly when the matrix has a structure with denser regions. *Tiling* can also increase parallelism opportunities. Software optimizations for sparse matrix computation, such as OSKI [111] and Sparse BLAS [29], are based on *tiling*. In previous sections, the study of the `IP`, `OP` and `Gust` algorithms highlighted the influence of rank ordering on the sequentiality and data movement, and thus, adding intermediate ranks with *tiling* breaks the regularity of pure `IP` or `OP` algorithms, requiring additional hardware support (see Section 1.5).

For example, let us consider the A-matrix in Figure 1.4 which is divided into four *tiles* (one level of *tiling*). For a SpMV, two new ranks appear for a total of four ranks, as shown in Algorithm 1.6: M_1 and K_1 allow *tile* selection and M_2 and K_2 allow data access within a *tile*. Thus, we could decide to apply an `OP` on *tiles* and conversely an `IP` within *tiles*, mixing-up the pros and cons of these two algorithms at different rank levels. In the case of the example in Figure 1.4, the upper-left A-*tile* (*red*) is evaluated first, sequentially updating the upper *tile* of the c-vector in an `IP` manner. The lower-left A-*tile* (*yellow*) is then computed the same way. At this point, *intersections* have been done with only the upper b-*tile*, and c has been updated sequentially (first *PO*). The two next A-*tiles* (*green*, then *blue*) will compute *intersections* with the lower b-*tile* and update the entire c-vector sequentially, showing the `OP` characteristic of having *POs* to *merge*, two in this case.

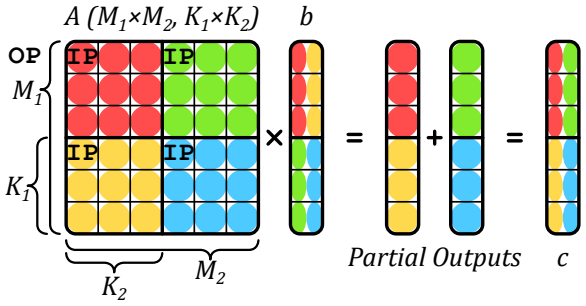


Figure 1.4 – An Example of *Tiling* on SpMV

Algorithm 1.6: Matrix-Vector Algorithm with Custom *Tiling* of Figure 1.4

```
1 for k1 ∈ [[0, K1 − 1]] do
2     for i1 ∈ [[0, M1 − 1]] do
3         for i2 ∈ [[0, M2 − 1]] do
4             for k2 ∈ [[0, K2 − 1]] do
5                 i ← i1 × M2 + i2
6                 k ← k1 × K2 + k2
7                 ci += Ai,k × bk
```

DEFINITION

Tiling is a method for partitioning matrices in non-overlapping sub-matrices, called ***tiles***.

NOTE

Note that in case of *tiling*, the A-matrix is no longer read sequentially, which can be problematic if it is stored in a classical sparse format like CSR that is specifically made for *row-wise* accesses. Custom sparse formats adapted for *tiling* can be used [52], but this requires preprocessing to perform the conversion.

CONCLUSION

Tiling methods are widely used to **distribute workloads** across multicore systems, and to **locally reduce the data dimensions**. Complex *tiling* algorithms with many possible rank ordering **inherit the behaviors of `IP` and `OP`**, and can be described as a combinations of both at different *tile* levels.

1.5 Hardware Challenges for Sparse Matrix Computation

In the previous section we presented a brief background on matrix multiplication without considering the underlying hardware. Although the various sparse formats reduce the memory footprint, they have the disadvantage that indirect accesses are required, which creates data-dependencies and reduces the effectiveness of caches. As we have seen, with all three algorithms, at least some of the accesses are irregular. The lack of spatial locality for these accesses reduces the benefit of having caches [79, 119]. In other words, a full cache-line is loaded, but only a few entries are read or modified. Moreover, the matrices of interest are extremely large and even the non-zero elements, and associated metadata, can not generally fit in the cache. Depending on the algorithm, successive accesses to the same element may be too far apart to benefit from temporal locality in the cache.

1.5.1 Inner-Product Algorithm

In an **IP** algorithm, the A-metadata needs to be read to indirectly access the B-matrix (or b-vector), which may itself be stored in a sparse format. In this case, the algorithm needs to check if the non-zero A-indices are present in B (or b). If an index appears in neither list, there is no need to perform the multiplication (ineffectual operation). More generally, this problem applies to any *fiber* and is called the *intersection* search. As a minimum, this requires reading all the metadata for A and B (or b), and identifying those present in both.

In Algorithm 1.7, we present the pseudo-code for an *Inner-Product* (**IP**) SpMSpM with A and B stored in CSR and CSC formats, respectively. The outer-loop (M, line 1) iterates over the rows of A. The middle-loop (N, line 2) iterates over the columns of B. The inner-loop (line 4) iterates over the non-zero elements of the current row (i) of A, and for each index (k_A of position pos_A in the idx array of A, line 5), a search is performed in the metadata of B (line 6), to see if the corresponding index in B is present. Assuming it is ($isMatch$ is true, line 7), the partial sum is computed and then accumulated locally (acc, line 8). This example highlights the need to efficiently search for the *intersection* of indices in the metadata of the inputs.

Different algorithms can be used to address the *intersection* search, depending on whether the elements are sorted. When the elements are sorted, the *intersection* becomes a *merge* and generally requires at most $2 \times nnz - 1$ comparisons (in some cases the number of comparisons can be reduced [8]). In Algorithm 1.7, we use sorted sparse formats (CSR and CSC). Thus, the naïve *intersection* search proposed (line 10) does at most one traversal of each the current row of A (i) and the current column of B (j). Note that computing an *intersection* in software induces the use of conditional loops and branches, which can be costly for classical CPU branch predictors. In case the matrix B (or vector b) is stored in a dense format, the *intersection* search is simplified, since all indices are implicitly present, but the accesses to B (or b) remain indirect. As we see in Chapter 2, certain accelerators provide support for performing the *intersection* operation directly in hardware.

DEFINITION

The ***intersection* search** consists in founding non-zero elements of two sparse arrays that have matching indices. This search is costly in software on classical CPUs, thus being key to improve performance of **IP** algorithm.

Algorithm 1.7: `IP` SpMSpM Performing *Intersection* Searches with CSR and CSC Formats

Data: $A(M, K)$ in CSR: $A.val, A.idx, A.ptr$
Data: $B(K, N)$ in CSC: $B.val, B.idx, B.ptr$
Data: $C(M, N)$ in dense format (for readability)
Result: $C = A \times B$

```
1  for i ∈ [0, M − 1] do                                   // Iterations over the rows of A
2      for j ∈ [0, N − 1] do                               // Iterations over the columns of B
3          acc ← 0; pos_B ← B.ptr_j;
4          for pos_A ∈ [A.ptr_i, A.ptr_{i+1} − 1] do
               // Iterations over the non-zero elements of the i-th row of A
5              k_A ← A.idx_{pos_A};
               // Column-index of the (i, pos_A) non-zero element of A
6              (isMatch, pos_B) ← IntersectionSearch(k_A, j, pos_B, B.idx, B.ptr);
               // Return true and the matching pos_B if idx exists in the j-th column of B
7              if isMatch then                             // If indices did match
8                  acc ← acc + A.val_{pos_A} × B.val_{pos_B};
                   // Local partial sum accumulation
9          C_{i,j} ← acc;                  // Write in memory of the resulting (i, j) element of C
10 function IntersectionSearch(k_A, j, pos_B, B.idx, B.ptr) is
       // Naive intersection search allowing the i-th row of A and the j-th column of B to
          be traversed only once at iteration (i, j)
11     while B.idx_{pos_B} < k_A and pos_B < B.ptr_{j+1} do
12         pos_B ← pos_B + 1;
13     if B.idx_{pos_B} = k_A then                  // Returns true if indices from A and B match
14         return (true, pos_B);
15     return (false, pos_B);
```

1.5.2 Outer-Product Algorithm

In an `OP` algorithm, the inputs A and B (or b) are accessed sequentially, but the updates of the output C (or c) require indirect accesses. The pattern of the accesses, while the output is generated, is irregular and determined by the structure of the inputs. Furthermore, if these "random" updates provoke cache misses where only one word within a cache-line is modified, the bandwidth to external memory is poorly utilized. The `OP` algorithm generates several *partial outputs* (*POs*) that must be accumulated to obtain the final output. We can identify two strategies: *immediate update* and *deferred update*. In the former, the final output is immediately updated as each partial sum is generated. If the output is stored using a sparse format, either the index being updated does not yet exist, and the new partial sum must be inserted. Or, if it exists, a lookup must be done to locate it, the update performed and the result written back. The problem associated with combining the *POs* is called *reduction*.

In Algorithm 1.8, we present two pseudo-code for an *Outer-Product* (`OP`) SpMSpM using *immediate update* (from line 1) and *deferred update* (from line 7) with A and B stored in CSC and CSR formats, respectively. The outer-loop (K, line 1 or 7) iterates over the columns of A and the rows of B (common rank). The middle-loop (line 2 or 9) iterates over the non-zero elements of the current column (k) of A. The inner-loop (line 4 or 11) iterates over the non-zero elements of the current row (k) of B. Because we only iterate over the non-zero values, there is no need for an *intersection* search. The partial sum is then computed (line 6 or 15). Finally, in the *immediate update* version, the partial sum must be accumulated to C (line 6). These accesses are irregular, making it necessary to lookup the value in C, update it, and write it

back. This is a *reduction* operation, performed on irregular data.

To avoid the complexities of these "random" *reductions* with *immediate updates*, the *deferred update* version postpones the accumulation and simply stores the partial sums in memory. One such technique is called *output batching* [61]. With this technique, arrays containing the indices and the partial sum are streamed sequentially to memory (PO_k, lines 13-15). Then, during a second pass (line 17), they are read back and the accumulations are performed, an approach which results in sequential memory accesses during both the first and second passes, with the downside that the volume of data transferred to memory is increased. As we see in Chapter 2, hardware accelerators may combine the *immediate* and *deferred update* techniques to optimize memory bandwidth.

DEFINITION

"Random" *reductions* occur when updating data from memory at irregular addresses, which happen on *immediate update* `OP`. With *deferred update* `OP`, the *reductions* are sequential but large sets of data need to be buffered in memory. Optimizing *reduction* operations is key to improving the performance on `OP` algorithm.

1.5.3 Summary

In Table 1.2, we summarize the advantages and hardware challenges for each algorithm, including `Gust` which presents a trade-off between `IP` and `OP`. The potential for parallelism and data-reuse varies across algorithms, `IP` being the most limited unless *tiling* methods are used. With `OP`, *POs* can easily be generated across a multicore system but at the cost of transferring a large volume of data to memory. With *immediate updates*, `OP` transfers less data but all accesses on the output are "random" *reductions*.

The issues presented with `IP` and `OP` can be generalized to two concepts: *gather* and *scatter*. The *gather* problem exists when the inputs are indirectly accessed, and need to be read from memory using metadata to present the processor with the correct terms for computing the partial sums. Typically, with `IP`, the *gather* problem is more difficult. The *scatter* problem exists when the order in which the output matrix/vector is generated is not sequential and is typically associated with the `OP` algorithm. In the case of *tiling* or with `Gust`, both *gather* and *scatter* must be performed creating a need for efficient *intersection* searches and random *reductions*.

CONCLUSION

There are **two main challenges** to address when computing sparse matrices. When reading irregular data from memory, addressing the ***gather*** problem is key. It consists of processing efficient ***intersection* searches**. When updating irregular data from memory, addressing the ***scatter*** problem is key. It consists of **managing *POs*** in memory or processing efficient **"random" *reductions***. These two challenges are tied to `IP` an `OP`, respectively, and are present in any higher rank algorithms for sparse matrix computing.

1.5.4 SpMSpM Algorithm Analysis

To better understand how the algorithms impact the number and types of memory accesses, we developed, and presented in Figure 1.5, a model based on memory accesses counts for SpMSpM. We identify five memory access patterns: (1) those where a matrix is read only

Algorithm 1.8: OP SpMSpM Performing *Reduction* with CSR and CSC Formats

Data: $A(M, K)$ in CSR: $A.val, A.idx, A.ptr$
Data: $B(K, N)$ in CSC: $B.val, B.idx, B.ptr$
Data: $PO_k(M, N)$ for all $k \in [\![0, K-1]\!]$ in COO: $PO_k.val, PO_k.row, PO_k.col$
Data: $C(M, N)$ in dense format (for readability)
Result: $C = A \times B$

```
// * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * *
// * * * 'Immediate updates' version * * * * * * * * * * * * * * * * * * * * * * *
1  for k ∈ [[0, K−1]] do                          // Iterations over the columns of A and the rows of B
2      for pos_A ∈ [[A.ptr_k, A.ptr_{k+1}]] do
           // Iterations over the non-zero elements of the k-th column of A
3          i_A ← A.idx_{pos_A};                   // Row-index of the (pos_A, k) non-zero element of A
4          for pos_B ∈ [[B.ptr_k, B.ptr_{k+1}]] do
               // Iterations over the non-zero elements of the k-th row of B
5              j_B ← B.idx_{pos_B};
               // Column-index of the (k, pos_B) non-zero element of B
6              C_{i_A,j_B} ← C_{i_A,j_B} + A.val_{pos_A} × B.val_{pos_B};
               // Reduction (to memory) of the partial sum into the (i_A, j_B) element of C
                  (immediate update)
// * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * *
// * * * 'Deferred updates' version  * * * * * * * * * * * * * * * * * * * * * * *
// Multiply phase
7  for k ∈ [[0, K−1]] do                          // Iterations over the columns of A and the rows of B
8      pos_PO ← 0;
9      for pos_A ∈ [[A.ptr_k, A.ptr_{k+1}]] do
           // Iterations over the non-zero elements of the k-th column of A
10         i_A ← A.idx_{pos_A};                   // Row-index of the (pos_A, k) non-zero element of A
11         for pos_B ∈ [[B.ptr_k, B.ptr_{k+1}]] do
               // Iterations over the non-zero elements of the k-th row of B
12             j_B ← B.idx_{pos_B};
               // Column-index of the (k, pos_B) non-zero element of B
13             PO_k.row_{pos_PO} ← i_A;
14             PO_k.col_{pos_PO} ← j_B;
15             PO_k.val_{pos_PO} ← A.val_{pos_A} × B.val_{pos_B};
               // The k-th partial sum of the (i_A, j_B) element of C is stored in memory in the
                  k-th partial output
16             pos_PO ← pos_PO + 1;
       // Each partial output is generated sorted, row-wise
// Merge phase
17 for k ∈ [[0, K−2]] do                          // Naive serial merge of the K partial outputs
18     PO_{k+1} ← 2DListsMerge(PO_k, PO_{k+1});
// PO_{K-1} is C stored in COO format
19 for pos ∈ [[0, length(PO_{K−1}) − 1]] do
       // Update C (dense) from the fully merged partial outputs
20     i ← PO_{K−1}.row_{pos};
21     j ← PO_{K−1}.col_{pos};
22     C_{i,j} ← PO_{K−1}.val_{pos};                // Write in memory of the (i, j) element of C
```

Table 1.2 – Comparison of Advantages and Challenges with Matrix Multiplication Algorithms

		***Inner-Product* (IP)**	***Outer-Product* (OP)**	***Gustavson* (Gust)**
Access Count	**MV**	***A*:** once ***b*:** $nnz(A)$ ($\sim M\times$) ***c*:** once	***A*:** once ***b*:** once ***c*:** $nnz(A)$ ($\sim K\times$)	*N/A*
	MM	***A*:** $N\times$ ***B*:** $M\times$ ***C*:** once	***A*:** once ***B*:** $M \times d(A)\times$ ***C*:** $nnz(A) \times \overline{nnz_r}(B)$ ($\sim K\times$)	***A*:** once ***B*:** $M \times d(A)\times$ ***C*:** $nnz(A) \times \overline{nnz_r}(B)$
Sequential Accesses		Accesses to A and C	Accesses to A and B	Accesses to A and C Accesses to B-rows
Input Format		Accesses to A and B benefit from alternate formats (CSR and CSC)		Allows consistent format for A and B
Parallelism Opportunity		Possible with *tiling*	Generation of *POs* over common rank of A and B	Reading A-rows and updating C-rows
Reuse Opportunity		Potentially B, but limited by its size	An A-column or B-row while computing *POs*	A set of B-rows, based on non-zero elements of A-rows
Challenges		*Intersection* search between non-zero elements in A and B	Storage for *POs* or in-place *merge* and *reduce*	*Merge* and *reduce* of *PO*-rows

once, thus there is no opportunity for reuse, (2) where the accesses are sequential, but a *fiber* is read multiple times, (3) where *fibers* are read consecutively, but the accesses within a *fiber* are "random", (4) where the *fibers* are accessed "randomly", but within a *fiber* the accesses are sequential, and (5) where the matrix elements are accessed multiple times, and with a "random" access pattern.

For each of the three algorithms, we counted the number of read and write accesses, based on the density of the input matrices, and the local memory storage available ("0D", "1D" and "2D"), as presented in Table 1.3. We refer to a system with no local storage or cache as '0 Dimensional' ("0D"), a system which has local storage that is sufficient to hold a single row or column as '1 Dimensional' ("1D"), and a system which can cache an entire matrix as '2 Dimensional' ("2D"). In Figure 1.5, we plotted the number of memory accesses for SpMSpM, in a logarithmic scale, based on a square matrix with $M = 10,000$. Indeed, the number of non-zero elements in C depends on the structure of the input matrices and in this model, we have assumed the non-zero entries in the input matrices are uniformly, randomly distributed. This corresponds to a *worst case* for sparse matrices (no spatial and temporal locality). If A or B are banded matrices, for example, there would be fewer non-zero elements in C. In each graph, we have separated the accesses into A (*bottom*), B (*middle*) and C (*top*). The color code shows the access pattern, based on the five categories described above.

In the top-row of Figure 1.5 (graphs ①, ② and ③), we start by considering a naïve system which has no local storage ("0D"), thus all data comes from main memory. From the graphs in this row, we clearly see that **IP** requires more accesses at low density, as the A-matrix must be read N times (graph ①). Whereas for **OP** and **Gust**, the A-matrix is read only once (thus the *grey* color). For **OP** and **Gust** the total number of accesses is the same, however, with **OP** (graph ②), the accesses to C are "random" (*red*). With **Gust** (graph ③), the accesses to B are sequential within the rows (*orange*) and the C-rows are generated sequentially (*yellow*), thus **Gust** avoids having any fully "random" accesses (*red*).

Since main memory accesses are the principal bottleneck, systems integrate local storage

Table 1.3 – Memory Access Count for Each Matrices of `IP`, `OP` and `Gust` SpMSpM

	Inner-Product (`IP`)	***Outer-Product*** (`OP`)	***Gustavson*** (`Gust`)
$\mathbf{A}(M,K)$	**0D**: $N \times nnz(A)$ **1D-2D**: $nnz(A)$	**0D-1D-2D**: $nnz(A)$	**0D-1D-2D**: $nnz(A)$
$\mathbf{B}(K,N)$	**0D-1D**: $M \times nnz(B)$ **2D**: $nnz(B)$	**0D**: $M \times d(A) \times nnz(B)$ **1D-2D**: $nnz(B)$	**0D-1D**: $M \times d(A) \times nnz(B)$ **2D**: $nnz(B)$
$\mathbf{C}(M,N)$	**0D-1D-2D**: $nnz(C)$ (max: $M \times N$)[1]	**0D-1D**: $2 \times M \times d(A) \times nnz(B)$ **2D**: $nnz(C)$ (max: $M \times N$)[1]	**0D**: $2 \times M \times d(A) \times nnz(B)$ **1D-2D**: $nnz(C)$ (max: $M \times N$)[1]

[1] We can not easily quantify $nnz(C)$ knowing $nnz(A)$ and $nnz(B)$. We thus use a statistic approximation.

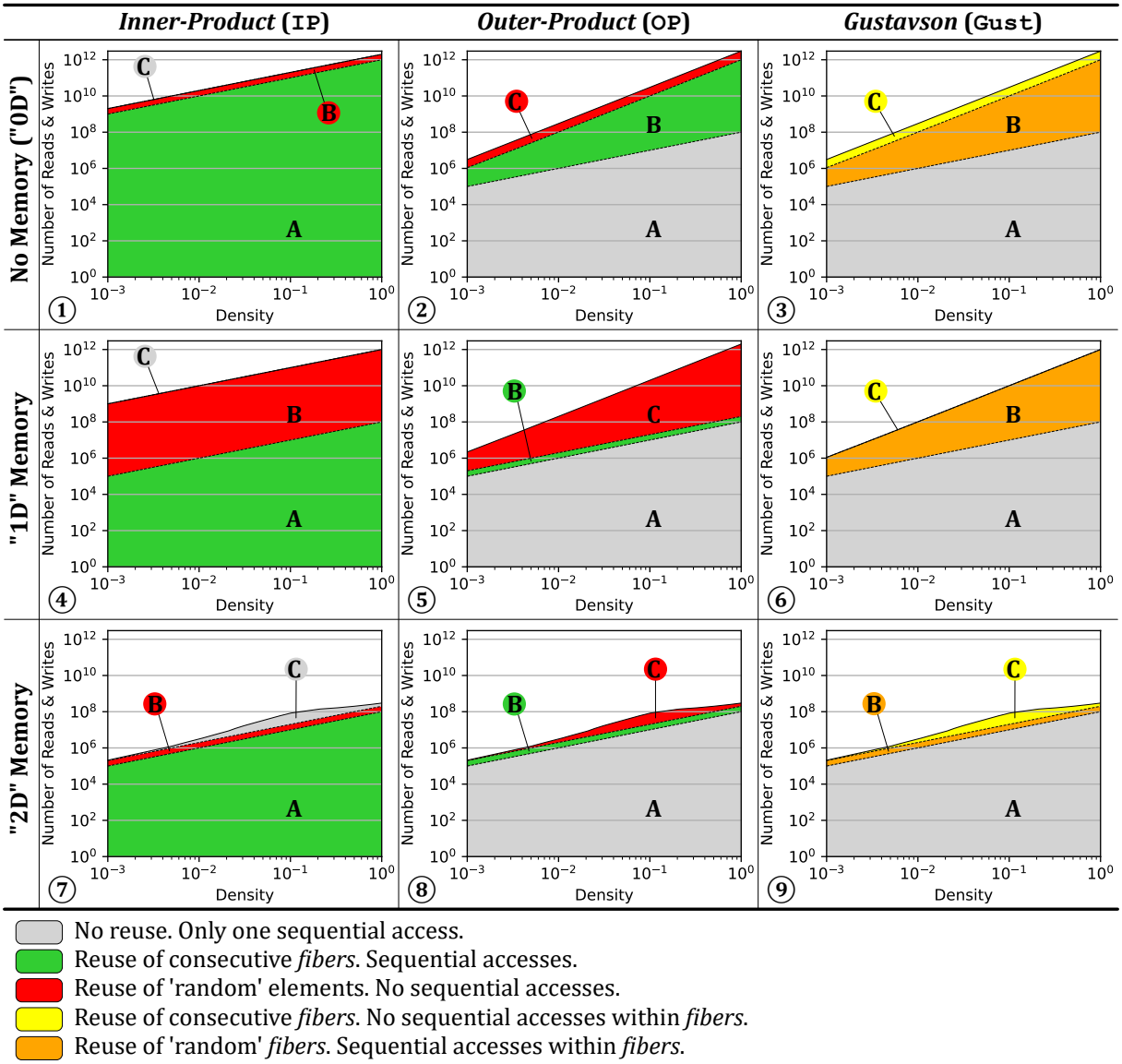


Figure 1.5 – Model of Memory Access Counts Depending on Algorithm and Local Memory Size

(such as buffers or caches). Thus, we consider the case of a system with sufficient local memory to store one full row, per matrix (equivalent to few sparse rows, "1D"). We assume this local storage is perfect (no misses) and we re-calculate the number of remaining main memory accesses, as shown in the middle-row of Figure 1.5 (graphs ④, ⑤ and ⑥). The matrices are assumed to be initially stored in main memory, thus they must be accessed at least once. As can be seen, for **IP** (graph ④), the number of A-accesses is reduced, because rows are reused, thus the total number of A-accesses becomes equal to that in **OP** and **Gust**. For **OP** (graph ⑤), the single row buffer greatly reduces the number of B-accesses, however, it does nothing for C (despite the appearance, due to the logarithmic scale). For **Gust** (graph ⑥), the single row buffer is sufficient to cache the *reduction* operations in C, and thus, it now has fewer overall accesses than **OP**. It is also interesting to note that **OP** and **Gust** scale better than **IP** with lower densities, as seen by the shallower slope.

Finally, we have considered the extreme case where an system has a buffer of sufficient size to store an entire matrix ("2D"), which is shown in the bottom-row of Figure 1.5 (graphs ⑦, ⑧ and ⑨). Although this case is not practical for large matrices, through the proper use of *tiling* to locally reduce the dimensions of a sub-matrix, this case can be achieved at the level of a *tile*. In this case, the total number of external accesses is equal for the three algorithms, as each matrix is accessed only once from main memory. For **IP** (graph ⑦), it is interesting to note that such a large buffer is needed to locally address the *intersection* search in B. Similarly for **OP** (graph ⑧), a large storage is required to address the *reductions* in C. For **Gust**, in fact, it is not necessary to buffer the entire B-matrix. Indeed, buffering a few rows (corresponding to $nnz_r(A)$) is sufficient to achieve the number of accesses shown in graph ⑨. This corresponds to an intermediate design point (multiple row buffers for B) not explicitly shown in the figure (between graphs ⑥ and ⑨).

When viewed overall, Figure 1.5 highlights the benefits of **Gust**. We clearly see that **Gust** does not require any fully "random" accesses (*red*). With a single row buffer, it is possible to address the *reductions* in C. And, with a limited number of row buffers, the number of external memory accesses for B can be reduced. Based on this analysis, it is clear that, among the three base algorithms, **Gust** is the best candidate for hardware implementation of SpMSpM.

NOTE

For SpMV, such an analysis does not show significant differences between **IP** and **OP**, as both algorithms require similar number of memory accesses. However, **OP** shows more potential for parallelism and data-reuse as seen in the previous section (1.5.3), thus being preferred for SpMV hardware accelerators.

CONCLUSION

Based on a high level analysis of the memory accesses on SpMSpM, ***Gustavson*'s algorithm** appears as the **best trade-off** to minimize memory traffic with reasonable amount of local storage. The second best option is **OP**, which requires fewer memory accesses than **IP** when the matrices have a low density.

1.6 Conclusion

In this chapter, we established the foundation for understanding the relationship between applications working with irregular data and the underlying challenges in hardware. Irregular accesses are common in sparse computing and cause reduced cache efficiency, limited parallelism, and higher memory latency exposure. We analysed the most representative algorithms for sparse matrix multiplication kernels, and identified the operation that provoke irregular accesses. *Intersections* and *reductions* on irregular data with classical systems are costly, thus they represent the central challenge for efficient computation on sparse data. To improve the performance of these operations, specialized hardware has been proposed as will be seen in the next chapter.

TAKEAWAYS

Sparse matrices: large, highly sparse, structured (such as banded matrices)
Sparse formats: diverse, reduce memory footprint

- COO, the simplest
- CSR and CSC, the mostly used
- BaCSC, our own sparse format for banded matrices

Sparse kernels: sparse MV and MM multiplications (such as SpMV and SpMSpM), generate indirect accesses
Algorithms: *Inner-Product* (`IP`), *Outer-Product* (`OP`) and *Gustavson* (`Gust`), extended with *tiling* methods

- Best potential to reduce memory transfers: `Gust`, followed by `OP` at low density

Problem: irregular accesses, generate high memory transaction latencies
Challenges:

- *gather*: *intersection* searches
- *scatter*: "random" *reductions*, or storing and *merging partial outputs* (*POs*)

Chapter 2

State-of-the-Art

Contents

2.1 Introduction

From SoA paper

It was recognized early in the 1960s [93] that working on these sparse data sets using dense array representations was inefficient. The seminal work of OSKI [111] in 2003 formalized the sparse representation alternatives and identified many optimizations. This opened the path for specialized software libraries for sparse algebra [117]. However, the single-core performance of software implementations can be deceiving, often only reaching 10% of peak performance [36].

The two matrix operations, SpMV and SpMSpM, pose orthogonal computational challenges: (1) access to sparse input data, (2) *reduction* operations on the intermediate result and (3) formatting and writing back the result to memory. The different algorithms, which are

detailed in Section **??**, are built on different articulations of these three aspects. The third aspect, *i.e.* result storage, is sometimes overlooked when the output is dense and not too large. For example, when running on a conventional CPU which is memory-bound, it becomes crucial to streamline the write accesses in order to optimize cache usage when writing a large dense vector output. The second aspect, *i.e.* the *reduction* operations, deserves different solutions according to the execution platform. On a conventional CPU, the bottleneck is memory throughput. Thus, most software implementations of SpMV deal with *a priori* reorganization of the input matrices, by row ordering, blocking and loop optimization, for improving the data locality of operations. The perspective for SpMV is different when dealing with GPU platforms. The challenge shifts to finding a balanced assignment of the numerous threads and warps, and to the use of local memory resources [71]. SpMSpM is far more complex: the intermediate *reductions* introduce new NNZ values at random locations, which trigger costly memory allocations and list management operations [37]. As a consequence, this phase dominates the execution time and becomes the main memory bottleneck on a regular CPU. The issue is similar on GPUs because of the limited size of the scratchpad memory for processing large matrices, which requires complex memory management and load balancing [26, 85, 124].

The main motivation to use custom hardware accelerators is to make better use of the intermediate memory hierarchy, ensuring the input data is available near the cores and also off-loading the accumulation and *reduction* operations from the processors.

The focus of this paper is to review these architectures, to compare them and to analyze their performance. Our main focus is on accelerators that eventually target custom or ASIC implementations, although we do touch on the contributions from accelerators that were prototyped on FPGAs. The reader is referred to [80] for a survey of acceleration techniques for CPUs, GPUs and FPGAs. Section **??** reviews the storage schemes and the three basic algorithms with a focus on the data access patterns and their impact on the memory hierarchy. In Section **??** we study how the different algorithms stress the memory systems and propose an analytical model based on these insights. In Section 2.2 we present the operation of several state-of-the-art hardware accelerators, in the light of these hardware challenges. In Section 2.3, we systematically compare these accelerators. Throughout this study, we try to be consistent, and we hope that our nomenclature will be useful for architects and designers who intend to implement dedicated hardware for problems with sparse datasets.

2.2 Hardware Accelerators for Sparse Matrices and Vectors

In this section, we describe the recent (2018-2023) hardware accelerators for sparse MM and MV, targeting ASIC-type implementation. We try to be consistent on notations and architecture schemes to simplify the comparison. Before going into the detailed overviews of the accelerators, in Table 2.1 we present a criteria-based classification of the designs.

A first criterion to classify the accelerators is between *standalone* accelerators that address the full kernel without the assist of a processor (*e.g.* OuterSPACE [87] – Section 2.2.1) and ones that address only a part of the kernel and need a processor to manage the computation (*e.g.* PHI [82] – Section 2.2.9). *Standalone* accelerators are directly connected to the main memory and internally manage their own custom memories. The other accelerators actually intervene at different levels of the memory hierarchy, giving a second classification criterion. Certain accelerators intervene near the processors (*e.g.* ASA [126] – Section 2.2.8), near the cache (*e.g.* PHI and SortCache [103] – Sections 2.2.9 and 2.2.7), or near the main memory

(*e.g.* SPiDRE [11] and SpaceA [120] – Section 2.2.9).

Each accelerator is optimized for a specific algorithm (**IP**, **OP** or **Gust**), our third criterion. For example, some accelerators (*e.g.* PHI and SortCache) address the *reduction* problem and are adapted to **OP** algorithms but *reductions* are also required with **Gust** and highly-*tiled* algorithms. ExTensor [46] (Section 2.2.2) is a general purpose accelerator that is demonstrated for a highly-*tiled* **OP** but also makes major contributions for **IP**.

From a memory perspective, the two main problems are *gather* and *scatter*, our fourth criterion. Certain accelerators offer solutions for one, the other or both. For example, some accelerators (*e.g.* SpArch [131] – Section 2.2.3) are specialized for an **OP** algorithm but optimize both *gather* and *scatter*. Finally, accelerators preferably focus on sparse MM, MV or both – our final criterion.

After this first comparison, in Sections 2.2.1 to 2.2.6 we present the *standalone* accelerators, followed by the *processor assist* accelerators in Sections 2.2.7 to 2.2.8. In Section 2.2.9, we briefly touch on other accelerators.

2.2.1 OuterSPACE

OuterSPACE [87] is an **OP** accelerator with a focus on SpMSpM kernels, although it can also handle SpMV. It is a highly parallelized architecture which employs Single-Program Multiple-Data (SPMD)-style *processing elements* (*PEs*) that fetch data from main memory through a two-level highly-banked cache hierarchy as shown in Figure 2.1. The input A- and B-matrices are stored in *(UOP, CP) column-* and *row-major* formats, respectively, to match the read access patterns of the **OP** algorithm. The *partial* and final outputs are stored in *(UOP, CP)* format.

The computation is temporally divided into the *multiply* and *merge* phases (Figure 2.2). During the first phase, inputs are divided into *tiles* of several rows (A) and columns (B) and distributed through the *processing tiles* (*PTs*) by a *control unit*. Within each *PT*, each *PE* processes multiplications of one element of the k^{th} A-column with all elements of the k^{th} B-row and an L0 cache in the *PT* facilitates the reuse of the B-rows between *PEs*. This phase generates a *partial output* (*PO*) per *PT*, corresponding to the *tile* of rows (A) and columns (B) that it was assigned.

Then, during the *merge* phase, the *POs* are combined to generate the final C-matrix. Each *PT* locally sorts the *POs*, and then the *PTs* work together to merge them. This algorithm was chosen to maximize data locality and parallelism, although it is less efficient. This *merge* phase only uses half of the *PEs* so the others are disabled (clock-gated) and the caches are reconfigured in a scratchpad mode to save energy.

Note, if NNZ elements in the *POs* exceeds the L0 cache capacity, it overflows to the last level cache, which results in reduced performance. OuterSPACE performs best when the $NNZ_c(A)$ and $NNZ_r(B)$ are both uniform which ensures a uniform distribution of work across the large number of *PTs* and *PEs*.

2.2.2 ExTensor

The authors created ExTensor [46] as a general purpose sparse tensor algebra accelerator (including SpMSpM). It is an **OP** accelerator but due to its support for three levels of fixed *tiling* of the inputs, it optimizes the *intersection* search between non-zero elements or *tiles*[1]. By doing the *intersections* ahead of time, at different levels of the memory hierarchy (which

1. Tensauraus [106] is another accelerator designed for tensors, but their focus is on SpMM.

Table 2.1 – Algorithmic and Architectural Classification of Existing Accelerators

		Autonomy		Position in Memory Hierarchy			Most Adapted Algorithm(s)			Hardware Support for		Sparse Kernels Addressed	
Name	**Year**	***Standalone***	***Processor Assist***	***Near Processors***	***Near Caches***	***Near Main Memory***	***Inner-Product***	***Outer-Product***	***Gustavson***	***Gather***	***Scatter***	***MV***	***MM***
OuterSPACE [87]	2018	✓				★[1]		✓		✓	✓	✓	✓
SDE [49]	2019	✓				★[1]	✓		★[3]	✓	★[2]	✓	★[3]
ExTensor [46]	2019	✓				★[1]	✓	★[2]		✓	★[2]	★[3]	✓
PHI [82]	2019		✓		✓			✓	★[3]		✓	✓	★[3]
SPiDRE [11]	2019		✓			✓	✓			✓		✓	★[3]
MatRaptor [105]	2020	✓				★[1]			✓	✓	✓		✓
SpArch [131]	2020	✓				★[1]		✓		✓	✓		✓
SpaceA [120]	2021	✓				✓	✓			✓	✓	✓	
Gamma [127]	2021	✓				★[1]			✓	✓	★[2]		✓
InnerSP [7]	2021	✓				★[1]			✓	✓	✓		✓
SortCache [103]	2021		✓		✓			✓	★[3]		✓	★[3]	✓
Sextans [100]	2022	✓				★[1]		✓		✓	✓		✓
ASA [126]	2022		✓	✓				★[3]	✓		✓	★[3]	✓
Spada [70]	2023	✓				★[1]			★[4]	✓	✓		✓
Flexagon [81]	2023	✓				★[1]	✓	✓	✓	✓	✓		✓
MOSCON [35]	2023	✓				★[1]		✓		✓	✓		✓
GSCSp [68]	2023	✓				★[1]			✓	✓	✓		✓
Funnel [77]	2024	✓				★[1]	✓		✓	✓	✓		✓
GAS [130]	2024	✓				✓		✓			✓	✓	✓
ACES [74]	2024	✓				★[1]			✓	✓	✓		✓
FullSparse [123]	2024	✓				★[1]		✓		✓	✓		✓
Sparm [76]	2024	✓				★[1]	✓	✓	✓	✓	✓		✓
SpDCache [II]	2024		✓		✓			✓	★[3]		✓	✓	★[3]
SPADES [51]	2024	✓				★[1]	★[3]	★[3]	★[3]	✓		✓	★[3]
DySpMM [114]	2024	✓				★[1]	✓			✓	✓		✓
Vesper [58]	2024	✓				★[1]	✓	✓	✓	✓	✓	✓	✓
Mentor [73]	2024	✓				★[1]			✓	✓	✓		✓
IOPS [107]	2025	✓				★[1]	✓	✓		✓	✓		✓
Leda [122]	2025	✓				★[1]		✓		✓	✓		✓
DMSA [84]	2025	✓				★[1]			★[4]	✓	✓		✓

★[1] *Most standalone accelerators connect directly to main memory.*
★[2] *Not main contribution but addressed in the article.*
★[3] *Could be adapted but not discussed in the article.*
★[4] *New complex algorithm inspired by.*

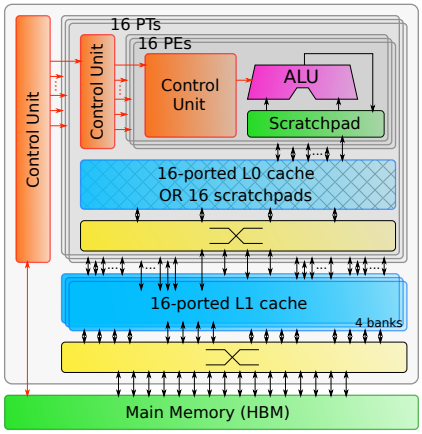


Figure 2.1 – Architecture of OuterSPACE

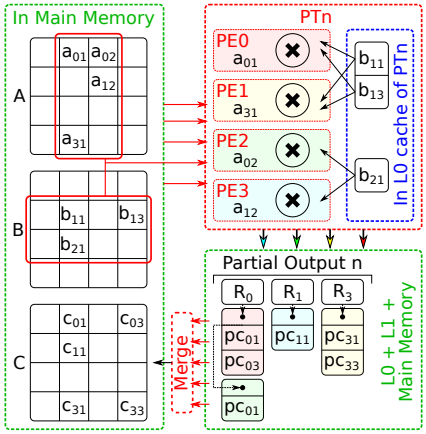


Figure 2.2 – OuterSPACE Data Flow

correspond to the *tile* levels), ExTensor is able to efficiently eliminate *ineffectual operations*, thus addressing the main issue in **IP** or complex *tiling* algorithms. Indeed, the input matrices would typically be stored in an *(UOP, UOP, UOP, CP)* format, corresponding to three levels of *tiling*. ExTensor is designed with a flexible architecture composed of different bricks (see Figure 2.3). To simply aid in the understanding, the interactions between the bricks are described using function calls:

- The *Scanner* is a storage unit that delivers a stream of coordinates in increasing order, fetching them from memory. The coordinates are delivered through the *Iterate()* operation call.
- The *Intersect* is a hardware block that co-iterates over several *Scanners* in parallel and compares the coordinate streams. This unit performs the *intersection* search required by **IP** type algorithms, delivering a single stream with the matching coordinates, which are the coordinates of non-zero *partial sums*. To optimise the *intersection* step, it is possible to skip a range of coordinates in a stream depending on the current coordinate of another stream. Thus, the *Intersect* sends skip messages with *SkipTo()* operations to the other *Scanners*.
- The *Coordinator* is a hardware unit that includes *Scanners* and *Intersect* units, and manages the input and output data depending on the current processed *tile*, as shown in Figure 2.3. Data and *metadata* are respectively stored in storage units and *Scanners*, while a *Sequencer* manages the *Iterate()* calls so that the *Intersect* brick delivers the stream of effective coordinates. In addition, in the *Tile Coord* brick, the offsets of the *tile* are added back, so the output of the *Coordinator* is in absolute coordinates.

These bricks can be placed at different positions in a memory hierarchy, depending on the *tiling* and selected hardware. Thus, the *Coordinator* is able to intersect at coarse-granularity with output data being *metadata* of the next *tile*, skipping an entire *tile* when appropriate, with low computation cost, and also at fine-granularity when output data are the non-zero elements that need to be multiplied.

ExTensor is designed with three levels of *tiling* that each have either input or output sequentiality. First, a *Sequencer* plus *Scanners* block ① is placed close to the main memory to determine which *tiles* to send to the next storage, the *Last Level Buffer* (*LLB*). Then, for each *tile*, a *Coordinator* ② in the *LLB* intersects the next level *tiles*, which are sent to several *PEs* that will process in parallel. Finally, in each *PE*, a *Coordinator* ③ intersects the non-zero elements

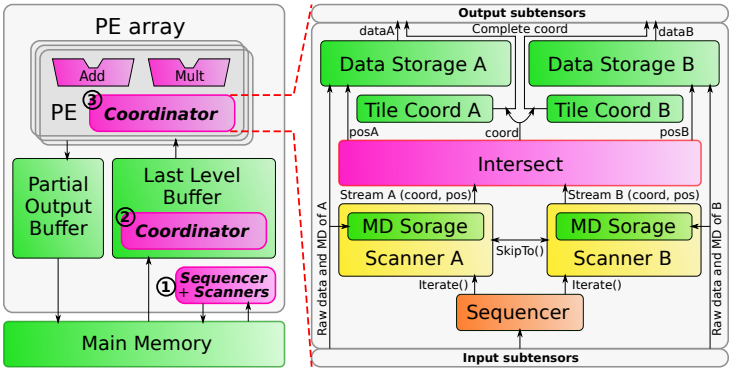


Figure 2.3 – Architecture of ExTensor

of the computed *tile* and sends them to the arithmetic unit of the *PE* for multiplication.

In addition to these bricks which are the main contributions, ExTensor still needs to perform the *merge* phase, including the *reduction* of the *POs* which are stored in CP^2 format. The *Partial Output Buffer* (*POB*), is managed by *tiles*, and the content of the *tile* is stored on-chip using a Content Addressable Memory (CAM) for quick access, enabling *reductions* to be performed. On overflow, the *tiles* are flushed to main memory. To benefit from ExTensor's support for *tiling*, the input matrices need preprocessing, making it difficult to directly compare performance with other accelerators.

2.2.3 SpArch

SpArch [131] uses an **OP** algorithm for SpGEMM in order to benefit from the simplified access pattern for the inputs. To address the management of the large volume of *POs*, SpArch pipelines the *multiply* and *merge* phases and uses condensing methods to reduce the number of *POs* and scheduling methods to reduce memory traffic. Unlike other **OP** accelerators [87], with SpArch, both A and B are stored in *(UOP, CP) row-major* format. As a consequence of this *CSR* format, the A-rows can easily be condensed to the left (see Figure 2.4). Thus, when a condensed A-column is read, it will require reading all the B-rows corresponding to the different A-columns that have been combined. The *MatA Column Fetcher* reads the combined A-columns and the *MatB Row Prefetcher* reads the necessary B-rows (see Figure 2.5), thus masking the memory latency. The *MatB Row Prefetcher*, stores multiple B-rows in a cache, and it uses information about the upcoming A-columns (provided by the *Distance List Builder*) to ensure the needed B-rows are kept in the cache.

A key contribution of SpArch is the merge unit that is able to merge sixty-four *POs* in parallel, through a binary tree type architecture. The core of the merger is a brick which compares and merges $4{\times}4$ $(index, value)$ pairs in one cycle. These are combined to build a hierarchical merger array whose output is then written to main memory, and read back for further merging by the *Partial Matrix Fetcher*. The authors note that the order in which the *reductions* are performed is important, specifically sparser *POs* should be merged first and they propose a Huffman tree scheduler to determine the best order to minimize memory accesses. Overall, SpArch is able to compute an **OP** SpGEMM [2] kernel with reduced main memory traffic for the *POs* mainly by addressing the *scatter* problem with a highly parallelized merger.

2. Since the merger can read *POs* from memory, the D-matrix can be treated like an initial *PO*.

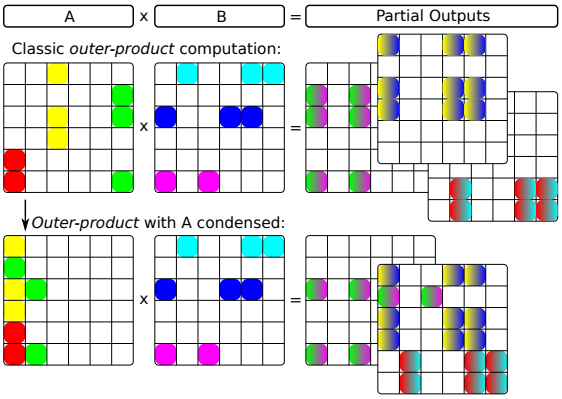


Figure 2.4 – *Outer-product* with a Condense A-Matrix

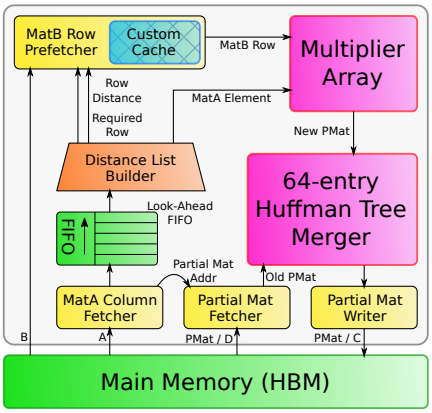


Figure 2.5 – Architecture of SpArch

2.2.4 MatRaptor

MatRaptor [105] is an accelerator designed to compute SpMSpM kernels. It is based on **Gust** algorithm [3], which makes it possible to process multiple A-rows (2 to 8 for MatRaptor) in parallel. In MatRaptor (see Figure 2.6) a *SpAL* (*Sparse Matrix A Loader*) fetches non-zero elements of a A-row and sends them to a *SpBL* (*Sparse Matrix B Loader*). With the *metadata* of this element, the *SpBL* fetches non-zero elements of the corresponding B-rows and sends pairs of A- and B-elements to a *PE*. This *PE* computes the *partial sums* and sends the results into queues for the *merge* process. Instead of separating *multiply* and *merge* phases, several queues are used to store *partial sums* and partially merged *partial sums*, allowing the two phases to be pipelined. The *PE* merges all the *partial sums* resulting from an entire A-row, before sending the final output (the corresponding C-row) to main memory.

In MatRaptor, multiple A-rows are processed independently and in parallel in different *PEs* and each *PE* is assigned to a channel in an HBM memory [57]. It is important that each *PE* can read its A-rows without interfering with the other *PEs*. With the classic *(UOP, CP) row-major* format, the rows are consecutive in memory and are not aligned to channel boundaries. To solve this problem, the authors propose a new format called C^2SR (*(FL, UOP, CP) channel-major*) that is based on a *(UOP, CP) row-major*. For example, with two *PEs*, all the elements of the even rows are stored in one channel, and the elements of the odd rows, in another channel, thus isolating the data. With this approach, the row pointers (*UOP*) are no longer monotonic, which means that the length of a row can not be deduced by subtracting adjacent row pointers. This requires the addition of a new table (which we call *fiber length FL*), which stores the length of the non-zero elements in each row (see Figure 2.7). This modified storage format enables multiple *PEs* to make effective use of the HBM.

MatRaptor is one of the first accelerators to use **Gust** algorithm, however, it has some limitations regarding effective reuse of B-rows, both between parallel *SpBLs* and temporally, as A is traversed. A key contribution is the proposed C^2SR input format, which makes it possible to separate rows into memory channels, to improve performance.

3. Called *row-wise product* in [105].

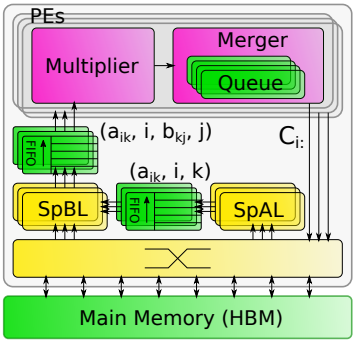


Figure 2.6 – Architecture of MatRaptor

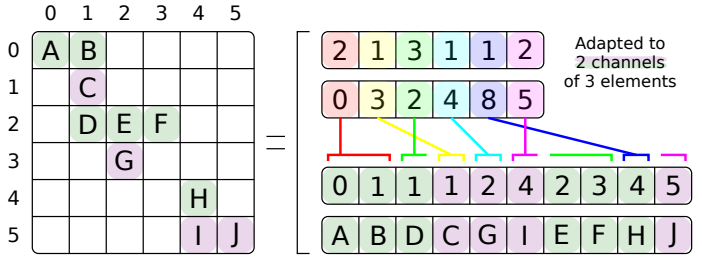


Figure 2.7 – *(FL, UOP, CP) Channel-Major* Format (C^2SR)

2.2.5 InnerSP

InnerSP [7] is a SpMSpM accelerator based on the **`Gust`** algorithm [4]. The authors of InnerSP justify their choice by showing quantitatively that with the **`OP`** the *POs* represent a large volume of data, and that for many matrices the *reductions* can not be done fully on-chip and thus must be buffered in main memory. Normally, with **`Gust`** algorithm, B must be read repeatedly, but InnerSP proposes a special cache for B which exploits the spatial locality.

The architecture of InnerSP is shown in Figure 2.8. The *A reader* reads the A-rows. Two separate caches are used for B, one that stores the row pointers and the other which stores column indices and values. The *B reader* reads the required B-rows, hopefully getting the data from the caches. Parallel multipliers perform the multiplications and then a hash-unit generates a hash based on the indices in C. A *reduction* unit based on hash tables contains parallel comparators which search for matching hash values and performs the *reductions*. When all the operations for a A-row are completed, the *C writer* writes the C-row back to main memory.

The architecture is efficient, as long as the *Hash Reduction Unit* does not overflow. When overflows occur, the extra entries are temporarily stored in main memory, but there is a significant performance hit. To minimize the chance of overflows, InnerSP uses a pre-scanner to estimate the number of non-zero entries in the upcoming C-rows. The estimate primarily considers the NNZ in the A-rows. If the estimated number of non-zero entries in C exceeds the capacity of the *Hash Reduction Unit*, then the row is split and processed in two passes. Conversely, if the number of non-zero entries in C is small, two A-rows can be processed simultaneously.

To improve the performance of the B-cache, the authors exploit a technique from [9]. The non-zero elements in the A-rows provide direct information about which B-rows are required. The column index table of A is already pre-fetched by the *A reader*, and when a row must be evicted from the B-cache, it chooses the ones which will not be required based on the upcoming entries in the column index table of A. This technique (also used by SpArch [131]) maximizes the reuse of B-rows.

InnerSP is an accelerator based on **`Gust`** algorithm which achieves high reuse of the B-rows, through a look-ahead in the A-*metadata*.

4. The authors call this *row-wise inner-product*, or confusingly sometimes *inner-product*.

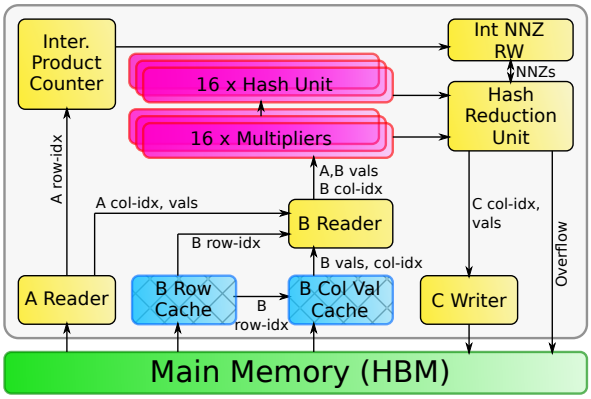


Figure 2.8 – Architecture of InnerSP

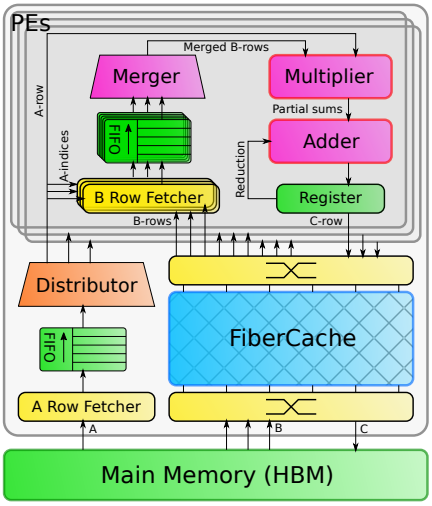


Figure 2.9 – Architecture of Gamma

2.2.6 Gamma

Gamma [127] is a dedicated accelerator for SpMSpM kernels, which uses **`Gust`** algorithm. The two input matrices are both stored in *(UOP, CP) row-major* format and the output is generated in the same format, providing consistency.

The Gamma architecture, presented in Figure 2.9, is composed of a fetcher for the A-rows that contains a *Distributor* to distribute them to 32 *PEs*. Each *PE* processes an entire A-row and computes the corresponding C-row. The *PE* fetches the required B-rows and merges and sorts them in a 64-wide *merge* block, before multiplying with the A-elements. As a result, the output C-row is generated in order. The *reduction* step is, thus, highly simplified, because any duplicate indices are adjacent. Normally, the C-row can be written back to main memory. However, if the A-row contains more than 64 non-zero elements, then it must be processed with multiple passes. After each pass, the partial C-row is stored temporarily. Finally, the temporary C-rows are read-back and merged by the *PEs*. Of course, the B-rows must be accessed repeatedly by the different *PEs*, and Gamma proposes *FiberCache* which is a highly banked cache which fetches the B-rows in advance, based on the column index table of A. It keeps track of which rows of B are most needed by the *PEs*, and when an eviction is necessary, the victim, is the row which is least needed.

Gamma encounters some difficulties when the A-matrix has rows that are too dense, requiring multiple iterations through the *PEs*, which creates a high-load on the cache. The authors propose a software preprocessing technique which splits dense A-rows and rearranges them to make best use of the *FiberCache*.

2.2.7 SortCache

The authors of SortCache [103] note that modern processors are able to load an entire cache line in parallel but in many sparse applications less than 50% of the data within a line is used, even with hand-tuned code. They propose a light SpGEMM accelerator which offloads the *reduction* step, based on the *Vectorized Binary Search Tree* (VBST), a data-structure from their earlier work [102, 104]. The VBST is a standard binary search tree where the size of node is a multiple of the cache block size and contains a sorted vector of K $(key, value)$ pairs.

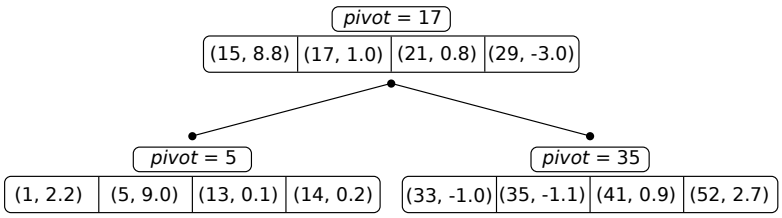


Figure 2.10 – VBST Data Structure Used by SortCache

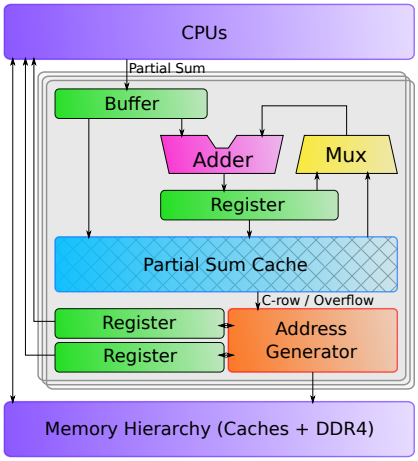


Figure 2.11 – Architecture of ASA

Associated with each node is a *pivot*; the pivots, with left and right child pointers, form a binary tree, as illustrated in Figure 2.10.

Using the VBST, SortCache focuses on accelerating the *scatter* updates to the output matrix, which are performed directly in the cache. The authors augment the processor with a single new instruction, *RED Key, Value*. After K updates have been issued, the hardware launches a *reduction* in the SortCache, where the K $(key, value)$ pairs are inserted into the VBST. This is done by traversing the tree from the root to the leaves. When duplicate keys are found, the *reduction* operation is performed in the cache. When new keys are found, they are inserted, which can result in a temporary, new node with up to $2K$ $(key, value)$ pairs. This temporary node is partitioned, with one half having exactly K entries. The insertion procedure may require visiting each level of the tree at least once ($\mathcal{O}(log(n))$ and it does not ensure the uniqueness of the keys, so at the end, an additional full traversal of the tree is required to remove duplicates.

The nodes closest to the root of the tree are accessed most frequently and are thus mapped to the caches closest to the processor. The authors propose to re-purpose cache tracking hardware (e.g. TAG values, LRU counters) to store the additional metadata (pivots, pointers). One of the difficulties with SortCache is that, in some cases, a large fraction of the VBST nodes are underutilized.

2.2.8 ASA

In the scope of graph analytic problems, the authors of ASA [126] observed that **`Gust`** algorithm achieves high performance for SpGEMM kernels but note that it still requires acceleration for the *reduction* step [5]. The software state-of-the-art for *reduction* uses hash tables [18]. Some hardware accelerators, such as HTA [129], enhance performance for hash operations, but those solutions are not optimized for SpGEMM kernels. ASA is a low area, near-core extension for a general purpose processor that specifically accelerates the *reduction* step for **`Gust`** algorithm.

ASA accelerates *reduction* operations received from software via custom instructions. As

5. The authors refer to the *reduction* step as SPA (Sparse Accumulation).

presented in Figure 2.11, the *partial sums* from these instructions are stored in a waiting buffer as $(index, value)$ pairs with a key that is a hash of the index, computed in software to maintain flexibility. The cache line, accessed with the key, is filled with the indices and the *partial sums*. If a corresponding *partial sum* is already stored, ASA accumulates it and writes back the result. If not, the *partial sum* is simply stored into the cache.

To minimize die area, the cache is not dimensioned for an entire C-row[6] but ASA handles cache overflows in hardware. When a *partial sum* needs to be stored in a fully filled cache line, a victim is selected and evicted, based on a LRU policy. The evicted entry is written into a pre-allocated FIFO queue in the memory hierarchy. ASA uses an address generator to manage evicted data and updates head and tail registers for the list. Software then traverses the list to merge duplicate keys to build the final output C-row. To avoid overflows, the authors advocate *tiling* methods to efficiently distribute work through one or several ASA cores.

ASA is an accelerator which optimizes the *scatter* aspect of SpGEMM kernel with low area overhead, while leaving high software flexibility. Unlike other **Gust** accelerators, there is no hardware *gather* support for reusing or caching the B-rows.

2.2.9 Other Accelerators

In the above sections, we have described a selection of the main sparse accelerators that have appeared in the literature, covering different matrix multiplication algorithms and architectures. This list is not exhaustive, and in the following paragraphs we present several other accelerators which we will cover in less detail, due to space constraints.

2.2.9.1 SPiDRE

The authors of SPiDRE (Sparse Data Rearrange Engine) [11] propose a solution for the poor utilization of the memory hierarchy in sparse applications. They propose a near-memory data rearrangement engine. This engine can preprocess data, near memory, to create new data-structures that are dense and benefit from the memory hierarchy. For example, with a SpMV kernel, the input vector could be rearranged through a user defined function to a dense format where all non-zero elements are in the right order of computation.

2.2.9.2 PHI

The authors of PHI [82] note that many large graph algorithms favour pull (each node reads inputs from its neighbours) compared to push implementations (each node propagates results to its neighbours), as push implementations require read-modify-writes which require accessing complete cache lines, even though a small unit of data is modified. The authors include **OP** SpMV (a push algorithm) in their evaluation.

PHI is a modified cache optimized to efficiently process commutative *scatter* updates. When an update is received but the line is not in the cache, the line is not fetched, but rather marked as being in *update mode*, where it simply stores the update. If subsequent updates hit the line, they are stored, or combined via the commutative operator. When a cache line is evicted, if it contains multiple updates, PHI assumes that the coalescing has been effective, and it processes the operations via a read-modify-write from main memory. If a cache contains few updates when evicted, PHI batches these updates as $(address, value)$ tuples, allocating a new

6. The authors present a *column-wise* **Gust** implementation. This choice is arbitrary and, for consistency, we stick with a *row-wise* presentation.

cache line to assemble the batch. Groups of updates are organized by *bins* corresponding to a small memory region. The batches are streamed to main memory. Later, the reading of the batches is also efficient, as they are contiguous.

In multi-core systems, private caches act as local buffers, thus coalescing the updates reduces coherency protocol traffic. This combination of in-cache coalescing and update buffering enables PHI to take advantage of locality when possible (like Coup [128]), but with the addition of *update batching*, PHI improves memory utilization, even when the data being updated is too large to fit in the cache.

2.2.9.3 GSCSp

GSCSp [**?**] is a FPGA-based accelerator based on **`Gust`** algorithm. In other **`Gust`** accelerators, a *PE* processes a full A-row but the authors note that when there is variability in the number of elements in the A-rows, *PEs* may be blocked, waiting for another *PE* to complete. Thus the authors propose to distribute the elements in an A-row to different *PEs* (*element-wise* parallelism). Initially, each *PE* buffers its output C-row, and when a *PE* advances to the next row, it pushes its partial C-row to a final merger. The authors also propose to use separate caches for the *metadata* for the B-rows and the values, and report that combined with the *element-wise* parallelism, the approach is effective for reducing the memory traffic for accessing B-rows.

2.2.9.4 Flexagon

Flexagon [81] is a SpMSpM accelerator for DNN applications. The authors claim that the optimal algorithm (**`IP`**, **`OP`** and **`Gust`**) depends on the matrix structure, potentially varying between DNN models and layers within a DNN, although in the majority of the cases, their data shows that **`Gust`** is the most efficient. Flexagon is configurable: it can implement all three algorithms. To achieve that, the authors designed a *Merger-Reduction Network* (*MRN*) that is able to perform multiplications (for *partial sum* generation), accumulations (for *reductions* operations) and comparisons (for *intersection* searches and *merge* phases) via a binary tree structure. Flexagon addresses the memory access issues of each algorithms via three custom memories: a FIFO for the A-matrix that is always read sequentially, a cache for the B-matrix that is subject to high temporal and spatial reuse in each of the three algorithms, and a PSRAM to store *POs* for **`OP`** and **`Gust`** algorithms.

2.2.9.5 Other Accelerators

In GraphR [101], the authors propose to use ReRAM technology and a cross-bar to perform graph operations directly in hardware, but this approach is not currently scalable to large problems. The ITS accelerator [94] focuses on an **`OP`** implementation of SpMV and they apply data-compression to the *metadata*. The authors of SPU [24] propose a more general hardware solution to input data dependencies and focus on graph applications. The focus of Alrescha [3] is an innovative and adaptive preprocessing of the matrix in hardware to simplify the accesses to the b-vector. The authors of CoSparse [32] focus on a software layer that selects the hardware acceleration that is best adapted to the given problem.

The SSSR accelerator [97] for RISC-V processors goes further by addressing *intersection* and *union* while streaming data to and from memory and it can be used for multiple kernels, however the RISC-V performance can not easily be compared with the other accelerators

presented in this paper. In passing we note three other accelerators EIE [44], SCNN [88] and CANDLES [42] which have hardware support for sparsity but which are highly focused on CNNs, besides SDMA [38] that focuses on GNNs.

The SpaceA [120] accelerator exploits near-memory computing inside a 3D DRAM memory to implement SpMV using an **IP** algorithm. With preprocessing, the rows of the matrix are assigned to banks within specific DRAM dies, while optimizing spatial locality. A separate die is used for storing the input and output vectors. Associated with each DRAM bank which stores the matrix, there is a *PE* which reads the non-zero values of the matrix, fetches the required vector entries and performs the computations. Bringing the compute near the DRAM and exploiting emerging 3D technology, is an orthogonal approach to reduce memory access overheads.

2.2.9.6 Other FPGA based Accelerators

Many sparse accelerators have been prototyped on FPGAs [80]. Indeed, AMD/Xilinx provide a turnkey library for SpMV [23, 69]. In [28], the authors propose a higher-performance SpMV using a modified *CSR* format where the data is packed for efficient, streaming access in HBM[7]. They also propose a highly-banked on-chip memory for the vector updates, and a hazard resolution strategy that allows efficient pipelining.

2.2.10 Further Classification

The classification can go further with more details in the different sparse formats used and the data location at each step of a multiplication as presented in Table 2.2.

For each inputs/outputs (including partial outputs), the sparse format used and the place it is stored in memory are described. Moreover, some information is added like the presence of data-rearrangements/hardware conversions for SPiDRE that convert sparse data into a reordered, dense version in main memory and SortCache that use a VBST storage scheme. As some accelerators only address a part of the kernel, the *SW* gray cells indicates whenever the input/output is managed by the software. Thus we naturally find these cells for PHI, SPiDRE, SortCache and ASA. Finally, the *NC* cells are used when the accelerator is not concerned by the partial outputs. This is obviously the case for SPiDRE since it only rearrange data in main memory ahead of time, PHI and SortCache directly generate the final output in caches (there eventually is overflow but we don't consider the resulting intermediate results as partial outputs of the kernel). MatRaptor [105], Gamma and InnerSP [7] manage partial output rows to sequentially build the output C with the *Gustavson's* algorithm.

2.3 Comparison of Existing Accelerators

In this section, we analyze and further compare the accelerators that were described in Section 2.2. We start with Table 2.3 where we summarize the accelerators, chronologically, showing the research group where they were developed. We see that MIT is very active in the field, as they were involved in developing four different accelerators (*e.g.* ExTensor, PHI, SpArch and Gamma).

7. GraphLily [50] is a precursor of HiSparse, also from Cornell University.

Table 2.2 – Target Applications and Matrix Formats of Existing Accelerators

Name	Sparse Kernel	**SpMSpV** ($A \times b = c$) or **SpMSpM** ($A \times B = C$) *(NC: not concerned ; SW: managed by the software ; →: hardware conversion / data-rearrangement ; RA: rearranged format)*							
		A		**b** or **B**		**Partial c** or **Partial C**		**c** or **C**	
		Format	**Stored in**	**Format**	**Stored in**	**Format**	**Stored in**	**Format**	**Stored in**
OuterSPACE	MM	*(UOP, CP) col.-maj.*	Main memory	*(UOP, CP) row-maj.*	Main memory	*(UOP, CP) col.-maj.*	Main memory	*(UOP, CP) col.-maj.*	Main memory
ExTensor	MM	*(UOP, CP)*, tiling	Main memory	*(UOP, CP)*, tiling	Main memory	CP^2	Buffer	CP^2	Main memory
PHI	MV	SW		SW		NC		*CP*	Caches, Main memory
SPiDRE	MV	SW		*CP* → *RA*	Main memory	NC		*CP* → *RA*	Main memory
MatRaptor	MM	*(FL, UOP, CP) channel-maj.*	Main memory	*(FL, UOP, CP) channel-maj.*	Main memory	NC		*(UOP, CP) row-maj.*	Main memory
SpArch	MM	*(UOP, CP) row-maj.*, condensed column	Main memory	*(UOP, CP) row-maj.*	Main memory	CP^2 *row-maj.*	Main memory	*(UOP, CP) row-maj.*	Main memory
Gamma	MM	*(UOP, CP) row-maj.*	Main memory	*(UOP, CP) row-maj.*	Internal cache	NC		*(UOP, CP) row-maj.*	Main memory
InnerSP	MM	*(UOP, CP) row-maj.*	Main memory	*(UOP, CP) row-maj.*	Main memory	NC		*(UOP, CP) row-maj.*	Main memory
SortCache	MM	SW		SW		NC		*(UOP, CP)* → *VBST*	Caches
ASA	MM	SW		SW		*CP*	Internal cache, Main memory	*(UOP, CP)*	SPiDRE Main memory
GSCSp	MM	*(UOP, CP) row-maj.*	Main memory	*(UOP, CP) row-maj.*	Internal cache	NC		*(UOP, CP) row-maj.*	Main memory

Table 2.3 – Chronological Summary of Sparse Accelerators

Name	Date	Institute / University	Summary
OuterSPACE [87]	2018	Univ. of Michigan / Arizona State Univ.	Highly parallelized SPMD, two levels cache – **OP** SpMSpM
ExTensor [46]	2019	Univ. of Illinois / NVIDIA / MIT	General purpose tensor algebra – With *tiled* SpMSpM
PHI [82]	2019	MIT (CSAIL) / Carnegie Mellon Univ.	In cache, hash-based accelerator for *reductions*
SPiDRE [11]	2019	UPC, BSC (Spain) / Arm Research	Near main memory data reorganisation for *intersections*
MatRaptor [105]	2020	Cornell	Row-parallel with HBM-aware custom format – **Gust** SpMSpM
SpArch [131]	2020	MIT / Stanford / NVIDIA	Compression method to reduce *POs*, *multiply/merge* pipe – **OP** SpGEMM
SpaceA [120]	2021	UC Santa Barbara	In memory accelerator for 3D DRAM – **IP** SpMV
Gamma [127]	2021	MIT (CSAIL)	Parallelized architecture with *FiberCache* for B – **Gust** SpMSpM
InnerSP [7]	2021	KAIST / DGIST (S. Korea)	A-look-ahead, caches for B, *Hash Reduction Unit* for C – **Gust** SpMSpM
SortCache [103]	2021	Georgia Tech / Zettaflops / Sandia Labs	In cache accelerator for *reductions*, based on the VBST
ASA [126]	2022	Lehigh Univ. / Lawrence Berkeley	Custom cache accelerator for *reductions* – **Gust** SpGEMM
Flexagon [81]	2023	Univ. of Murcia / Georgia Tech / NVIDIA	Configurable accelerator supporting all three algorithms – SpMSpM
GSCSp [**?**]	2023	Nanyang Technological Univ.	Element-wise, FPGA-based accelerator – **Gust** SpMSpM

2.3.1 Benchmarks, Evaluation and Performance

Up to this point, we have discussed the architecture of the various accelerators, without reporting the performance they achieve. Most of the matrices used for benchmarking come from the SuiteSparse Matrix Collection [63] that contains a large number of matrices from diverse, real applications. Domain specific matrix libraries are also used [6, 65, 91, 99], as well as some synthetic matrix generators [1, 13]. The densities are in general less than 1% and can be as low as 10^{-5}%. ExTensor, SortCache and Flexagon also benchmark with denser matrices. In all cases, large matrices with several million NNZ, exceeding the size of a normal cache, have been evaluated.

In Table 2.4, we note any required preprocessing, assuming, arbitrarily, that matrices are initially stored in *CSC* format[8]. We see that a few accelerators require a conversion to their specific format (*e.g.* ExTensor and MatRaptor). PHI, SortCache and ASA are not concerned by preprocessing since they only address *scatter*. In any case, fair benchmarking must take into account the cost of specialised preprocessing.

Not all accelerators have reached the same level of design maturity, and in Table 2.4, we have shown how each design was analyzed or simulated. All designs report that they have been simulated at a cycle-accurate level. Most of the accelerators use custom simulators or tools like *gem5* [72], *ZSim* [95] and STONNE [83], but unfortunately there is no common approach, making it difficult to fairly compare the reported performance. RTL descriptions are often only generated for key parts of the design. To estimate power and area, authors used tools such as CACTI [10] and McPAT [67][9]. In the last column of the table, we show the size of the local memory storage used in simulation by each accelerator. Except for SPiDRE and SpaceA which work directly in main memory, the local memories are in the range of 0.3MiB to

8. This choice is based on the SuiteSparse Matrix Collection where matrices are stored *column-major*.
9. We have not attempted to compare power/energy as the data in the literature is heterogeneous and can not be fairly compared.

Table 2.4 – Qualitative Analysis and Evaluation of Sparse Accelerators

Name	Preprocessing[1]	Functional Evaluation (Tool / Evaluation Specifics / Host[2])	Local Memory Size
OuterSPACE [87]	$A \rightarrow$ *row-major*	*gem5* / Model only key elements; Skip memory allocation; Ignore start-up time / ∅	528KiB (33.8k entries)
ExTensor [46]	A, $B \rightarrow$ highly *tiled* custom format	Custom Python simulator / Only the *Coordinator* is evaluated; Remainder modeled analytically / ∅	∼38MiB (2.5M entries)
PHI [82]	*N/A*	*ZSim* / Entirely evaluated / 16-cores system with 3 level cache	32.3MiB (2.8M entries)
SPiDRE [11]	Not needed	*gem5* / *Unknown* / Single ARM core	∅ (main memory size)
MatRaptor [105]	A, $B \rightarrow C^2SR$	*gem5* / Entirely evaluated / ∅	320KiB (27.3k entries)
SpArch [131]	A, $B \rightarrow$ *row-major*	Custom C++ simulator / Entirely evaluated / ∅	940KiB (62.5k entries)
SpaceA [120]	$A \rightarrow$ row-aligned to DRAM banks	Custom simulator / Simulation tracking values stored in 3D-DRAM / ∅	4GiB (268.4M entries)
Gamma [127]	A, $B \rightarrow$ *row-major*	Custom simulator / Entirely evaluated / ∅	3MiB (262.1k entries)
InnerSP [7]	Not needed	Custom simulator / Entirely evaluated / ∅	≤976.5KiB (119.9k entries)
SortCache [103]	*N/A*	Custom simulator / Entirely evaluated / Single threaded, in-order core with 3 level cache	16.3MiB (1.4M entries)
ASA [126]	*N/A*	Modified *ZSim* / Entirely evaluated, one ASA per core / 8-cores with 3 level cache	8×8KiB (8×1k entries)
Flexagon [81]	Not needed	Custom simulator based on STONNE / Entirely evaluated / ∅	1.3MiB (327.7k entries)
GSCSp [**?**]	A, $B \rightarrow$ *row-major*	HLS / FPGA Prototype / ∅	2.1MiB (178.2k entries)

[1]We assume matrices are initially stored in *(UOP, CP) column-major* format (*CSC*).
[2]Only for *processor assist* accelerators.

Table 2.5 – Benchmark Matrices for Sparse MM and MV

Name	# Matrices	Density Range	Largest NNZ	Sources
OuterSPACE [87]	20	from 0.0001% to 1.082%	16,518,948	SuiteSparse and SNAP
	unknown	*unknown*	*unknown*	Graph500 R-MAT synthetic matrices
ExTensor [46]	14	from 0.0002% to 20.3%	11,524,432	SuiteSparse and FROSTT
PHI [82]	1	0.0001%	1,949,412,601	SuiteSparse
SPiDRE [11]	*unknown*	*unknown*	*unknown*	*unknown*
MatRaptor [105]	14	from 0.00061% to 1.1%	5,105,039	same as OuterSPACE
SpArch [131]	20	from 0.0001% to 1.082%	16,518,948	same as OuterSPACE
SpaceA [120]	15	from 0.0003% to 0.347%	14,148,858	SuiteSparse
Gamma [127]	37	from 0.0001% to 1%	16,518,948	same as OuterSPACE plus SuiteSpars
InnerSP [7]	set of 755	*unknown*	5,105,039	SuiteSparse and SNAP
	14	from 0.00061% to 1.1%	5,105,039	from OuterSPACE
SortCache [103]	11	from 0.4% to 78.5%	2,097,566	SuiteSparse
ASA [126]	9	from 0.00002% to 0.3%	∼104,000,000	HipMCL, SNAP and GAP
Flexagon [81]	9	from 6% to 100%	2,718,144	MLPerf
GSCSp [**?**]	10	from 0.09% to 2.80%	484,256	SuiteSparse

38MiB, corresponding to 27k to 2.8M entries. Thus, in the model presented in Section 1.5.4, most of the accelerators are situated in the middle-row of Figure 1.5, with a local memory corresponding to a few matrix rows.

The accelerators performance are reported as a speedup versus a baseline, which may differ according to their hardware and software platforms: typically the Intel MKL library [53] and the Armadillo library [96] for CPUs, or cuSPARSE [86] and CUSP [25] for GPUs. Since different baselines with different numbers of threads and cores are used, these speedups need to be interpreted with care. The speedups are in general higher for the *standalone* accelerators as they provide optimized hardware for the entire kernel, while *processor assist* accelerators have less impressive speedups but aim to be general purpose and have lower area overhead.

The only accelerators that have a comparable baseline are the *standalone* ones addressing SpMSpM. They all compare themselves to OuterSPACE with the same benchmark matrices, so we can sort them by their speedup: Gamma ($6.6\times$)[10], InnerSP ($4.57\times$), SpArch ($4\times$), MatRaptor ($1.8\times$) and OuterSPACE ($1\times$). Three of the five *standalone* accelerators use the **Gust** algorithm (Gamma, InnerSP and MatRaptor). SpArch, an **OP** accelerator employing a complex architecture, outperforms MatRaptor, but does not achieve the same performance as InnerSP or Gamma, which suggests that **Gust** is most adapted to high-performance hardware accelerators.

These measurements are consistent with our model presented in Section 1.5.4 (Figure 1.5). OuterSPACE, the baseline, corresponds to graph ⑤. MatRaptor, the first **Gust** accelerator, was able to achieve higher performance by reducing the number of *random* accesses, and creating better regularity (red to yellow), as seen in graph ⑥.

SpArch, is an **OP** accelerator with a much larger local memory, which is used to address the red region in graph ⑤. Their A-matrix condensing method, allows them to reduce the number of *POs*, and approach the performance achievable in graph ⑧, although, *in fine*, it is impossible to store all the *POs* in local memory. InnerSP and Gamma correspond to optimizations of the **Gust** approach in graph ⑥, to approach the performance in graph ⑨. This is achieved by custom caches for the B-matrix to address the orange region. With the **Gust** algorithm, it is only necessary to store a limited number of B-rows, which is achievable with the large local memories in these accelerators. The model presented in Section 1.5.4 is helpful to understand the local memory requirements for SpMSpM accelerators.

2.3.2 Analysis for Designers

There is no single "best overall" accelerator. A designer must consider the required performance, the available area and the class of problems to be addressed. In Figure 2.12 we present a flow diagram which can guide a designer towards an architecture based on their requirements. Starting from the top-right of the figure, the designer must first decide between a *specialized accelerator* for a specific kernel or a *general purpose accelerator*. The former is preferable if performance must be maximized, requiring a *standalone* accelerator addressing both *gather* and *scatter* (*e.g.* MatRaptor, SpArch, Gamma and InnerSP).

A *general purpose accelerator* can either take the form of a highly configurable *standalone* block (*e.g.* OuterSPACE and ExTensor) or a *processor assist* that boosts performance for a specific task. With a *processor assist* (rightmost branch in the figure), the processor still performs important parts of the kernel, typically the multiplications, but off-loads the *scatter*

10. This speedup increases to $7.7\times$ with their proposed preprocessing.

Table 2.6 – Reported Performance of Sparse Accelerators

Name	Date	Baseline for Comparison		Kernels	Average Speedups
		Hardware	Software		
OuterSPACE	2018	6 cores/12 threads, 3.6 GHz Intel Xeon E5-1650V4 CPU	Intel MKL	SpMV, SpMSpM	7.9×
		NVIDIA Tesla K40 GPU	cuSPARSE		1.13×
			CUSP		1.14×
ExTensor	2019	12 cores/24 threads, 3 GHz Intel Xeon E5-2687W v4 pair	Intel MKL	SpMSpM	3.4×
				SpMM	1.3×
PHI	2019	2.2 GHz, 16 core x86-64 system	outer-product	SpMV	7×
			inner-product		1.4×
SPiDRE	2019	single out-of-order Arm processor	*unknown*	SpMV	1.62×
MatRaptor	2020	single-threaded, 2.20 GHz Intel Xeon E5-2699 v4 server-class CPU	Intel MKL	SpMSpM	77.5×
		12 threads, 2.20 GHz Intel Xeon E5-2699 v4 server-class CPU			7.9×
		NVIDIA Titan Xp GPU	cuSPARSE		37.6×
		OuterSPACE	∅		1.8×
SpArch	2020	4-core ARM A53 CPU	Armadillo	SpMSpM	1285×
		6-core Intel Core-i7 5930K CPU	Intel MKL		19×
		NVIDIA TITAN Xp GPU	cuSPARSE		18×
			CUSP		17×
		OuterSPACE	∅		4×
Gamma	2021	4-core, 8-thread Skylake Xeon E3-1240 v5	Intel MKL	SpMSpM	33×
		OuterSPACE	∅		6.6×
		SpArch			1.84×
InnerSP	2021	6-core Intel Core-i7 5930k CPU	Intel MKL	SpMSpM	18.0×
		OuterSPACE	∅		4.57×
		SpArch			1.068×
		MatRaptor			2.45×
SortCache	2021	single-threaded	*unknown*	SpGEMM	1.56×
ASA	2022	8-core, 2.6 GHz architecture hash-based with main memory based on Micron MT40A2G4 and cache sizes and latency based on Intel i7-6700	*unknown*	SpMSpM	2.25×
		HTA with similar cache sizes	∅		1.67×
GSCSp	2023	Xilinx Zynq UltraScale ZCU106 (quad-core ARM Cortex-A53)	row-wise	SpMSpM	1.62×

Table 2.7 – Main Memory Technology and Reported Die Area of Existing Accelerators

Name	Date	Technology (nm)	Area (mm^2)	Normalized Area	Main Memory Technology	Main Memory Bandwidth
OuterSPACE	2018	32	87	2081.54	HBM2	128GB/s
ExTensor	2019	32	96.89	2318.17	*unknown*	*unknown*[1]
PHI	2019	14	0.008	1 (base)	DDR4	51.2GB/s
SPiDRE	2019	*unknown*	*unknown*	*unknown*	*unknown*	*unknown*
MatRaptor	2020	28	2.257	70.53	HBM2	128GB/s
SpArch	2020	40	28.49	436.25	HBM2	128GB/s
Gamma	2021	45	30.6	370.22	HBM2	128GB/s
InnerSP	2021	40	18.7	286.34	HBM2	128GB/s
SortCache	2021	15	0.79	86.02	*unknown*	*unknown*
ASA	2022	14	0.014	1.75	DDR4	38.4GB/s
GSCSp	2023	ZCU106: 74% LUTs / 47% FFs / ∼ 60% U-BRAM			DDR4	6.4GB/s

[1]The memory bandwidth is unknown but the CPU has a theoretical maximum bandwidth of 68.256GB/s.

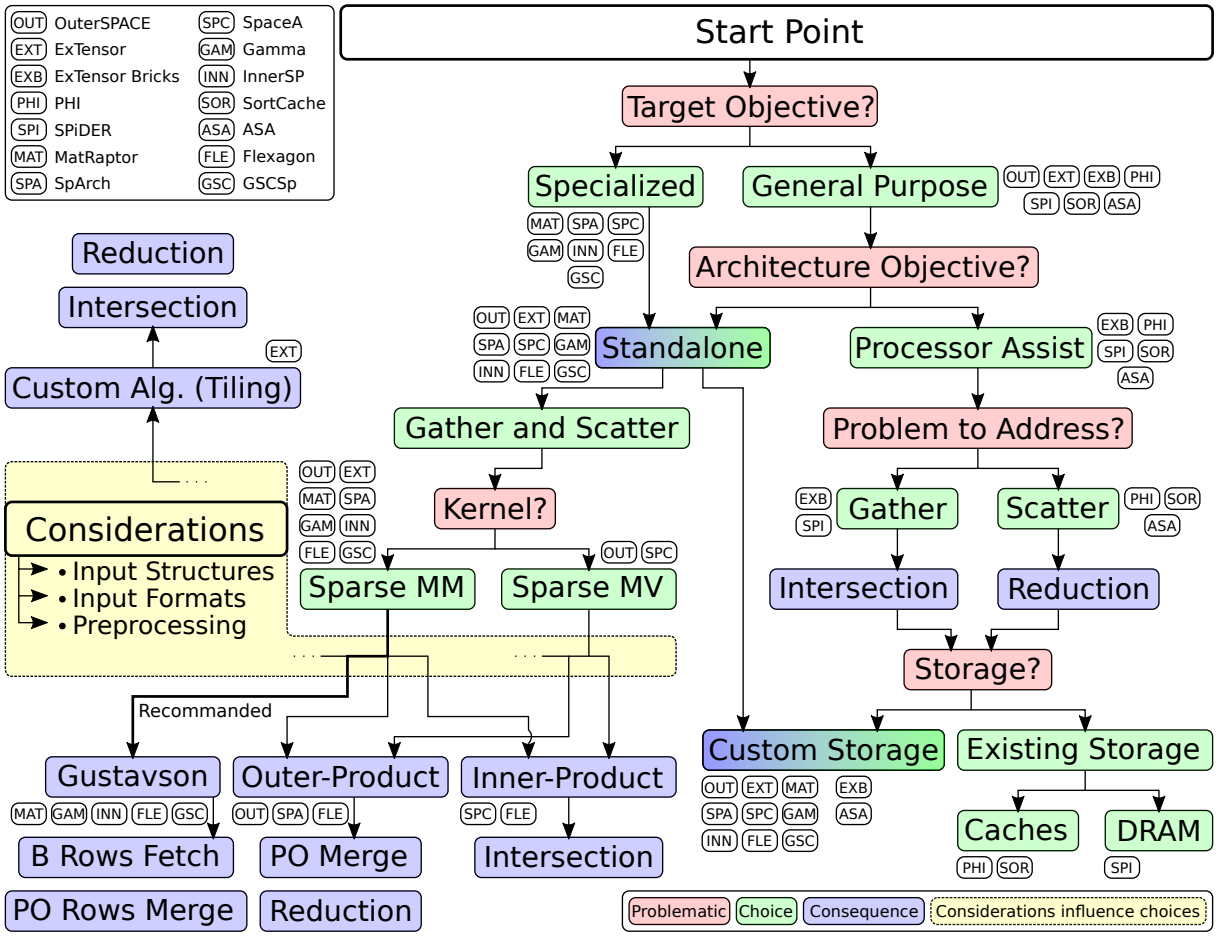


Figure 2.12 – Flow Diagram for Architecture Selection

or *gather* tasks to the accelerator, as these are the tasks which are the bottleneck with a traditional cache hierarchy. For example, for an **IP** algorithm, the difficult part of the *gather* task is performing the *intersection* (e.g. the ExTensor bricks and SPiDRE). Conversely, with **OP** or **Gust**, the challenge with the *scatter* is to efficiently perform *reductions* (e.g. PHI, SortCache and ASA). Finally, the designer must decide whether the storage is done by reusing existing memory resources (*e.g.* PHI, SPiDRE and SortCache) or through custom caches/memories.

Going back to the case of a *standalone* accelerator, the algorithms differ according to the supported kernels. For sparse MV, the accelerators studied here have opted for an **OP** algorithm. For sparse MM, the designers of three of the five highest performing accelerators have chosen **Gust** algorithm, as this algorithm provides a good trade-off, simplifying the *gather* by reusing input B-rows and the row-by-row generation of the output, simplifying the *scatter* aspect. The fact that the outputs are generated row-by-row, facilitates parallel implementations. **Gust** works well with *(UOP, CP)*, providing consistent input and output formats. Beyond the basic algorithm, *tiling* approaches, as proposed by ExTensor, can be explored.

Orthogonal to previous choices, other design considerations are important. First, the starting point needs to be defined: some accelerators require software to perform *tiling* (*e.g.* ExTensor) or to rearrange the rows to improve performance (*e.g.* Gamma). The cost of such preprocessing can be high, and is usually only justified if the same matrix is reused multiple times.

2.4 Conclusion

The size of the datasets considered in computational physics, machine learning and graph analytics is continuing to grow and over the last six years, we note the emergence of many hardware accelerators designed to improve upon the performance of CPUs and GPUs. In this paper, we have presented the most important designs and established a comparison, while highlighting the underlying hardware challenges associated with sparse multiplication kernels.

As we have seen, with this class of problem, the challenge is to efficiently use the memory system despite the dereferencing required to access the *metadata* associated with the *sparse data formats*. Depending on the selected MM algorithm, the major challenge is either with the *gather* and *intersection* or with the *scatter* updates. We have identified that accelerators based on *Gustavson's* algorithm achieve a trade-off, where, on the *gather* side, a limited number of B-rows need to be accessed, and on the output side, the result is generated row-by-row, simplifying the *scatter* updates. The best *Gustavson* accelerators have implemented further optimizations such as smart cache eviction strategies where the victim is selected by looking ahead at the A-indices. Several of the smaller *processor assist* accelerators reuse existing cache memories and simply modify the behaviour of the control logic (directories, tags, eviction policies) to optimize the operation for sparse matrices. On the other hand, the highest performance accelerators propose dedicated, massively parallel *reduction/mergers*, to maximize parallelism.

Given the number and diversity of existing designs, it is clear that further hardware optimizations and improvements are possible. As stated earlier, there may never be a single "best" sparse accelerator, but rather designs that are best suited to specific classes of problems. We believe that the structure of the matrices is critical in terms of the sizing of caches and other resources, and that in the coming years, new accelerators will be increasingly tailored based

on the structure of the matrices for specific classes of problems.

TAKEAWAYS

TODO

Chapter 3

Architectural Presentation of SpDCache

Contents

3.1 Introduction

In the previous chapters, we analyzed the causes and effects of irregular memory accesses in sparse computations and identified that the *Outer-Product* (**OP**) algorithm of Sparse Matrix-Vector multiplication (SpMV) ($A \times b = c$) has potential for high performance and optimization. The **OP** algorithm generates irregular updates on the output vector c, called "random" *reductions*, as each non-zero element of the input matrix A produces updates to distinct, and often distant, location in the output vector c. These "random" *reductions* render stress the memory system.

Conventional cache hierarchies are designed for regular, predictable access streams that exploit spatial and temporal locality. In contrast, **OP** SpMV exhibits poor locality, leading to low cache utilization, frequent evictions, and thus requiring more memory bandwidth for the output vector c.

Previous works, such as PHI [82], SortCache [103], and ASA [126], have proposed *reduction*-aware cache mechanisms to alleviate irregularity by improving accumulation and storage efficiency. However, these solutions do not exploit the structural properties of the processed matrices, and therefore miss optimization opportunities when the matrix has some regularity. This observation motivates our focus on banded sparse matrices, which combine overall

sparsity with a dense region near the diagonal. Banded matrices are very common and serve as an initial example to study how dense and sparse regions can be leveraged to enhance performance.

This chapter begins by reviewing the main concepts of generic data-caches, introducing certain terminology and mechanisms which are required to discuss cache behavior under irregular access conditions. It then presents a *reduction*-aware cache architecture, designed to efficiently support "random" *reductions* for the `OP` SpMV kernel. The proposed design, SpDCache (presented in Paper II), extends traditional caching models with a region-based organization and native *reduction* support. Finally, we evaluate its performance through high-level modeling on banded and non-banded matrices, highlighting the influence of data regularity on cache efficiency.

3.2 Generic Data-Caches

MODERN processors and accelerators integrate data-caches to bridge the latency and bandwidth gap between computation units and main memory [47]. A cache is a small, fast memory that stores recently accessed data anticipating that the data will be reused in the near future. Its efficiency relies on two main forms of locality:

- temporal locality, where recently accessed data is likely to be reused within a short interval of time, and
- spatial locality, where data stored near recently accessed addresses is likely to be used next.

By exploiting these properties, caches reduce the average access latency and increase the overall bandwidth efficiency.

A cache is composed of several cache-lines, each grouping contiguous memory words. The cache-line is the minimum granularity of data movement between memory levels. When an access occurs, the cache controller searches for the requested address among the currently stored cache-lines. If the address is found, a hit occurs and the data is served directly to the core. Otherwise, a miss occurs, triggering a memory fetch from a lower cache level or from main memory. If necessary, an existing cache-line is evicted to make room for the new data. Each cache-line is associated with a *tag* that records its address and additional bits to keep track of its state.

Several mapping policies exist. In a direct-mapped cache, each memory block maps to a single cache-line. This approach is simple to implement but results in conflicts and poor utilization. A n-*way set*-associative cache allows each block to be placed in one of n possible cache-lines within a *set*, improving hit rate with only a moderate additional hardware cost. A *fully associative* cache removes these constraints, as any block may occupy any cache-line, but requires more complex *tag* searches. In all cases, when a line must be evicted and multiple candidates exist, a replacement policy such as *Least Recently Used* (*LRU*) or *Random* decides which entry is evicted.

Caches have different policies when writes occur. With a *write-through* policy, every store operation updates both the cache and main memory, maintaining consistency but increasing traffic. A *write-back* policy delays the update until eviction, reducing memory bandwidth at the cost of additional state bits (dirty flags) to maintain consistency.

For workloads dominated by sequential accesses, these mechanisms and policies are highly effective. However, irregular accesses tend to degrade cache efficiency by increasing miss rates and provoking frequent evictions.

Modern processors implement hierarchical cache organizations to balance latency, capacity, and throughput. A typical CPU integrates small L1 caches close to each core, larger L2 caches shared by a group of cores, and a high-capacity L3 cache shared at the chip level. Accelerators and domain-specific architectures often employ local data storage or scratchpad memories serving the same purpose, providing low-latency access to frequently reused data (see Chapter 2).

The generic cache assumes regular read and write operations with predictable memory access patterns. Yet workloads operating on sparse data violate this assumption. In particular, “random” *reductions* often have poor performance because each *reduction* triggers a read and a write of a full cache-line, to simply update one word. Furthermore, the lack of spatial locality means each *reduction* can trigger an eviction, in the worst case.

In the following sections, we introduce cache architectures with native *reduction* support, specifically designed to handle irregular data. These architectures extend the classical cache model with *reduction*-aware mechanisms, improving bandwidth efficiency under irregular access patterns.

CONCLUSION

Generic data-caches are effective when access patterns exhibit spatial and temporal locality. This is not the case for sparse data and their performance is particularly poor for "random" *reduction* operations. This limitation motivates the introduction of ***reduction*-aware cache architectures** which are optimized for such "random" *reductions*.

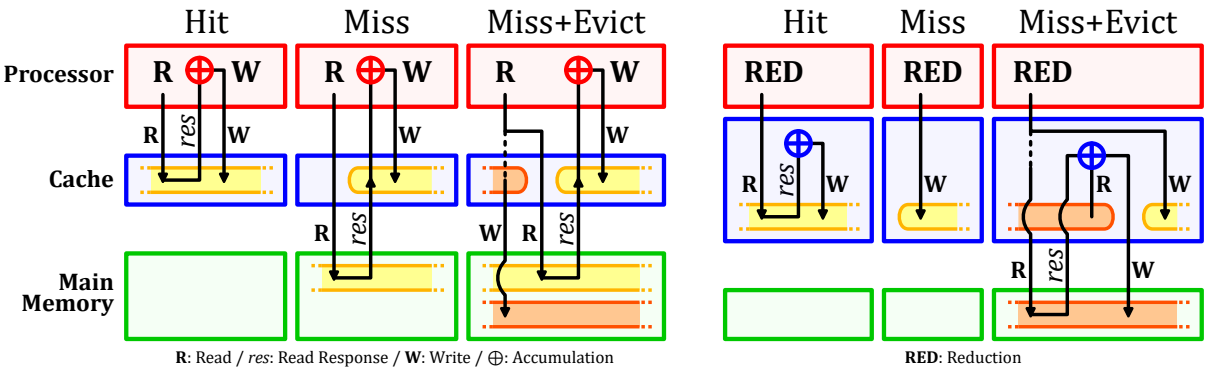
3.3 Reduction Cache

PERFORMING a *reduction* in hardware requires a read from memory, followed by an accumulation and a write back. This operation is denoted $RED(key, value)$ [103] with the *key* being the position in the output vector c.

In Figure 3.1a, we show three cases that occur in a generic data-cache when a *reduction* is performed. If the vector entry (c_{key}) hits in the cache, the update is performed by the processor (in *red*) with no external memory access (on the left in the figure). When there is a miss, the operation is blocked until the read from main memory completes (in the middle of the figure). Potentially, the miss will also trigger an eviction, which exacerbates the blocking condition, because a cache-line must first be written back, before the read from main memory can be performed (on the right in the figure). In scientific applications, the matrices are extremely large and only a small portion of the vector c can reside in the cache, thus the *miss+evict* case arises frequently.

A *reduction cache*, such as [103] or [82], is a data-cache which accepts *reduction* instructions (*RED*) from the processor and performs accumulations locally (near-memory). In the case of a hit, see Figure 3.1b, the *reduction* takes place in the cache (in *blue*, on the left in the figure). For a miss, the cache allocates a new line with zeros, and simply inserts the value (in the middle of the figure). In this case, the cache-line is inconsistent with main memory. Only when the line needs to be evicted, is a read performed (on the right in the figure). In the cache, the values in the line are accumulated with the main memory data, and the result is written back.

As seen in the figure, for hit and *miss+evict* cases, the number of memory transactions is the same as a generic cache. However, a *reduction cache* reduces the number of main memory accesses in the case of a miss, by postponing the main memory update until eviction. Instead of having the classic ascendant policy where data is stored in the cache to be pulled-up again by the processor latter-on, the *reduction cache* has a descendant policy: data is written/updated in the cache to be pushed-down to memory latter-on. Instead of two memory transactions that can stall the processor, the *reduction cache* only needs a single "fire-and-forget" instruction, meaning it can be issued without waiting for an answer, thus avoiding processor stalls. At the end of the algorithm, the *reduction cache* must be flushed.



(a) Generic Cache (Write-Back) **(b)** Reduction Cache

Figure 3.1 – Read (R) and Write (W) Transactions of a *Reduction* Operation in Generic and *Reduction* Caches

DEFINITION

A ***reduction cache*** is a data-cache containing an arithmetic unit and a dedicated support for *reduction* operations. One implementation is to provide the processor with a *RED* instruction which tells the cache to perform the *reduction*. For the processor, this instruction is "fire-and-forget".

CONCLUSION

To improve the processing efficiency of *reduction* operations, a ***reduction cache*** can be used to **avoid unnecessary memory transactions and stalls**, by offloading the accumulation near-memory, and updating the main memory only at eviction.

3.4 Outer-Product SpMV with a Reduction Cache

A widely used sparse format to compress the non-zero elements in a matrix A is CSC [93], where A is a sparse matrix of dimensions (M, N) with a number of non-zero elements nnz. As seen in Chapter 1, it consists of three tables (val, idx, ptr) used to store non-zero values, row indices and column positions. When performing an `OP` SpMV ($A \times b = c$) with this format (Algorithm 3.1), we see that a *RED* is needed to accumulate the output vector c (line 6). Moreover, A is read sequentially while c is updated "randomly". In fact, the *key* is determined via an indirection with an access to idx.

Algorithm 3.1: *Outer-Product* SpMV – Single Threaded

```
1 for n ∈ [[0, N − 1]] do                          // Loop over the columns of A
2     for pos ∈ [[ptr_n, ptr_{n+1} − 1]] do
3         key ← idx_pos;                                          // Row index
4         val ← val_pos × b_n;                          // val = A_key,n × b_n
5         RED(key, val);                                        // c_key += val
```

In memory, A is stored as three dense tables (CSC), but conceptually, as we perform the `OP` multiplication, the matrix is traversed from top to bottom, then left to right. A row in the matrix corresponds to a *key*, or equivalently, a position in the output vector c. In Figure 3.2, we show an example of a sparse matrix with a few non-zero values (① - ⑥). As we traverse the

matrix, we first perform a *reduction* for the value on row key_{10}, then on row key_{20} and then key_{30}. Each of these results in a *RED* on c. As we continue, we then hit another value ⑤ on the row key_{10} which must be *reduced* with an existing entry ① (as shown on the left of Figure 3.2, *Conceptual View*).

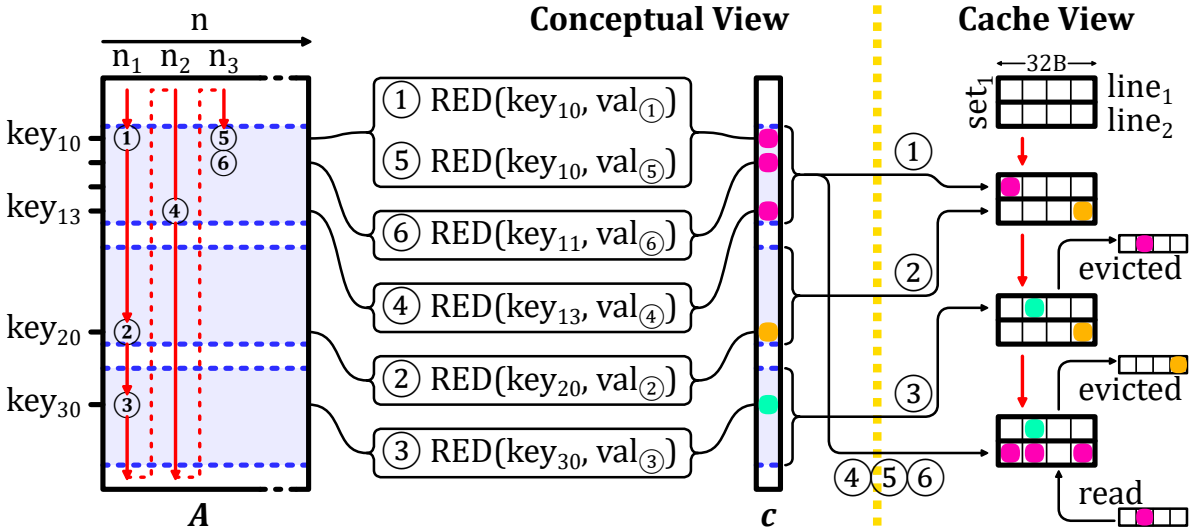


Figure 3.2 – Traversal of Non-Zero Elements of Input Matrix for `OP` SpMV (left) and Impact on Cache (right)

Let us consider how c is stored in a two-*way* *set*-associative cache with 32B lines. We assume that all the *keys* map to set_1. When ① is updated, there is a miss and $line_1$ is allocated for key_{10}. Similarly with ②, $line_2$ is allocated for key_{20}. When key_{30} comes along, an eviction must be performed. We assume $line_1$, holding key_{10}, is evicted. We then come to ④ which has key_{13}, adjacent in memory to key_{10}, and thus stored in the same line. This line must be brought back to process ④, ⑤ and ⑥.

This example illustrates how a conventional cache can be inefficient. First, we note the line holding key_{10} was evicted, although it was frequently accessed after ①. This eviction was triggered by key_{30}, which was only accessed once. Furthermore, for key_{20} and key_{30}, a full line was loaded, only to modify a single entry. A "smarter" cache would have maintained the line with key_{10} to avoid an unnecessary main memory access. Furthermore, this "smarter" cache would not have allocated full lines for key_{20} and key_{30} to only modify a single word.

The globally low local and temporal localities prevent caches, and even *reduction caches*, to take advantage of any reuse opportunity. However, sparse matrices have various types of structure which may have reuse potential in some parts of the matrix. In the example in Figure 3.2, the first rows (with key_{10} to key_{13}) are denser than the rest of the matrix. A "smarter" cache would prefer to focus on these rows, keeping the corresponding cache-line and accumulating every elements before a final eviction. On the contrary, the sparser rows of the matrix would not be kept by the "smarter" cache, thus avoiding thrashing data from denser regions.

The matrix structure is key to designing an efficient *reduction cache* for `OP` SpMV. Going forward, we focus on the case of banded matrices, a widely used type of sparse matrix, which have a denser region around the diagonal. For a *reduction cache* to work efficiently with such matrices, the dense band should be prioritized, and the isolated elements out of the band should be processed separately to avoid these sparse elements disturbing the storage

of the dense data. This is the main idea of SpDCache (Paper II), which is a *reduction cache* with dedicated memory regions for the in-band and out-of-band of a matrix. We describe its behavior in the following sections.

CONCLUSION

When computing an `OP` **SpMV kernel** on a very **large matrix**, a **conventional *reduction cache* does not provide enormous benefit** because the cache is not large enough to reuse data from one column iteration to the next.

We propose **SpDCache**, a *reduction cache* with dedicated memory regions, **leveraging matrix structure** with dense and sparse regions which correspond to the structure of **banded matrices**.

3.5 Architecture of SpDCache

We design a specialized data-cache architecture, called SpDCache, that exploits the two-region structure of banded matrices to reduce memory traffic during *reductions* operations on the output vector c. To better understand SpDCache behavior, our study focuses on a single-level cache hierarchy with a single core executing SpMV kernels on banded matrix case. However, SpDCache is designed to extend to multilevel memory hierarchies and multi-core architectures (see Chapter 6), and to adapt to other region configurations. As an isolated data-cache, SpDCache must satisfy three key requirements:

- **Generic cache functionality:** SpDCache operates as a standard cache for load and store instructions from the processor, enabling sequential reads of matrix A and vector b to leverage spatial locality.
- **Reduction capability:** SpDCache functions as a *reduction cache* to minimize inherent memory traffic costs of *reduction* operations on output vector c.
- **Region-wise reduction:** Dedicated cache regions for sparse and dense matrix regions mitigate the irregular access patterns of *reductions*.

As shown in Figure 3.3, SpDCache is divided into three memory regions. The first, *Generic Region Cache* (*GRC*), is for all memory transactions unrelated to the output vector c, including reads of A and b. It is a classic, *set*-associative cache and is required for the processor to operate normally, but is not the focus in this chapter. The other two regions, called *Dense Region Cache* (*DRC*) and *Sparse Region Table* (*SRT*), are able to perform *reductions* on c. The *DRC* works as a line-wise, *set*-associative *reduction cache* to hold high density data in the *In-Band Region* of the matrix (*IBR*, *orange*). The difference with the *GRC* is that it is able to perform *reductions* and it is dedicated to the *banded* region, so that spurious accesses outside of the band do not provoke undesired evictions. Finally, the *SRT* works as an element-wise, fully-associative table whose role is to handle sparse regions, such as the *Out-of-Band Region* in the matrix (*OBR*, *blue*). This region is well-suited for handling isolated data and reducing memory traffic, with further analysis of this design choice presented in the next chapter.

Figure 3.4 shows a simplified view of a memory hierarchy with SpDCache. The *Processor Interface* (*PI*) takes *reduction* instructions, *RED(key, value, region)*, from the processor. The *RED* instruction contains a 'region' field which indicates whether the value is expected to be in a dense region (*DRC*) or a sparse region (*SRT*). As we will see, the hardware may override the software selection to avoid duplication. The *DRC* communicates with the memory hierarchy

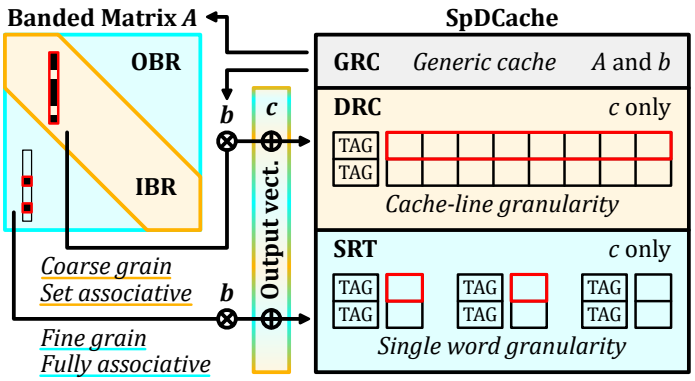


Figure 3.3 – Data-Cache Partitioning into Regions (*GRC*, *DRC*, *SRT*)

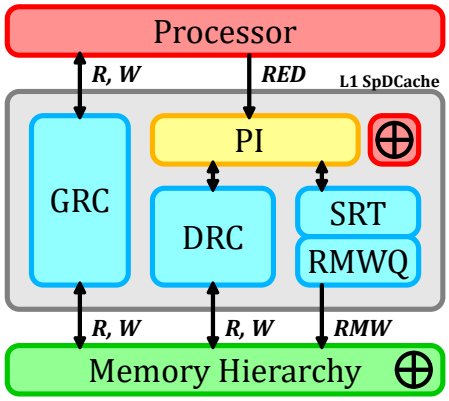


Figure 3.4 – Simplified View of the Memory Hierarchy with SpDCache

using read and write transactions over entire cache-lines, to propagate the local *updates* to the downstream memory. As *SRT updates* are for isolated elements, it does not make sense to bring a full line into the L1 to modify a single word. The *SRT* thus propagates *updates* by issuing atomic, element-wise *Read-Modify-Write* transactions (*RMWs*), which must be processed remotely and close to the main memory. This is a stringent requirement for the memory hierarchy, but it is realistic. Indeed, existing protocols such as AXI-4 [2] already support atomic transactions for certain operations (logical, integer add, min, max) and these could be extended to include floating point addition. Moreover, this requirement becomes more manageable when extending SpDCache to multilevel cache hierarchies (see Chapter 6). The *RMWs* from the *SRT* are temporarily buffered in a small queue (*RMWQ*), to avoid congestion.

SpDCache is driven by three key design principles. First, we ensure a given *key* is in only one region at a time, to avoid coherency problems. Second, the *DRC* is given priority over the *SRT*, to enable coalescing. Third, we avoid bringing a full line from memory to the L1 to simply update one or a few entries.

In Figure 3.5, we describe the operation of SpDCache, via a series of scenarios, each shown in a different color. When a new *reduction RED(key, value, region)* is received from the processor, we test for a hit in the *DRC*, regardless of the *region* argument. If it hits, the value (case `0b` – *turquoise*) can be accumulated in the corresponding cache-line of the *DRC*. We give this priority to the *DRC* as part of the mechanism to avoid duplication.

If there is a miss in the *DRC*, then the target region is determined based on the *region* argument of the *RED* transaction. If it is *IBR*, then it is necessarily a miss case, and the value (`02` – *orange*) must be inserted into a free cache-line of the *DRC*. When we allocate this line, we ensure that it does not create a duplication of existing entries in the *SRT*. If there is duplication, as in our example, these entries (`00`, `02`, `06`) are deleted from the *SRT* and merged into the newly allocated cache-line ($set_1, line_1$).

If the target region is the *OBR*, then the *reduction* (`3d` – *purple*) is processed in the *SRT*, either allocating a new entry (miss case) or accumulating in cache with an existing entry (hit case).

Now, we describe the eviction scenarios. In the *DRC*, the line to evict is selected ($set_2, line_2$ – *pink*), a read to main memory is performed, the values from the evicted cache-line are accumulated and a write to main memory is issued. If the evicted cache-line only has a

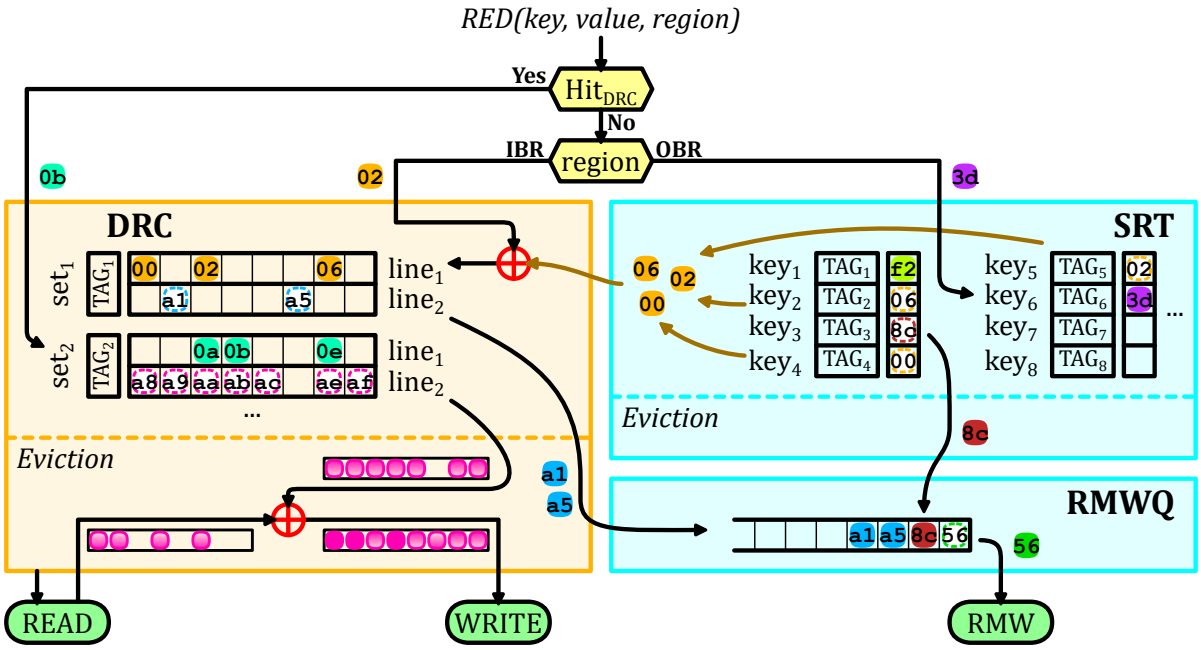


Figure 3.5 – Internal Operation of SpDCache with Examples

few non-zero elements and the *RMWQ* has enough free entries, we enter into a special case. To avoid bringing a full line from main memory, the non-zero values (`a1` and `a5` – *blue*) are converted to atomic *RMWs* and pushed onto the *RMWQ*. The decision to convert evicted cache-lines to atomic *RMWs* is based on a configurable density threshold. PHI [82], an accelerator similar to SpDCache, used a threshold of one non-zero word per eight words in a cache-line. Given our queuing mechanism to handle congestion, we use a more aggressive threshold of three non-zero words per cache-line. This threshold is chosen empirically, but its specific value has minimal impact, as our evaluation shows that very few elements are transferred through this mechanism.

The width of the *band* (w_B), must be selected so that one column of the *band* remains blocked in the *DRC*, otherwise, this region of the cache is always thrashing. This is our approach to selecting w_B, which we detail in the next chapter. Note that this approach selects w_B based on the *DRC* size, independently of the matrix and its structure.

In the *SRT*, an eviction simply consists of moving the element from the table to the *RMWQ* (`8c` – *brown*). If the *RMWQ* is not full, *SRT* evictions are issued in advance when the *SRT* table is nearly full, using another configurable threshold set to the *SRT* size minus half the size of the *RMWQ* in our experiments. The *RMWQ* issues atomic *RMWs* when it is not empty (`56` – *dark green*).

NOTE

We make the assumption that memory regions in the *GRC* and the *DRC*/*SRT* are disjoint. Currently, we do not manage coherency between the two, although this could be a future extension.

NOTE

SpDCache is not limited to banded matrices. Any sparse matrix with a denser region can benefit from this architecture, by adapting the definition of the *band* accordingly, and mak-

ing sure that the blocking effect in the *DRC* is preserved. The selection of where the dense region is located is done in software via the region argument of the *RED* instruction.

CONCLUSION

SpDCache is a data-cache splits in **three regions**:

- *GRC*, a **generic** cache region which handles the reads of A and b and any other memory accesses performed by the processor,
- *DRC*, a *reduction cache* region dedicated to *reductions* on c when computing the dense region of the matrix, which is the **band** (*IBR*) in the cases of banded matrices,
- *SRT*, a *reduction cache* region dedicated to *reductions* on c when computing the sparse regions of the matrix, which correspond to the **out-of-band** elements (*OBR*) for a banded matrix.

The two last regions use **memory transactions adapted to the region density** (line-wise or element-wise) to avoid transferring unnecessary data.

3.6 High Level Evaluation of SpDCache

To validate the SpDCache architecture, we simulate its behavior to assess memory traffic gains and explore various parameters. Combined with the theoretical analysis presented in the next chapter, this approach enables us to validate both the principle and performance of SpDCache. We modelled the behavior of SpDCache based on the flowchart in Figure 3.5, using a transaction level model in Python, for a single-core, single-thread system executing an OP SpMV kernel (Algorithm 3.1). We do not model timing, as the order of the memory accesses determines the behavior of the cache (hits, misses, evictions), which depends only on the SpMV algorithm. The model checks the final numerical result and reports the main memory traffic associated with the *updates* to the output vector (*Output Memory Traffic*), where we count the bytes transferred (64B, 8 words of 8B, for each read and write, and 8B per atomic *RMW*, with our configuration). For each write transaction, it tracks how many words in the line were actually used (*Evicted Cache Line Utilization*) and it tracks how many words in the cache held useful data (*Cache Memory Utilization*). We consider a word in the cache to be useful if it is updated or inserted by the processor. Our goal is to validate the principle of SpDCache using real application matrices from the SuiteSparse Matrix Collection [63].

As baselines, we implemented a fully generic cache (GC) that handles reads and writes for both inputs (A, b) and output (c), following the data movements shown in Figure 3.1a, and a simple *reduction cache* (RedC) that processes *RED* transactions in a dedicated region using the flow shown in Figure 3.1b. In the RedC, 25% of the memory is allocated for the inputs (A and b) and the other 75% for *REDs* on c. In SpDCache (SpDC), 25% is allocated for the *GRC*, 50% for the *DRC* and 25% for the *SRT*. In the next section, we present more results with varying region sizes, confirming that this allocation is indeed a good compromise. All three caches use a *Least Recently Used* (*LRU*) eviction policy, except the *SRT* region of SpDCache which uses a *random* policy. All three caches have a total data storage of 32 KiB, not including the storage. The *set*-associative regions have 4 *ways* and 64B cache-lines (8 *double* words per line).

We simulated 48 matrices that have been used by the state-of-the-art accelerators [7, 46, 87, 103, 105, 126, 127], which are listed in Appendix D. We excluded matrices whose output vector c fits entirely in the cache (fewer than 4096 elements), since they do not stress

the cache. In Table 3.1, we report the results for all three caches, along with statistical indicators. Most, but not all, of the matrices have a dense band and even some matrices without an obvious band benefit from the blocking effect in the *DRC*.

Table 3.1 – Decrease in Memory Traffic and Improvement in Cache Utilization from Python Simulation

	Output Memory Traffic			**Evicted Cache Line Utilization**			**Cache Memory Utilization (%)**		
	RedC	**SpDC**		**GC**	**RedC**	**SpDC**	**GC**	**RedC**	**SpDC**
	%/GC	*%/GC*	*% RMW*	*Avg*	*Avg*	*Avg*	*Avg*	*Avg*	*Avg*
Geometric Mean	**89.2**	**24.5** [a]	40.5	3.60	3.56	7.21 [b]	36.4	45.8	75.2 [c]
Standard Deviation	29.8	20.9	33.8	2.45	2.77	2.08	10.9	32.3	18.6
Minimum	28.5	7.00	0.00	1.06	1.00	4.36	16.6	11.9	30.0
Maximum	172	98.4	100	8.00	8.00	8.00	49.6	97.5	98.9
First Quartile	76.0	11.4	35.9	1.76	1.73	6.03	28.4	22.0	65.5
Third Quartile	115	46.6	91.7	6.60	7.65	8.00	48.6	88.8	93.9
Outperform GC	45.8	100	Ø	Ø	45.8	91.7	Ø	64.6	100
Outperform RedC	Ø	93.8	Ø	Ø	Ø	85.4	Ø	Ø	77.1

[a] *Decrease by 4.1× compared to GC, and 3.6× compared to RedC.*
[b] *Increase by 2.0× compared to GC, and 2.0× compared to RedC.*
[c] *Increase by 2.1× compared to GC, and 1.6× compared to RedC.*

As expected, the simple *reduction cache* (RedC) decreases the main memory traffic to 89.2% of the GC baseline (10.8% decrease). SpDCache has significantly better performance than RedC, reducing the traffic to 24.5% (75.5% decrease). SpDCache thus achieves a 4.1× decrease in memory traffic compared to GC, and a 3.6× decrease compared to RedC, on average over the set of matrices. SpDCache outperforms GC on all matrices, and RedC on 93.8% of them.

The benefits of SpDCache can be partly explained by the improved utilization of the cache-lines. On average, a cache-line evicted from SpDCache has 7.21 modified words, compared to 3.56 and 3.60 with the baselines. This improved utilization is the result of blocking the dense region in the *DRC*, where multiple *reductions* have the time to accumulate in the cache before being evicted. The *SRT* also contributes to this effect, as it captures irregular accesses outside the band, which would otherwise evict useful data from the *DRC*. The overall cache memory utilization is also significantly improved, with 75.2% of the cache holding useful data in SpDCache, compared to 45.8% and 36.4% for RedC and GC, respectively.

We need to consider an additional metric, the percentage of traffic sent as atomic *RMW* transactions (% RMW). Indeed, if this value is too high, it means we have simply shifted the workload to the lower level caches, which inherently have limited compute capability. On average, 40.5% of the memory traffic of SpDCache is composed of atomic *RMW*s, which is we consider acceptable given the significant overall reduction in memory traffic. However, this ratio varies widely between matrices. There are some matrices where most of the traffic is composed of atomic *RMW*s, meaning that most of their non-zero values are outside the band. For such matrices, which are basically not banded, SpDCache still greatly decreases the memory traffic, but better performance could be achieved by choosing a matrix-fitting region partitioning strategy. For example, the matrix *cit-Patents* has no elements in the band, and all non-zero values are scattered under the bottom part of the matrix (an unusual extreme

case). While having a memory traffic decreased to 7.2% compared to GC, it could benefit from a better use of the *DRC* by choosing a horizontal band covering the non-zero elements, with correct sizing to leverage the blocking effect. On the other hand, some matrices have very low *RMW* usage, indicating that most non-zero values are within the band and efficiently handled by the *DRC*. In this case, 25% of the available memory was unused. The *SRT* size could be adjusted to gain performance.

CONCLUSION

SpDCache achieves a 4.1× **decrease in memory traffic** on the output vector c compared to a generic cache, over a set of 48 matrices. It also **outperforms a simple *reduction cache*** by 3.6×, showing the importance of having separate dedicated cache regions for the in- and out-of-band of a matrix.

3.7 State-of-the-Art Comparison and Region Sizing

To compare SpDCache with the state-of-the-art, we simulated PHI [82], the existing solution that is closest to our proposal, using the same methodology as in the previous section. PHI is a *reduction*-aware cache that uses a single region to process *RED* transactions (see Section **??** from previous chapter). In PHI, the cache is not separated into regions, however, it treats load/store and *RED* operations differently. It also handles isolated elements, but do not consider the matrix structure. Unlike SpDCache, PHI relies on the *deferred update* version of the `OP` SpMV algorithm (see Algorithm 1.8). We thus adapted our simulation to use this algorithm for PHI. To ensure a fair comparison, we allocated the same amount of memory to PHI as for SpDCache and the baselines (32 KiB). In Table 3.2, we report the results for PHI alongside our previous results (columns nº 1, 2, 5 and 6).

Table 3.2 – All Results from Python Simulation

GMeans (48 mat.)		**GC**	**RedC**			**PHI**	**SpDC**					
sets	GRC	***128***	***32***	*8*	*4*	***128***	***32***	*8*	*8*	*4*	*4*	*4*
	DRC	***Ø***	***96***	*120*	*124*		***64***	*104*	*112*	*92*	*116*	*122*
	SRT	***Ø***	***Ø***	*Ø*	*Ø*		***32***	*16*	*8*	*32*	*8*	*2*
Output Memory Traffic (% over GC)		100 ■1.0	89.2 ▼1.1	81.6 ▼1.2	80.3 ▼1.3	50.4 ▼2.0	24.5 ▼4.1	24.3 ▼4.1	25.5 ▼3.9	23.2 ▼4.3	25.3 ▼4.0	32.0 ▼3.1
Ratio of RMWs in Memory Traffic (%)		Ø	Ø	Ø	Ø	Ø	40.5	35.7	36.5	36.5	36.2	41.1
Total Memory Traffic (% over GC)		100 ■1.0	93.9 ▼1.1	91.9 ▼1.1	93.2 ▼1.1	98.8 ■1.0	56.1 ▼1.8	56.6 ▼1.8	56.9 ▼1.8	57.7 ▼1.7	58.2 ▼1.7	59.2 ▼1.7
Evicted Cache Line Utilization (max: 8)		3.60 ■1.0	3.56 ■1.0	3.66 ■1.0	3.67 ■1.0	5.00 ▲1.4	7.21 ▲2.0	7.21 ▲2.0	7.23 ▲2.0	7.22 ▲2.0	7.22 ▲2.0	7.25 ▲2.0
Cache Memory Utilization (%)		36.4 ■1.0	45.8 ▲1.3	47.0 ▲1.3	47.1 ▲1.3	30.6 ▼0.8	75.2 ▲2.1	68.3 ▲1.9	65.0 ▲1.8	72.5 ▲2.0	65.1 ▲1.8	60.9 ▲1.7
Column nº		*1*	*2*	*3*	*4*	*5*	*6*	*7*	*8*	*9*	*10*	*11*

As expected, PHI (nº 5) achieves a significant reduction in memory traffic compared to the GC (nº 1), a 50.4% reduction on average. This represents a 2.0× decrease compared to the GC, which is consistent with the results reported in the original PHI paper [82], and a 1.8× decrease compared to the RedC (nº 2). However, SpDCache (nº 6) still outperforms PHI, cutting

memory traffic by 2.1×. The improved performance of SpDCache can again be explained by the better utilization of the cache-lines, with 7.21 useful words per evicted line, compared to 5.00 for PHI. The overall cache memory utilization is also significantly better with SpDCache, with 75.2% of the cache holding useful data, compared to only 30.6% for PHI.

As explained in Chapter 2, PHI mixes the 'immediate updates' and 'deferred updates' strategies for **OP** SpMV. In the first phase of the algorithm, it processes *RED* transactions as 'immediate updates', and PHI handles them with a mechanism similar to a classical *reduction cache*. The main differences are that the *GRC* and *DRC* are merged, which can cause unnecessary evictions of useful data, and more importantly, PHI uses a batching mechanism at eviction to 'defer' some updates to the second phase. When a cache-line is evicted and contains fewer non-zero entries than a fixed threshold, PHI batches the entries in *partial outputs* (*POs*) defined by the software and stored in memory. These *POs* are processed in the second phase of the algorithm, as in the 'deferred updates' variant. If the threshold is set too high, PHI batches many entries and increases second-phase memory traffic. If it is set too low, PHI batches few entries, reducing batching effectiveness and increasing the number of 'immediate updates', making PHI behavior closer to a *reduction cache*. PHI sets this threshold to one non-zero word per eight words in a cache-line, meaning that for each batched evicted line, PHI reduces bytes transferred by 4× (4 $(word, address)$ tuples fitting in a single 64B cache-line, instead of 4 full 64B lines with 7 unused words each). However, we observed that, on the geometric mean over the 48 matrices, only 4.2% of PHI's output memory traffic comes from these batched updates (maximum 28.6%), indicating that PHI's overall behavior remains close to a *reduction cache*. In contrast, SpDCache addresses the drawbacks of a *reduction cache* by using dedicated regions for denser and sparser accesses, avoiding unnecessary evictions, without incurring the additional second-phase traffic introduced by PHI's batching mechanism.

We also explored different region sizing for SpDCache, varying the sizes of the *GRC*, *DRC* and *SRT* while keeping the total cache size constant at 32 KiB. The results are also reported in Table 3.2 (columns n° 7 to 11). Reducing the size of the *GRC* from 8 KiB (32 *sets*) to 2 KiB (8 *sets*) and 1 KiB (4 *sets*) has little impact on performance, as the *GRC* is mainly used for temporary storage of input vector elements. However, reducing the *GRC* under 1 KiB (4 *sets*) increases the total memory traffic, as more frequent evictions occur before the data is used, an effect already noticeable when going from 2 KiB to 1 KiB. With constant *GRC* size, when reducing the size of the *SRT*, which means allocating more space to the *DRC*, we observe a slight increase in memory traffic, as the *SRT* becomes less effective at capturing irregular accesses. Thus, the *SRT* sizing appears to be more critical than the *DRC* sizing. We will explore the reasons behind this effect in the next chapter. Overall, fine tuning the region sizes can lead to small performance improvements, but the original sizing of 8 KiB for the *GRC*, 16 KiB for the *DRC* and 8 KiB for the *SRT* appears to be a good compromise for the set of matrices we evaluated.

We also explored different sizing strategies for the RedC baseline (columns n° 3 and 4), varying the size of its single *reduction* region while keeping the total cache size constant at 32 KiB. Allocating more than 75% of the cache (96 *sets*) to store *RED* transactions slightly improves performance, as it increases the likelihood of capturing multiple updates to the same output element before eviction, without overly compromising the storage of input elements.

CONCLUSION

SpDCache outperforms PHI, a state-of-the-art *reduction* cache, achieving a 2.1× decrease in memory traffic, due to the dedicated regions which improve the effectiveness of the cache in dense regions, and the use of external atomic accesses to reduce the cost of isolated updates. Additionally, we show that the initial region sizing of SpDCache (25/50/25%) provides good performance with only minor variations observed when adjusting the sizes of the *GRC*, *DRC*, and *SRT*.

3.8 Conclusion

We have presented the concept of SpDCache, a *reduction cache* that has optimized storage and compute techniques for the sparse and dense regions of banded matrices. This approach does not reduce the number of compute operations, however, it significantly decreases memory traffic (4.1×). These benefits can be attributed to three techniques: (1) blocking the dense part of the band in the *DRC* to take benefit of the existing spatial locality, (2) using fine-grained storage in the *SRT* to adapt to isolated elements, and (3) off-loading the *reductions* for isolated elements to the downstream caches to prevent local evictions and reduce traffic.

Exploiting the coarse-grain structure of matrices is key to designing hardware optimized for sparse matrix kernels. In the next chapter, we present a more theoretical analysis of the behavior of SpDCache before moving onto an actual RTL implementation in Chapter 5.

TAKEAWAYS

Reduction cache:

- Provides hardware support for performing ***reductions***: $RED(key, value)$.
- The accumulations are done **near memory**, in the cache.
- **Reduces memory traffic** for kernels with a large number of *reduction* updates.

SpDCache:

- Considers that the matrix has a **dense region** and a less dense, **sparse region**. For banded matrices, these correspond to the in-band (*IBR*) and the out-of-band (*OBR*).
- Divides the L1 data-cache into **three regions**:
  - ***GRC***, generic region cache for regular memory accesses (such as reading A and b);
  - ***DRC***, dense region cache for *reductions* in the dense region of the matrix;
  - ***SRT***, sparse region table for *reductions* in the sparse region of the matrix.
- Uses memory transactions adapted to the region density: **cache-line-wise versus element-wise** for the *DRC* and the *SRT*, respectively.
- For a set of 48 matrices, results in **4.1× less memory traffic** for `OP` SpMV kernel.

Chapter 4

Analysis of Reduction Cache Behavior and Parameter Exploration

Contents

4.1 Introduction

SPARSE matrix processing exposes processor systems to irregular memory access patterns, which degrade cache efficiency and increase memory traffic. The magnitude of these effects on memory system performance remains challenging to predict due to the diversity of data patterns. In this chapter, we develop a theoretical model to analyze and optimize the behavior of a *reduction cache* when executing an *Outer-Product* (`OP`) SpMV kernel.

We propose a hybrid probabilistic/fast simulation model that predicts the memory transactions of a *reduction cache* when executing an `OP` SpMV kernel ($A \times b = c$). This model allows us to quickly explore the impact of different cache configurations on memory traffic, using banded matrices as a representative case study. By isolating the sequential reads of the input data and analyzing the density distribution of non-zero elements, we identify key design principles for optimizing cache performance.

The chapter demonstrates the importance of separating the in-band and out-of-band regions of the matrix into dedicated cache regions. This separation allows the cache to exploit

the structural regularity of banded matrices while efficiently handling irregular accesses in sparser regions. The theoretical analysis validates the architectural choices of SpDCache that was presented in previous chapter and provides insights for sizing and configuring cache regions to minimize memory traffic.

In our analysis of *reduction caches*, we use a set-associative configuration, with 8 byte elements which we consider as the basic memory unit. s_C is the size of the cache, in elements. The cache is divided into *sets*, *ways* and cache-lines, of size S, W and l, respectively. Thus, $s_C = S \times W \times l$ elements. We recall that a banded square matrix of dimension M is defined by its band width w_B, or its half-band width w_{hB}, such that $w_B = 2w_{hB} - 1$. The in- and out-of-band densities are named d_{IB} and d_{OB}, respectively. When d_{OB} is zero, the matrix is said to be 'perfectly' banded.

4.2 Isolating the Input Reads in a Generic Cache

WHEN computing an **OP** SpMV kernel, all the irregular accesses come from updating the output vector c. Indeed, the input matrix A and vector b are read sequentially, and can thus be efficiently cached by a generic cache that naturally exploits spatial locality.

With the matrix A stored in CSC format, each iteration of the **OP** algorithm necessitates reading four tables: $A.ptr$, $A.idx$, $A.val$ and b. In the same iteration, a fifth table, c, is updated "randomly". If we consider cache-lines of size l elements, then each cache-line read of the $A.ptr$ table brings l consecutive entries, which are pointers to l consecutive columns of the matrix. Similarly, each cache-line read of the b table brings l consecutive entries, which will be multiplied by the non-zero elements of l consecutive columns of the matrix. These reads are needed in the outer-loop of the **OP** algorithm. In the inner-loop, each cache-line read of the $A.idx$ and $A.val$ tables brings l consecutive non-zero elements of the matrix to process, which will generate l "random" updates to the output vector c, thus l different cache-lines of c could be accessed in the worst case.

To avoid unnecessary evictions, at each iteration, the generic cache should, obviously, keep $A.ptr$, $A.idx$, $A.val$ and b cache-lines up until they are no longer needed. That means l iterations for $A.idx$ and $A.val$, and around $l \times nnz_c$ iterations for $A.ptr$ and b. However, the cache must also keep the cache-lines of c that are being updated, which, over the course of $l \times nnz_c$ iterations, can necessitate up to the same number of different cache-lines in the worst case, on unpredictable addresses. Thus, it would be difficult for a generic cache to keep all the useful data until it is no longer needed. $A.ptr$ and b cache-lines, which are accessed once every nnz_c iterations, could easily be evicted by a *LRU* policy which is unaware of their future use. By isolating the input reads in a dedicated cache, as proposed with SpDCache and the *reduction cache* baseline in the previous chapter, we greatly increase the chances that all the cache-lines of A and b remain in the cache until they are no longer needed. With a dedicated cache region for the inputs, the *GRC* introduced in previous chapter, only the four sequentially read tables need to be considered. These tables are read only once, and isolating them in the *GRC* decreases the risk of undesired evictions.

With a *LRU* policy, cache-lines holding $A.ptr$ and b are more at risk of being evicted than $A.idx$ and $A.val$ because they are accessed less frequently, only once every $\sim \frac{nnz_c}{l}$ times per column. $A.idx$ and $A.val$ are accessed every iteration of the inner-loop. To avoid premature evictions of $A.ptr$ and b, the *GRC* must be sized to handle the $\frac{nnz_c}{l}$ cache-line reads which occur while computing a column. In Equation 4.1, we show the minimum size of the *GRC* in elements needed to avoid unnecessary evictions during the traversal of the columns, thus optimizing memory traffic.

$$s_{C_{min}}(GRC) = 2 \times l + 2 \times nnz_c \tag{4.1}$$

This relationship is not exact, as it does not consider address alignment effects nor temporal variability in the downstream memory, but it gives a good approximation of the minimum size needed for the *GRC* to avoid unnecessary evictions. It explains why, in the previous chapter, the total memory traffic slightly drops when the *GRC* size is decreased. If we return to Table 3.2, in the previous chapter, in the columns for SpDC, we now understand why the total memory traffic is higher when the *GRC* size is reduced from 32 to 4 *sets*.

Separating the inputs in a dedicated cache region also has the advantage of allowing prefetching techniques [98] to be used efficiently, as prefetched cache-lines will not be unde-

sirably evicted before use. For the **OP** SpMV kernel, prefetching does not reduce the memory traffic, but it does improve performance by masking the main memory latency.

CONCLUSION

To avoid unnecessary evictions, it is advantageous to **separate** the sequential reads of the inputs A and b from the "random" updates of the output c, into **dedicated cache regions**. A generic cache of reasonable size can **mask the latency** of the main memory when reading A and b, and ensure this data is **read exactly once**.

4.3 Modeling a Reduction Cache

To go beyond simply observing the behavior of a specific accelerator processing a given matrix, a model of the matrix and the cache is required. We present a hybrid probabilistic/fast simulation model for a *reduction cache*, processing **OP** SpMV, which predicts the volume of memory traffic issued by the L1 cache, starting from a matrix density distribution and a cache configuration.

The model represents the non-zero density both in the matrix and in the cache, with cache-line granularity. Using these coarse grain densities, a fast simulation of the **OP** SpMV is performed. The state of the cache can be predicted at each step of the simulation. By counting the evictions, we evaluate the output memory traffic, the memory trafic of the output vector c. We can thus predict the total number of memory accesses for different types of matrices and quickly explore different *reduction cache* configurations.

We do not consider the sequential accesses to the input matrix A and vector b in our model. Indeed, the matrix tables val, idx and ptr, and the vector b are each read once, giving a total memory traffic of $2nnz+(M+1)+M$ elements. We showed in the previous section that a simple generic cache of reasonable size is sufficient to minimize the associated memory traffic.

Our model uses the cache configuration parameters S, W and l. Although our model is not limited to banded matrices, they are the focus of this work. The matrix density distribution thus takes on two different values, either d_{IB} in the band, or d_{OB} outside of the band.

We normalize the memory traffic by dividing the number of elements that are read or written by the number of non-zero elements (nnz) in the matrix, so we can compare performance between matrices of different sizes and densities. Indeed, we can interpret the normalized memory traffic, $normMT$, as the number of elements transferred from/to memory per non-zero matrix element. If the memory traffic remains constant while the matrix density increases, we expect $normMT$ to decrease.

4.3.1 Operation of the Analytical Model

Matrix Representation The square matrix is of dimension M, as represented in Figure 4.1. Its rows and columns are indexed by $0 \leqslant i, j \leqslant M-1 \in \mathbb{N}$. Each column is split into vertical-strips of size l, where l is the cache-line size. Indeed a vertical-strip is a portion of a column j of size l, from row-indices i to $i+l-1$, as illustrated by the *red* rectangle within the matrix in the figure. Within a column, vertical-strips are indexed by $k \in [\![0, M_l - 1]\!]$ [1], with $M_l = \lceil \frac{M}{l} \rceil$, and $k = \lceil \frac{i}{l} \rceil$.

1. We use $[\![a, b]\!]$ for the integer intervals, and $[a, b]$ for real intervals.

The matrix is thus represented as a static, dense two dimensional table, MDT (*Matrix Density Table*), of size (M_l, M) that contains the density in $[0, 1]$ of each vertical-strip. We assume that the distribution of the non-zero elements within a vertical-strip is uniform, as assumed in related work [109] where the authors studied a directly-mapped generic cache for an *Inner-Product* SpMV kernel, only for the case of 'perfectly' banded matrices.

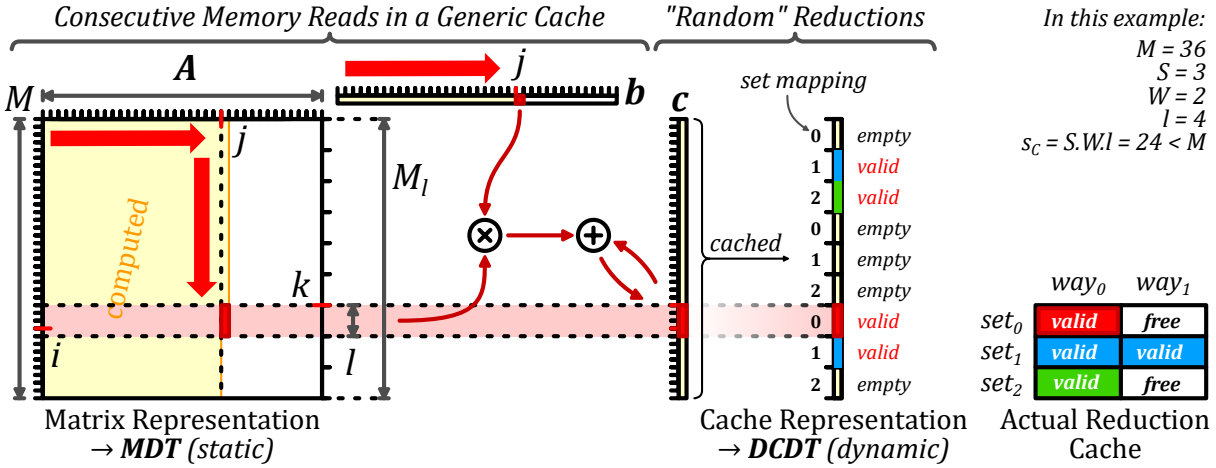


Figure 4.1 – Input Matrix and Cache Representation of the Model

Cache Representation Still in Figure 4.1, the cache is represented as a single column, which maps directly to the output vector c. This "column-representation" of the cache is partitioned into cache-lines, corresponding directly to the vertical-strips of a given column of the matrix, as shown by the horizontal *light red* rectangle. A cache-line $k \in [\![0, M_l - 1]\!]$ is stored in a unique *set* $s \in [\![0, S - 1]\!]$ of the cache such that $s = k\%S$. Unlike the matrix representation which is static, the cache representation is continuously updated as the model evaluates **OP** SpMV.

Of course, the real cache-size ($S \times W \times l$) is not necessarily equal to the output vector size (M). Thus, at a given time, the number of valid (non-empty) cache-lines contained in the cache representation must not exceed the actual cache-size. Specifically, we ensure the number of valid cache-lines in a *set* of the cache representation does not exceed the number of ways W. In the example in Figure 4.1, the actual cache is holding four valid cache-lines, which are the only valid cache-lines present in the cache representation.

To count the number of valid cache-lines, we further distinguish them from empty ones. For this reason, we set the cache-line densities to be either zero (the cache-line is empty), or a density is in the range $\left[\frac{1}{l}, 1\right]$ (the cache-line is valid). A valid cache-line has a density between $\frac{1}{l}$ and 1, corresponding to the probabilistic number of non-zero elements.

The state of the cache is thus represented with the dynamic table $DCDT$ (*Disjoint Cache Density Table*), of dimension M_l that contains the disjoint-density in $\{0\} \cup \left[\frac{1}{l}, 1\right]$ of each cache-line, at a given iteration. Initially, all cache-lines are set to zero density, since the cache is considered empty.

Iterative Process In the fast simulation phase, the analytical model [33] iterates over the vertical-strips in the matrix, following the dense traversal of the **OP** algorithm, as described in Chapter 1 and shown with *red* arrows in Figure 4.1. In a given iteration (column j, vertical-

strip κ), we use the matrix density of the current vertical-strip, $MDT_{\kappa,j}$, and the current state of $DCDT$ to determine if an eviction occurs, and update it for the next iteration, if necessary.

Disjoint-Density of the Current Vertical-Strip The density of the current vertical-strip in the matrix has to be disjointed at each iteration, to determine whether the strip is empty or not, as done for the cache-lines. We define the vertical-strip disjoint-density as $d_{lM}(\kappa) \in \{0\} \cup [\frac{1}{l}, 1]$.

- When $MDT_{\kappa,j} = 0$, the vertical-strip in the matrix is empty, thus, $d_{lM}(\kappa) = 0$.
- When $0 < MDT_{\kappa,j} < \frac{1}{l}$, the vertical-strip can have either 0 or 1 non-zero elements. To resolve this, we maintain a running counter of the accumulated $MDT_{\kappa,j}$ values, and when this counter exceeds $\frac{1}{l}$, the current vertical-strip is considered non-empty, so we set $d_{lM}(\kappa) = \frac{1}{l}$. If the counter is below this threshold, $d_{lM}(\kappa) = 0$.
- When $MDT_{\kappa,j} \geqslant \frac{1}{l}$, the vertical-strip has one or several non-zero elements. Thus, $d_{lM}(\kappa) = MDT_{\kappa,j}$.

With this definition, we can distinguish empty ($d_{lM}(\kappa) = 0$) and non-empty ($d_{lM}(\kappa) \neq 0$) vertical-strips.

Eviction Processing With the above definitions, we can determine if a miss occurs at the current iteration. A miss occurs when the cache-line κ is not valid and the vertical-strip is not empty. Recall that in a *reduction cache*, a miss without eviction results in the allocation of a free cache-line, initially filled with zeros, and produces no memory traffic. We define $miss(\kappa) = (DCDT_\kappa = 0)\ and\ (d_{lM}(\kappa) \neq 0)$, a boolean indicating whether a miss occurs in the current iteration.

When allocating a free cache-line, it is necessary to ensure that the number of valid cache-lines in the *set* $s = \kappa \% S \in [\![0, S-1]\!]$ does not exceed W. The new allocation may thus trigger an eviction. Stated formally, $nnzcl_\kappa$ is the count of the cache-lines k from the *set* s that verify $DCDT_k \neq 0$. We define $evict(\kappa) = (miss(\kappa))\ and\ (nnzcl_\kappa = W)$, a boolean indicating whether an eviction occurs in the current iteration.

In the case of an eviction, for the probabilistic model, we use a *random* eviction policy to select a valid cache-line in the *set* s (when $DCDT_k \neq 0$). The index of the evicted cache-line is $e \in [\![0, M_l - 1]\!]$.

Cache Update The last step is to update the $DCDT$ before proceeding to the next iteration. (1) If a cache-line is evicted, we must account for the memory traffic and reset the victim line e; (2) The density of the current cache-line κ may increase if there is a hit or if it is newly allocated:

1. In the case of an eviction ($evict(\kappa)$ is true), we reset the victim cache-line (e) by setting $DCDT_e = 0$.
2. We update the density of the cache-line κ. In the case of a miss ($DCDT_\kappa = 0$), the cache-line density is set to the disjoint-density of the matrix vertical-strip κ. In the case of a hit, the cache-line and the vertical-strip densities are merged:
   $DCDT_\kappa \leftarrow DCDT_\kappa + (1 - DCDT_\kappa) \times d_{lM}(\kappa)$
   The updated cache-line density is either 0 or greater than $\frac{1}{l}$, thus respecting the constraint that the cache-line is either empty or valid.

Final Iteration Starting with an initially empty cache, we run all $M \times M_l$ iterations and count the total number of evictions ($nbEvict$). After the last iteration, we account for the memory transactions required to flush the cache ($nbFlush$), writing the valid cache-lines to memory, by counting lines where $DCDT_k \neq 0$, for $k \in [\![0, M_l - 1]\!]$. $nbFlush$ will never exceed $S \times W$.

In a *reduction cache*, evictions and flushes result in a line-wise memory read, a modification, and a line-wise memory write. The update memory traffic is $2l(nbEvict + nbFlush)$ elements, and $normMT$ is this value divided by nnz.

CONCLUSION

We have presented a **hybrid probabilistic/fast simulation analytical model** for a *reduction cache*, processing `OP` SpMV, which predicts the volume of memory traffic issued by the L1 cache, starting from a **matrix density distribution** and a **cache configuration**. The model represents the non-zero density both in the matrix and in the cache, with **cache-line granularity**. Using these coarse grain densities, a **fast simulation** of the `OP` SpMV is performed. In this way, we can quickly explore different *reduction cache* configurations to evaluate their impact on memory traffic.

4.4 Analysis of Reduction Cache Behavior

We now study the *reduction cache* behavior when computing `OP` SpMV, starting with the simple case of 'perfectly' banded matrices with varying band widths. We then use the analytical model to analyse the *reduction cache* behavior, varying the out-of-band density and the cache-size. We show the benefit of storing the in- and out-of-band of the matrix in separate regions of the cache, to minimize memory traffic.

4.4.1 Behavior of a Cache for a Perfectly-Banded Matrix

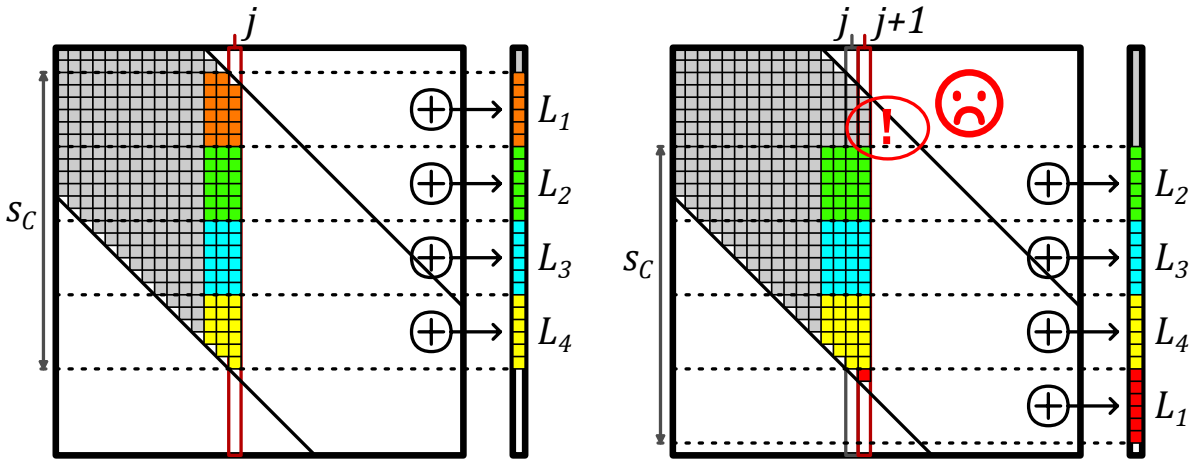
In the previous chapter, we presented SpDCache and a *reduction cache* baseline with a specific region (*DRC*) dedicated to storing the banded part of the matrix. We defined the bandwidth of the matrix based on the *DRC* size. In this section, we explain and justify this choice by analyzing the behavior of a *reduction cache* when processing an `OP` SpMV kernel with a 'perfectly' banded matrix.

We show that the *reduction cache* must be 'properly' dimensioned with respect to the band width of a 'perfectly' banded matrix, in order to avoid unnecessary evictions. Indeed, if the cache is too small compared to the band, evictions occur before the column iteration is over, leading to poor data-reuse for the next iteration. To achieve high data-reuse, we show that the cache-size (s_C) and the band width (w_B) must respect the relationship given in Equation 4.2.

$$s_C \geqslant w_B + l - 1 \tag{4.2}$$

We consider two examples with a cache having four cache-lines (L_1 to L_4) with $l = 6$. In Figure 4.2, we illustrate the case when the cache is too small ($s_C < w_B + 5$). In Figure 4.2a, we see the state of the cache after iterating up to column j. Then, in Figure 4.2b, we see the state after processing the next column, $j + 1$. The cache-line L_1 (*orange*) is evicted, and allocated to new elements at the bottom of the band, below the *yellow* area. In this case, there are still elements of the band above the *green* area, whose corresponding cache-line

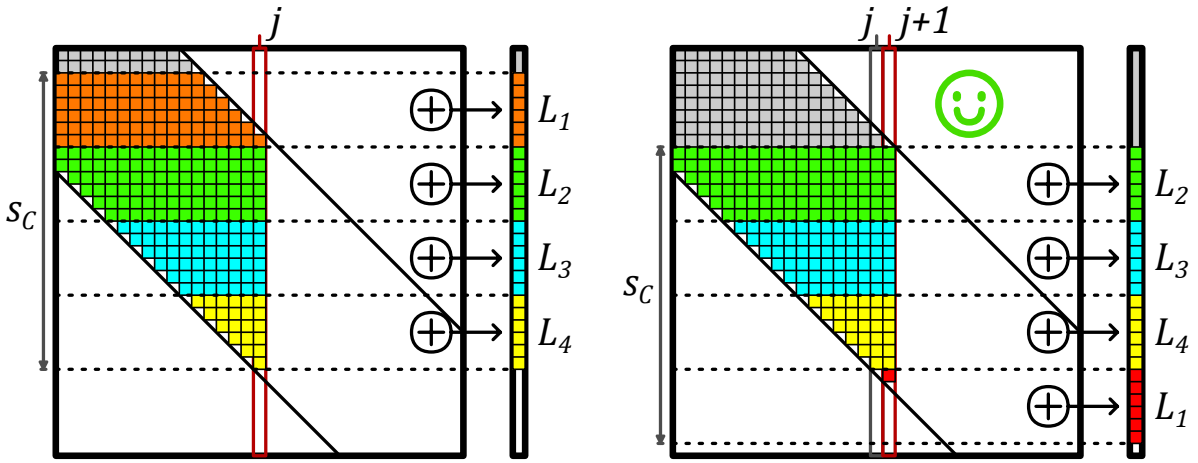
has, unfortunately, been evicted. Thus, on iteration $j+2$, cache-line L_1 must be fetched again and this pattern continues creating *thrashing*.



(a) Column iteration j **(b)** Column iteration $j+1$

Figure 4.2 – Example of the Eviction Rotation in a Perfect Band when the Cache is Undersized ($s_C < w_B + l - 1$): $s_C = 24$, $w_B = 23$ and $l = 6$

In Figure 4.3, we illustrate the case when the cache is 'properly' sized ($s_C \geqslant w_B + 5$). As in the previous example, Figures 4.3a and 4.3b show the state of the cache at iterations j and $j+1$, respectively. This time, the cache-line L_1 is evicted only when all elements of the band in the *orange* region have been processed. Indeed, the eviction of cache-line L_1 occurs just in time so that it can be allocated to the new region at the bottom of the band, below the *yellow* area. In this way, when processing columns, the cache-lines do a rotation where the top one is evicted and allocated at the bottom, with one eviction occurring after the processing of every l columns.



(a) Column iteration j **(b)** Column iteration $j+1$

Figure 4.3 – Example of the Eviction Rotation in a Perfect Band when the Cache is 'Properly' Sized ($s_C = w_B + l - 1$): $s_C = 24$, $w_B = 19$ and $l = 6$

If the cache-size is larger than required by Equation 4.2, the rotations and the eviction-rate are the same. To produce this perfect cache rotation, we assume an LRU eviction policy and dense vertical-strips, although, even with a *random* policy, the general pattern still holds.

This analysis shows that, with a "properly-sized" *reduction cache*, a 'perfectly' banded matrix provokes no unnecessary evictions and the cache-lines are fully utilized. With a given cache-size, Equation 4.2 defines the maximum band width that can be efficiently stored in the *reduction cache* or in the *DRC* of SpDCache. For example, with a cache-size of $s_C = 2048$ elements (16 KiB) and a cache-line size of $l = 8$ elements, the maximum band width is $w_B = 2041$ elements (which means a half-band of $w_{hB} = 1020$ elements).

Large, 'perfectly' banded matrices are rare in practice, especially with such narrow -width. In the next sections, we extend this analysis to banded matrices, studying the impact of out-of-band non-zero elements on the *reduction cache* behavior.

CONCLUSION

A *reduction cache* can achieve an efficient **rotating pattern** when processing a 'perfectly' banded matrix, provided that the cache-size and band width respect the **relationship given in Equation 4.2**: $s_C \geqslant w_B + l - 1$. This relationship defines the **maximum band width** that can be efficiently stored in the *reduction cache* for a given cache-size.

4.4.2 Impact of Out-Of-Band Density on Memory Traffic

We now apply the analytical model of Section 4.3.1 to study the performance of a *reduction cache* for banded matrices, processing an `OP` SpMV kernel ($A \times b = c$). We consider banded matrices of dimension $M = 1000$, with a half-band of size $w_{hB} = 125$ and an in-band density $d_{IB} = 1$. We analyse a 4-way set-associative cache ($W = 4$) with a cache-line size of $l = 8$ elements, with a *random* eviction policy. The size of the cache varies from $S = 1$ to $S = 32$ *sets*. When the cache-size reaches $S = 32$, the cache can hold the full output vector: $S \times W \times l \geqslant M$. We developed a C program which implements the hybrid probabilistic-fast simulation model and in Figure 4.4, we plot the normalized memory traffic, $normMT$, for the output vector c, depending on the out-of-band density d_{OB}, for varying cache-sizes.

On the right of the graph, we notice that $normMT$ reaches a peak when the out-of-band density equals $\frac{1}{l} = 0.125$. Indeed, when there is at least one non-zero element per cache-line, the number of main memory accesses approaches that required for a dense matrix. As the density increases further, no additional memory accesses are required, but each line becomes denser, thus $normMT$ drops, as seen in the graph in Figure 4.4.

For the plot corresponding to $S = 32$, the cache is large enough to store the entire output vector c, thus, there are no evictions and $normMT$ is constant, corresponding to the flush traffic at the end. This plot gives the lower bound of memory traffic for the output vector c.

The first takeaway from this graph is that variations in d_{OB} heavily impact $normMT$ (note the logarithmic scale). Isolated non-zero elements outside the band trigger misses and evictions of cache-lines corresponding to the banded region. Furthermore, when a cache-line is allocated for an isolated out-of-band element, most of the cache-line remains empty. In this case, instead of the efficient rotating pattern described in Section 4.4.1, the cache is *thrashing* with low utilisation. Regardless of the cache-size, the lower the out-of-band density, the lower $normMT$, as seen by the plots that tend downward toward the left of the graph. Avoiding the perturbations induced by the out-of-band elements would enable the cache to achieve minimal memory traffic, as with a 'perfectly' banded matrix.

Not surprisingly, at the left of the graph, $normMT$ varies depending on the cache-size. This is due to the relative size of the band and the cache. Indeed, if the matrix band width is large

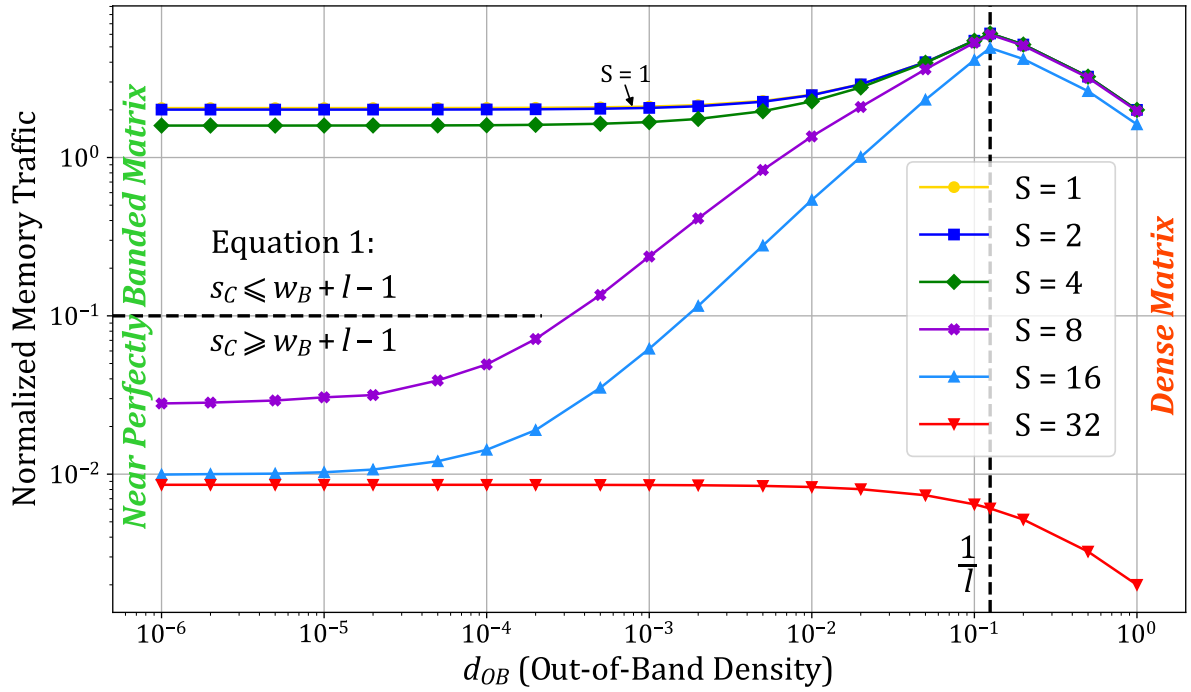


Figure 4.4 – Normalized Output Memory Traffic Depending on the Out-of-Band Density for Several Cache-Sizes

compared to the cache-size, processing one column of the band will generate many evictions. To avoid evictions in the band, the relationship given by Equation 4.2 in Section 4.4.1 must be fulfilled. Indeed, for this example, for $S \geqslant 8$, this relationship holds and the corresponding plots have significantly lower $normMT$, up to two orders of magnitude lower. Conversely, the first three plots ($S = 1$ to 4) do not respect this condition, explaining their high $normMT$. Thus, the second takeaway from this graph is that, given a cache memory size, there is an ideal band width that generates minimal $normMT$. Increasing the cache-size beyond this threshold only produces a minor reduction in memory traffic, between $\sim 9 \times 10^{-3}$ and $\sim 2 \times 10^{-2}$, since at the beginning of matrix traversal, the first eviction is deferred before reaching the steady state rotating pattern, as discuss in Section 4.4.1.

These observations explains the need of dividing the *reduction cache* into two regions. One which is optimized for storing the band and the other which handles all the other elements. The in-band region of the cache can thus operate with maximal efficiency corresponding to the bottom-left of Figure 4.4. In subsequent sections, we analyse the out-of-band traffic.

CONCLUSION

The out-of-band density heavily impacts the normalized memory traffic, $normMT$, as the isolated elements trigger unnecessary evictions, preventing the efficient rotating pattern shown in the previous section. Thus, to minimize memory traffic, it is important to **separate** the band and out-of-band regions of the matrix into **dedicated regions** of the *reduction cache*.

4.4.3 Design Analysis for Out-of-Band Region

For the type of large matrices under study, there are usually too many non-zero elements in the out-of-band region for them to be cached from one column to the next, thus making

it difficult to achieve a high-level of reuse from column to column. Therefore, the size of the cache for the out-of-band is not as important as the choice of associativity and data granularity. To justify this affirmation, we use the analytical model from Section 4.3.1 to analyse the performance of the *reduction cache* when processing only the non-zero elements outside of the band. As we have already discussed the handling of the in-band region, we now consider only the out-of-band elements, which is equivalent to setting the in-band density to $d_{IB} = 0$. Using the same matrix parameters as in the previous section, we explore different choices of cache-line size ($l \in \{1, 8\}$ elements), associativity (set-associative or fully-associative), memory transaction type (*R/W* or *RMW*), and total cache-size ($s_C \in \{128, 256\}$ elements). Because the out-of-band data is highly sparse, we want to explore having an element-wise cache ($l = 1$) instead of the typical line-wise cache ($l = 8$). As this cache region can be relatively small, a fully-associative configuration ($S = 1$, $l = 1$) is reasonable in terms of hardware implementation, and this full associativity enables more efficient storage of the data elements. Indeed, when dealing with sparse elements, it is wasteful to bring a full line into the cache to only modify one element and then evict it, which we denote as *R/W*. Thus, we consider offloading the *reduction* to a higher level cache, thus reducing unnecessary data transfers. We assume the higher level cache can support this element-wise operation which we denote as *RMW*, following the design choice of SpDCache in Chapter 3. We recall that protocols such as AXI [2] support atomic, element-wise transactions (AMOs), which could easily be extended for *reduction* operations.

To analyse the impact of the cache-size, we consider two different sizes (*solid* and *dashed*). For a given cache-size, we adjust the number of ways (W), based on the associativity, to keep the cache-size constant. In Figure 4.5, we plot $normMT$ for the output vector c, depending on d_{OB}, for varying cache configurations.

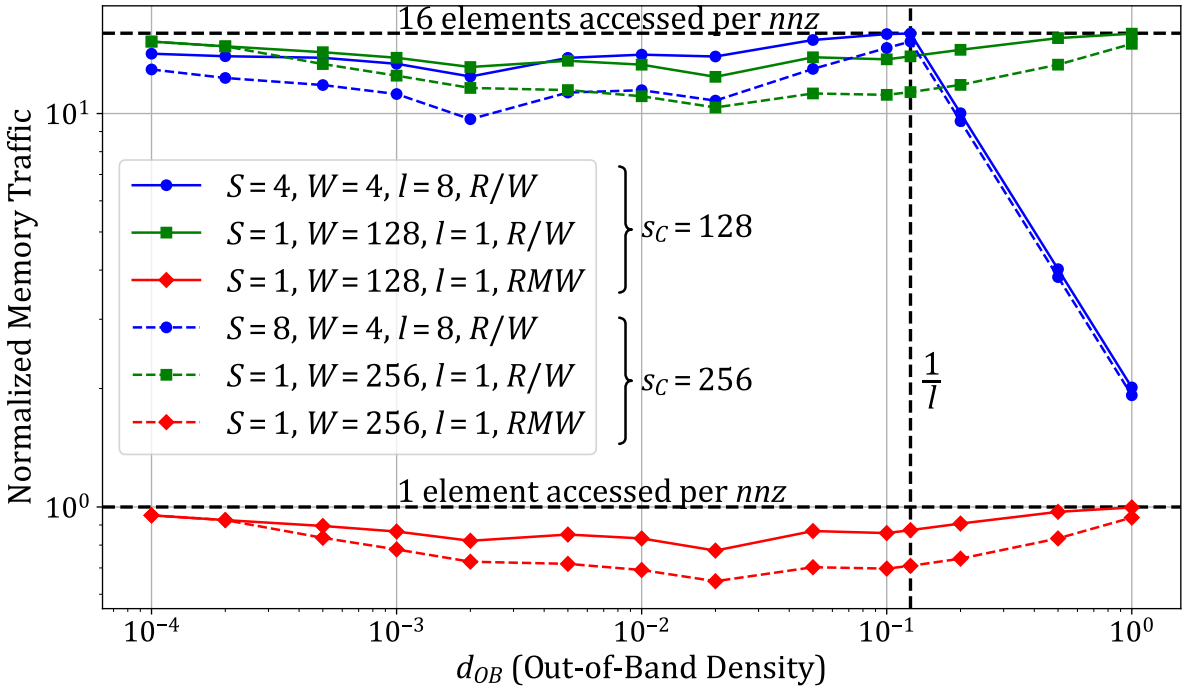


Figure 4.5 – Normalized Output Memory Traffic of the Out-of-Band, Depending on the Density for Several Cache-Sizes

Let us start by considering a classic set-associative cache with a line-wise memory interface (*solid blue*, *dot*), where we see a dependence of $normMT$ on d_{OB} on the right of the graph ($d_{OB} \geqslant \frac{1}{l}$). Indeed, we observe the same behavior as in Section 4.4.2, where the cache behavior approaches that seen for a dense matrix.

Looking at the plot for the fully-associative cache with a line-wise memory interface (*solid green*, *square*), we see that the normalized traffic is not impacted by the density of the out-of-band, as expected. With the fully-associative cache, $normMT$ is constant near 16, as, on average, there are two line-wise transactions per non-zero element processed, one read to fill the line, one write to evict it.

The *red* plots (*diamond*) correspond to a fully-associative cache with an element-wise memory interface, which can offload element-wise *RMW* transactions. We see that this approach results in over an order magnitude reduction in $normMT$. First, we avoid transferring full cache-lines which are mostly empty, and we only send the *RMW* in one direction ("fire-and-forget"). Indeed, $normMT$ is now reduced to below one transaction per non-zero element processed. The fact that this value is below one shows that this cache region does hit and reduce memory traffic. Increasing the cache-size from 128 (*solid*) to 256 (*dashed*) elements reduces $normMT$ by at most 10^{-1}, the hit-rate being slightly increased as expected.

CONCLUSION

For the out-of-band region of the cache, the most important characteristics are: (1) a **fully-associative** element-wise data-storage to avoid virtually empty cache-lines, and (2) an **element-wise memory interface** to reduce memory traffic. With these two characteristics, the normalized memory traffic can be reduced to below one transaction per non-zero element processed.

4.4.4 Impact of In-Band Density

Up to now in this chapter, we considered a dense band ($d_{IB} = 1$) to show the importance of having a rotating eviction pattern to decrease memory traffic. However, matrices from real applications do not have fully dense bands. Indeed, from our set of 48 matrices used in the previous chapter, considering a band width of $w_B = 2041$ that fits into a *DRC* of 16 KiB ($s_C = 2048$ elements), the average in-band density is $d_{IB} = 2.11 \times 10^{-3}$, ranging from 2.00×10^{-8} and 5.44×10^{-2}, compared to an average out-of-band density $d_{OB} = 1.28 \times 10^{-4}$, ranging from 0.00 and 1.04×10^{-2}. Considering that a part of the matrices of our set are not considered as banded, this order of magnitude difference between d_{IB} and d_{OB} on average across all 48 matrices shows that, even in matrices that do not appear to be banded, it is reasonable to assume that the region along the diagonal is much denser ($10\times$) than the region away from the diagonal.

Since the in-band density is lower than 1 in practice, we use the model to analyse the impact on normalized memory traffic of d_{IB} variations. We use the same parameters as in the previous sections, a matrix of dimension $M = 1000$ and a cache with four *ways* ($W = 4$) and cache-lines of size $l = 8$. We do not consider the out-of-band ($d_{OB} = 0$) since it is processed in a separate region. We focus on the in-band region for several cache configurations, making sure that the band width for each *set* size follows the relationship in Equation 4.2. In Figure 4.6, we plot $normMT$ for the output vector c, depending on d_{IB}, for varying cache configurations.

We observe that $normMT$ increases as d_{IB} decreases, generally following a constant nega-

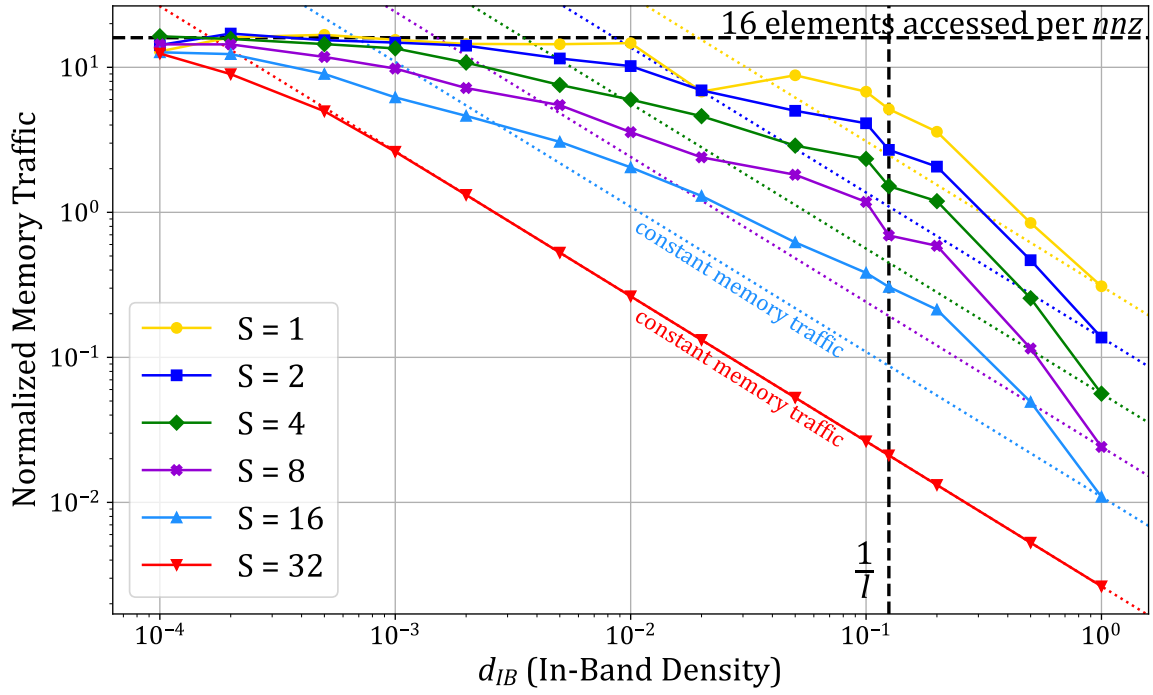


Figure 4.6 – Normalized Output Memory Traffic of the In-Band, Depending on the Density for Several Cache-Sizes

tive slope (*dashed*) on the right side of the graph for each cache size. Thus, across a broad range of in-band densities, the raw memory traffic remains relatively constant. As noted in Section 4.4.2, the $S = 32$ curve indicates the lower bound of memory traffic since the entire matrix can fit within the cache. In this scenario, there are no evictions in the band until the density drops sufficiently low ($< 10^{-3}$), resulting in isolated non-zero elements. For other cache sizes, variations around the constant raw memory traffic are observed. With an ideal eviction policy, allowing the rotating pattern to function as intended (refer to Section 4.4.1), we would expect all curves to align with their respective *dashed* lines, indicating constant raw memory traffic. However, the model employs a *random* policy, which disrupts the rotating pattern and accounts for the observed deviations. An *LRU* policy would likely yield improved results, aligning with the constant line for densities above $1/l$ and deviating for lower densities. The optimal policy would prioritize evicting lower addresses (the top of the band), thereby maintaining the rotating pattern.

For low values of d_{IB} (left side of the graph), $normMT$ plateaus at 16 elements transferred per non-zero element for all cache sizes. With very low density the non-zeros are isolated and provide no reuse: each non-zero triggers a full cache-line read and a full-line write-back. With $l = 8$ this corresponds to 16 elements transferred per non-zero.

It is therefore important to select the densest region of the matrix to be processed in a dedicated cache region such as the *DRC* of SpDCache. However, the selected region needs not be highly dense: for a 'perfectly' banded matrix with $M = 1000$, columns become nearly empty for $d_{IB} \lesssim 10^{-3}$, yet the raw memory traffic remains nearly constant in the range 10^{-2} to 10^{-3} depending on cache size. Because real, larger matrices often have a mean d_{IB} around 10^{-3}, a *DRC* sized to capture this denser region still provides good memory-traffic performance.

NOTE

In the SpDCache architecture, the *DRC* employs an *LRU* or pseudo-*LRU* eviction policy instead of an ideal custom policy. Despite this, we show that these simple, well-known policies still deliver good performance.

CONCLUSION

The **in-band density** has only a **minimal impact on memory traffic**, with real matrices typically exhibiting in-band densities around 10^{-3}.

4.4.5 Resulting Approach and Discussion

In the previous sections, we analyzed the behavior of *reduction caches* executing an `OP` SpMV kernel with banded matrices. We evaluated the expected memory traffic using an analytical model, separately considering the in- and out-of-band regions. We identified the cache characteristics required for both regions, which are quite different. We thus validate the proposition to divide the available cache memory into two dedicated regions: one for the in-band, and one for the out-of-band; the *DRC* and *SRT* of SpDCache, respectively.

To obtain the efficient rotating pattern for the in-band region, which is the key to massively reducing memory traffic, Equation 4.2 must be respected. In practice, this defines the size of the band that can be allocated to the in-band cache region. As we have seen, this cache region can operate efficiently with a set-associative line-wise configuration, on a wide range of in-band densities.

For the out-of-band region, a fully-associative element-wise cache with an element-wise memory interface can significantly reduce the memory traffic.

When sizing the two regions, once Equation 4.2 is satisfied, enlarging the in-band region yields only a marginal reduction in memory traffic (on the order of 10^{-2}). By contrast, a moderate increase of the out-of-band region produces a larger reduction (on the order of 10^{-1}). Therefore, the out-of-band region should be made as large as practical, subject to the hardware limits imposed by full associativity. This correlates with the results presented in Chapter 3, where we showed better performance with SpDCache configurations having a larger *SRT* (Table 3.2).

Although this analysis is sufficient to guide the design of SpDCache for `OP` SpMV with banded matrices, it is important to note that the analytical model that we have presented is based on the assumption of a uniform distribution of the non-zero elements. In practice, this is not always the case, and this non-uniformity can impact the cache behavior, as will be seen later in Chapter 5. Indeed, if the non-zero elements are clumped, the cache-lines tend to be better filled, resulting in lower memory traffic. In other words, our uniform assumption leads to a pessimistic prediction of memory traffic.

CONCLUSION

To minimize memory traffic when processing banded matrices with a *reduction cache*, it is important to **divide** the cache into two dedicated regions: (1) an **in-band region** dedicated to a band in the matrix whose size respects the relationship in Equation 4.2, and configured as a **set-associative line-wise** cache, and (2) an **out-of-band region** configured as a **fully-associative element-wise** cache with an **element-wise memory interface**. The **out-of-band region** should be made as large as practical, as it has a greater impact on memory

traffic than enlarging the in-band region.

4.5 Conclusion

THIS chapter presented a theoretical model to analyze the behavior of *reduction caches* when processing sparse matrices, specifically focusing on the `OP` SpMV kernel. A C program implementing a hybrid probabilistic/fast simulation model was developed, and we demonstrated how the structure of banded matrices can be leveraged to optimize cache performance.

Our analysis highlighted the importance of separating the in-band and out-of-band regions of the matrix into dedicated cache regions. For the in-band region, a set-associative, line-wise cache configuration ensures efficient data reuse by maintaining a rotating eviction pattern, minimizing unnecessary memory transactions. For the out-of-band region, a fully-associative, element-wise cache with an element-wise memory interface significantly reduces memory traffic by avoiding the transfer of mostly empty cache-lines.

The theoretical insights gained from this model validate the architectural choices of SpDCache, particularly the use of distinct regions for denser and sparser data. These findings provide a robust foundation for designing and sizing cache regions to minimize memory traffic, ultimately improving the efficiency of sparse matrix computations. In the next chapter, we will delve into the microarchitectural details of SpDCache, translating these theoretical insights into practical hardware design.

TAKEAWAYS

Simulating *reduction caches* and SpDCache

To study the concepts presented in this thesis, three models were developed with varying levels of abstraction. Starting from the most abstract model:

- **High level probabilistic/fast model in C** (4.3):
  - fast simulation ($\sim 10^5$ nnz/s), using generic matrix density distributions, easy parameter exploration, deep understanding of cache behaviors, but,
  - very high level (some approximations and hypothesis), limited on eviction policies.
- **Transaction level model in Python** (3.6):
  - faster than a full RTL simulation ($\sim 10^3$ nnz/s), accurate performance on memory traffic, functional testing, but,
  - slower than the probabilistic model, demands real matrices as input, two factors that alleviate the parameter exploration capabilities, no latency performance compared to RTL simulation.
- **RTL implementation** (next chapter):
  - accurate results in both memory traffic and latencies, hardware testing, but,
  - slow simulation ($\sim$ 10 nnz/s), demands real matrices in input, time-consuming implementation.

Validation and sizing of SpDCache three regions

- ***GRC***: can take advantage of prefetching methods, size depends on nnz_c.
- ***DRC***: must respect the relationship $s_C \geqslant w_B + l - 1$ (Equation 4.2).
- ***SRT***: internal storage and memory interface benefit from element-wise granularity, benefits from a larger size but must achieve a compromise with hardware cost.

To simplify address decoding, the initial region sizing for SpDCache was chosen somewhat arbitrarily to be **25/50/25%** for the *GRC*, *DRC* and *SRT*, respectively. Our analysis later

confirmed that this partitioning is actually close to optimal for memory traffic performance.

Chapter 5

Microarchitecture and Evaluation

Contents

5.1 Introduction

In the previous chapters, we established the theoretical foundations and high-level design principles for SpDCache, a specialized cache architecture tailored to optimize memory transactions for sparse matrix computations. This chapter bridges theory and practice by presenting the microarchitectural implementation of SpDCache and evaluating its performance through Register Transfer Language (RTL) simulations.

We begin by detailing the microarchitecture of SpDCache, focusing on its three distinct regions: the *Generic Region Cache* (*GRC*), the *Dense Region Cache* (*DRC*), and the *Sparse Region Table* (*SRT*). Each region is designed to handle specific types of memory accesses, sequential reads, dense *reductions*, and sparse *reductions*, respectively, thereby addressing the irregular memory access patterns inherent in sparse matrix operations. The RTL implementation of SpDCache is described, highlighting the hardware components and pipeline stages that enable efficient data processing and *reduction* operations.

Following the microarchitectural overview, we present the evaluation methodology and results obtained from RTL simulations. These simulations provide a detailed assessment

of SpDCache's performance, including memory traffic reduction and latency improvements. We analyze the behavior of SpDCache across a diverse set of sparse matrices, comparing its performance against baseline cache architectures and validating the theoretical insights gained from earlier chapters.

By translating the theoretical model into a practical hardware design, this chapter demonstrates the effectiveness of SpDCache in minimizing memory traffic and improving computational efficiency for sparse matrix kernels. The results underscore the importance of region-based caching and *reduction* support, paving the way for further optimizations and scalability enhancements discussed in the subsequent chapter.

5.2 SpDCache Microarchitecture

SpDCache is implemented as an L1 data cache for a CVA6 RISC-V core, building upon an existing open-source high-performance set-associative data cache known as HPDcache [34]. In contrast to previous RISC-V data caches, HPDcache introduces new features, including buffering techniques for memory writes, set-associative storage to manage read miss requests, and dynamic configurability of each cache-line to operate in either write-back or write-through modes. It is designed for high performance, utilizing a three-stage pipeline to handle multiple instructions while minimizing stalls through out-of-order cache execution. The CPU can issue multiple types of read and write requests to the cache including atomic operations (AMOs), cache management operations (CMOs) as well as the new *reduction* operation (*RED*) introduced by SpDCache. In this way, we can say that HPDCache and SpDCache executes "instructions" which correspond to these different types of operations.

An overview of SpDCache's microarchitecture is shown in Figure 5.1, where *black* and *blue* components are directly reused from HPDcache. The first stage (*st0*) decodes instructions and issues reads to the cache directory and data RAMs (*Dir* and *Data*). These reads require one cycle to respond. The directory holds the state of the selected cache-line which is required in the second stage (*st1*). The response from the directory at *st1* indicates whether a hit or miss occurred and if an eviction is necessary. The instruction proceeds to the final stage (*st2*) only if a miss or eviction must be processed. At *st0*, the instruction is not yet consumed. Depending on the response and the availability of cache resources at *st1*, a *no-operation* (*nop*) may be inserted in the pipeline, and the instruction may be placed in a *replay table*. The instruction is then replayed with the highest priority once its dependencies have been met. If the instruction does not need to be replayed at *st1*, it is consumed and processed. This pipeline serves as the foundation for implementing SpDCache's *reduction* operations.

The *cache controller* coordinates several auxiliary units. The *miss handler*, is allocated requests from *st2*, tracks outstanding memory read requests and notifies the cache controller with a *refill* request when responses arrive. An optional *write buffer* gathers recent writes to memory, allowing coalescing of transactions that target the same cache-line. The *flush controller*, also invoked by *st2*, reads cache-line data from the *Data* RAMs and issues memory writes. It temporarily stalls the pipeline to prevent RAM conflicts. HPDcache further provides an *UC* unit to handle uncached transactions. Finally, the *CMO handler* performs internal cache operations (e.g., invalidations and flushes). When active it takes control of the cache and can invoke the *flush controller* if necessary. Most of these units are reused and extended to implement SpDCache.

HPDcache provides extensive static and dynamic configurability, making it a flexible foundation for further extensions. The changes to SpDCache were done in a way such that, by simply changing parameters, the same RTL code can implement (1) the original generic cache (GC) as the first baseline, (2) a standard *reduction cache* (RedC) as the second baseline, or (3) SpDCache (SpDC), the solution presented in the previous chapters.

CONCLUSION

HPDcache is an **open-source**, high-performance cache which we used as the **starting point** for our work. We extended it to support *reduction* operations, **operating configurably** as either a simple *reduction cache* (RedC) or as the full SpDCache outlined in the previous chapters.

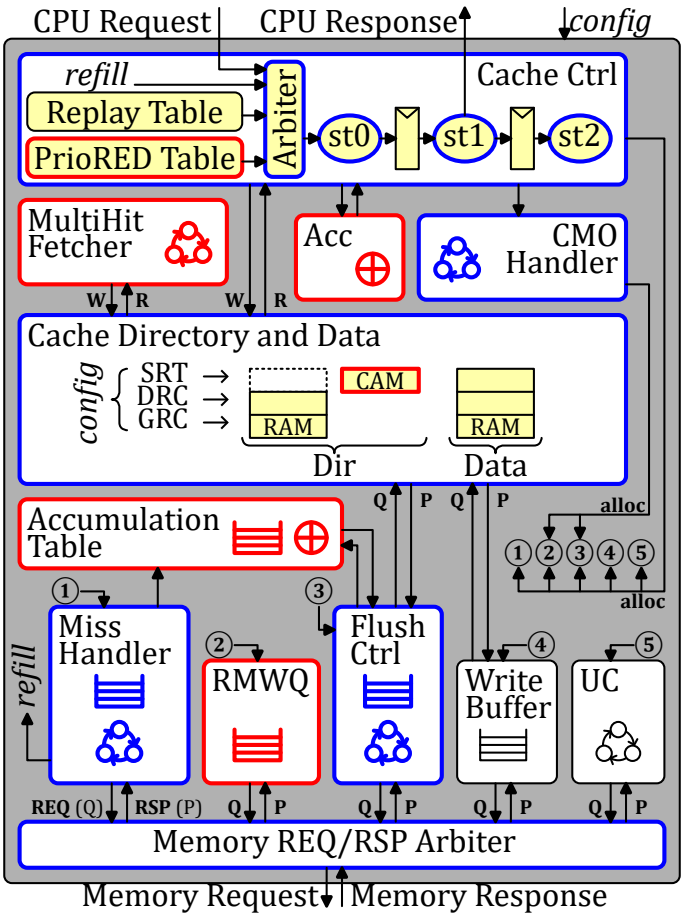


Figure 5.1 – SpDCache Microarchitecture, based on HPDcache

5.2.1 RTL Implementation of SpDCache

We configure the original HPDcache as a 4-way set-associative cache with 128 *sets* and 64-byte cache-lines, resulting in a total cache size of 32 KiB. We extend the three-stage pipeline to handle the new *RED* instruction. Figure 5.1 provides an overview of SpDCache's microarchitecture, highlighting new blocks in *red*, modified blocks in *blue*, and untouched components in *black*. The overall architecture aligns with the design presented in Chapter 3, adapted here for hardware implementation. Let us briefly describe the operation of the *DRC* under three scenarios: hits, misses with available cache-lines, and misses requiring eviction.

In the case of a hit, the value of the *RED* instruction is accumulated with its corresponding value in the cache. At *st1*, the cache value, read in *st0* from *Data*, is accumulated with the *RED* value using a dedicated accumulator (*Acc*). The result is written back to *Data* at *st2*.

In the case of a miss, if there is an available cache-line, the value of the *RED* instruction is simply stored in this free entry of the cache. A free cache-line is selected at *st1*, and at *st2*, the *RED* value is written to *Data* along with zeros to reset the selected cache-line. A write to *Dir* is also performed to change the state of the cache-line from free to valid.

If a miss occurs but no cache-line is free, an eviction must be processed before handling the *RED* instruction. The *RED* instruction is placed in the *PrioRED table* to be replayed with higher priority than the *replay table*. Meanwhile, at *st2*, entries are allocated in the *miss handler*, *flush controller* and *accumulation table* to process the eviction. The eviction is a "fire-and-forget" operation, meaning the *RED* instruction only needs to wait for the entry in *Data* to be freed, not for the entire eviction to finish. The eviction process begins by fetching and resetting the chosen 64B cache-line from *Data*, which is done in one cycle by the *flush controller*. Simultaneously, the *miss handler* issues a read to the main memory. The *accumulation table* waits and temporarily stores both the old cache-line from the *flush controller* and the 64B response data from the *miss handler*. It then accumulates the cache-line with the data, word by word, and sends the 64B result back to the main memory using the existing write interface.

With the changes described above, we have transformed HPDcache into a *reduction cache*. To achieve multiple cache regions, we virtually separate the cache RAMs. To maintain HPDcache's set-associative operation, we divide the cache memory along its *sets*. Thus, each region of the cache has the same structure as the overall cache but is smaller in size: each *set* contains a fixed number (the number of ways) of 64B cache-lines, as depicted in Figure 5.2. Each region can thus map any possible address. To ensure that every address matches a unique *set* within each region, we allocate a power-of-two number of consecutive *sets* to a region. Considering the total number of *sets* S_T and our region R mapping on $S_R < S_T$ of them, starting at the *set* s_{offset}, if an address matches one of the *sets* of this region, no action is required. However, if the address matches a *set* s_{addr} outside the region, we need to map it to a unique *set* in the region such that $s_{region} = (s_{addr} \% S_R) + s_{offset}$, as illustrated in Figure 5.2. The three regions *GRC*, *DRC*, and *SRT* (*green*, *orange* and *blue*, respectively, in the example) are each defined by their number of *sets* S_R and their first *set*, s_{offset}. SpDCache can thus be reconfigured to operate as a generic cache, a traditional *reduction cache* or a region-based *reduction cache* based on the needs of the application (*config* in Figure 5.1).

The *SRT* region requires additional adjustments to function as an element-wise fully-associative cache. This region operates as if all the *sets* were combined into a single *set*, containing numerous elements. Each element requires its own directory entry. Therefore, we replace the *Dir* RAMs of this region with a CAM (*Content-Addressable Memory*) containing

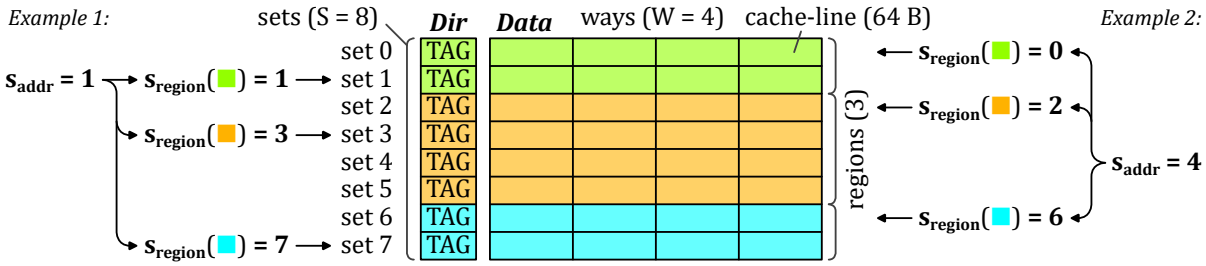


Figure 5.2 – Set Redirection Depending on the Region

the addresses of each valid element in the *SRT* region. With the CAM, we can check for a hit across the entire region and retrieve the matching value stored in the *Data* RAMs. We also added a queue (*RMWQ*) to manage the element-wise *Read-Modify-Write* (*RMW*) memory transactions.

To avoid address duplication between regions, the *MultiHit fetcher* in Figure 5.1 manages coherency between the *DRC* and *SRT*, following the rules defined in Section 3.5. Since software selects the region for each operation, it is possible that addresses that were allocated in the *SRT* need to be move to the *DRC*. The *MultiHit fetcher* performs two tasks. First, given an address, it probes the CAM for entries whose cache-line address matches. If one or more matches are found, it records their positions locally and notifies the *cache controller*. Second, when activated, the *MultiHit fetcher* gathers the corresponding elements from the *Data* RAMs of the *SRT* and assembles them into a single 64B line that will be written into the *DRC*.

Contrary to the description in Chapter 3, we do not transfer isolated elements from the *DRC* to the *RMWQ* on eviction. In our Python simulations we observed that only a negligible fraction of elements were transferred (mean 0.3%, range 0 to 5% across 48 matrices), which demonstrates the effectiveness of the *DRC* blocking behavior. To simplify the RTL implementation, this transfer is therefore omitted.

CONCLUSION

The SpDCache implementation introduces two primary extensions. First, it integrates **support for *reduction* instructions (*REDs*)** into the cache pipeline while preserving the generic cache behavior. This entails mechanisms for accumulating entire cache-lines and for handling element-wise atomic *RMW* transactions. Second, it implements a **virtual partitioning of the cache RAMs**: the memory is logically split and a *set*-redirection scheme is applied to dynamically create multiple regions.

5.2.2 Software Interface

We have added several custom instructions to the CVA6 RISC-V core [115]. The primary addition is the *reduction* instruction (*RED*), which takes an address and a value as arguments and expects no response. This instruction is accompanied by a separate region instruction (*REG*) that specifies in which region subsequent *reductions* should be processed. We have also added an instruction to flush the *reduction* regions (*DRC* and *SRT*) of the cache, which is necessary at the end of the algorithm. It is performed in the cache by the *CMO handler*. Finally, we have included an instruction to enable and disable the *reduction* mode of SpDCache.

We demonstrate a simple example in Source Code 5.1 that accumulates multiple values at a single address. Before executing *RED* transactions, the *reduction* engine must be enabled

(line 3). The cache transitions from a fully generic state to SpDC with multiple regions using a hardware-defined static configuration. This example assumes a configuration with three regions: *GRC*, *DRC*, and *SRT*. For a simpler *reduction cache* configuration with only two regions (*GRC* and *DRC*), the *REG* instruction would be unnecessary. Additionally, the instruction can accept a dynamic configuration argument specifying word granularity and operation type. A future optimization would allow passing the entire configuration as a parameter, enabling dynamic configuration of SpDC regions at runtime. Next, at line 8, we select the *IBR* region using the *REG* instruction. The subsequent two *RED* instructions are then executed and stored in the *DRC*. At line 13, we switch to the *OBR* region, causing the following *RED* instruction to be processed by the *SRT*. Once all *RED* operations complete, SpDCache is flushed at line 17 before disabling the reduction engine, reverting to standard cache operation.

Source Code 5.1 – A Simple Example of *RED* Operations Usage

```
1  double data = .0; /* Initialize a data with 0 */
2
3  redEngineOn(config); /* Start the RED engine */
4  /* In this example, SpDCache is configured into three regions */
5  /* We can pass on a dynamic configuration to set the word granularity
6     and the type of operation */
7
8  REG(IBR); /* Following reductions are processed by the DRC */
9
10 RED(&data, 1.); /* data += 1 */
11 RED(&data, 2.); /* data += 2 */
12
13 REG(OBR); /* Following reductions are processed by the SRT */
14
15 RED(&data, 3.); /* data += 3 */
16
17 redFlush(); /* The DRC and SRT are flushed to memory */
18
19 redEngineOff(); /* Stop the RED engine */
20 /* SpDCache is now back to one fully generic cache region */
21
22 printf("%lf", data); /* Should print 6 */
```

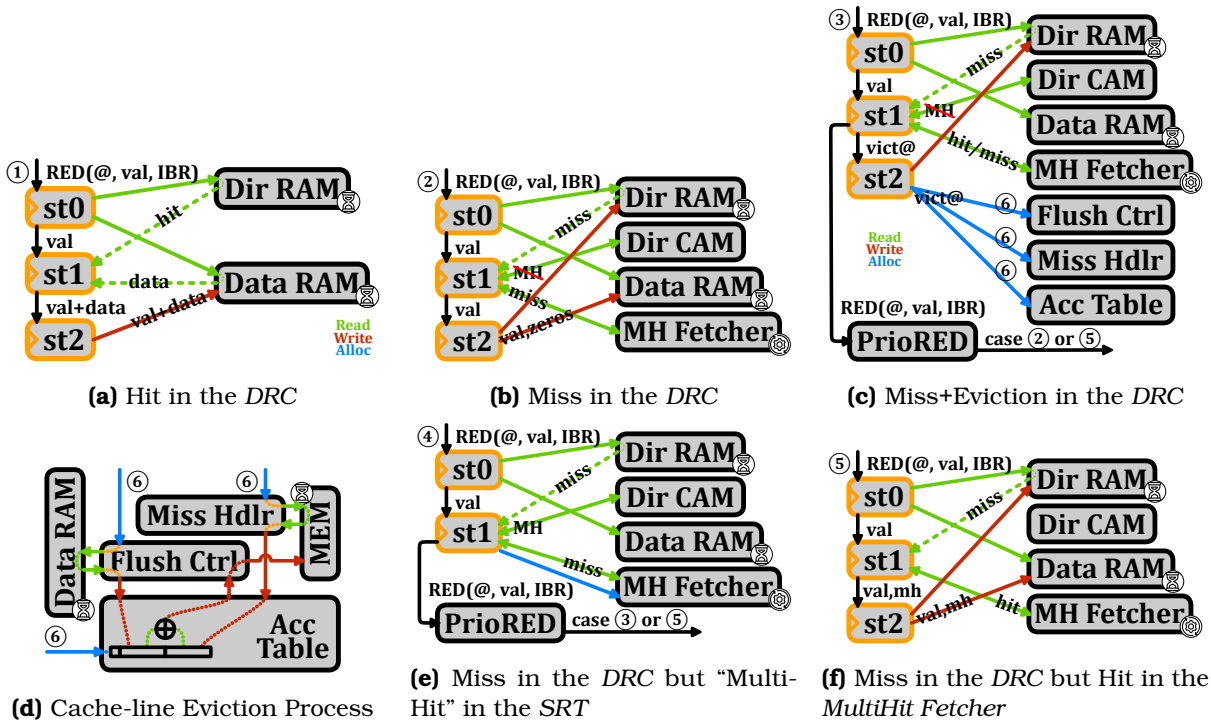
CONCLUSION

SpDCache provides a simple and flexible software interface through a minimal set of custom RISC-V instructions. The ability to dynamically enable and disable *reduction* mode allows the cache to seamlessly transition between generic and specialized operation, adapting to application requirements.

5.2.3 Details on SpDCache Pipeline Operation

In this section, we detail the SpDCache pipeline architecture, which extends the HPDcache original pipeline by adding new behavior for *reduction* operations while preserving existing functionality. Figures 5.3 and 5.4 illustrate the various scenarios that occur when processing *RED* instructions, depending on the selected region (*IBR* or *OBR*, respectively). We represent the three pipeline stages (*st0*, *st1*, and *st2*) and their interactions with the auxiliary units involved in processing *RED* instructions. The *green*, *red*, and *blue* arrows represent read and write operations, and unit allocations, respectively. The *Dir* and *Data* RAM units, marked with an hourglass, require one full cycle to respond.

We now describe the scenarios when the region argument is *IBR*. In Figure 5.3a, the



(a) Hit in the *DRC*

(b) Miss in the *DRC*

(c) Miss+Eviction in the *DRC*

(d) Cache-line Eviction Process

(e) Miss in the *DRC* but "Multi-Hit" in the *SRT*

(f) Miss in the *DRC* but Hit in the *MultiHit Fetcher*

Figure 5.3 – Pipeline Operations when the Region Argument is *IBR*

pipeline issues reads to the *Dir* and *Data* RAMs at *st0*. Responses arrive one cycle later at *st1*, indicating whether a hit or miss occurs in the *DRC*. This initial pipeline stage is identical across all scenarios. In case ①, a hit is detected, so the data received at *st1* is accumulated with the *RED* value and forwarded to *st2* for write-back to the *Data* RAM.

When a miss occurs, multiple scenarios are possible (Figures 5.3b to 5.3f), as an eviction may be required and the *MultiHit Fetcher* may be active. In case ②, upon detecting a miss at *st1*, the pipeline checks the *Dir* CAM of the *SRT* for a "multi-hit". As described in Chapter 3, when a miss occurs in the *DRC*, we gather elements with the same cache-line address from the *SRT* to prevent duplication. This check completes within the cycle and notifies the *MultiHit Fetcher*. If a "multi-hit" is found, the *MultiHit Fetcher* allocates an entry and takes control of the pipeline to read and invalidate matching entries from the *Data* RAM, storing them locally for later use. If no "multi-hit" occurs (case ②), we handle a simple miss by writing the *RED* value to a free cache-line in the *Data* RAM at *st2*.

In case ④, a "multi-hit" is detected and the *MultiHit Fetcher* is allocated. The *RED* instruction is placed in the *PrioRED Table* for immediate replay once the pipeline becomes available, proceeding to case ③ or ⑤ depending on whether an eviction is needed.

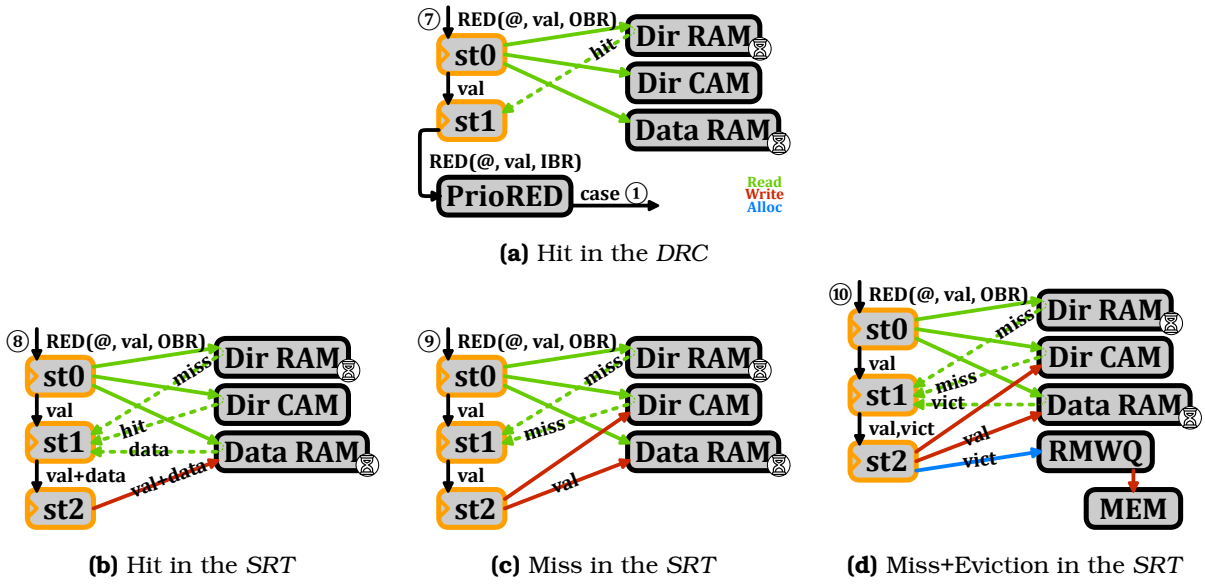
In case ⑤, which occurs only upon replay, the *RED* instruction misses in the *DRC* but hits in the *MultiHit Fetcher*, which has cached the gathered data from the previous case ④. The *RED* value is then accumulated with the *MultiHit Fetcher* data and written to the *Data* RAM at *st2*.

Case ③ handles the eviction scenario, which applies whether or not a "multi-hit" replay is involved. When an eviction is required, a victim cache-line is selected at *st1* and must be evicted before inserting the new value. The eviction process begins at *st2*, with the *Miss Handler*, *Flush Controller*, and *Accumulation Table* allocated for the victim address. Concurrently, the *RED* instruction is placed in the *PrioRED Table* for immediate replay. Once the victim

cache-line is freed, the instruction is replayed as case ② or ⑤, depending on whether data exists for the same cache-line address in the *MultiHit Fetcher*. Note that when a "multi-hit" occurs (case ④), the *MultiHit Fetcher* retains the gathered data until reaching case ⑤. Subsequently, since the data is no longer present in the *Dir* CAM and *Data* RAM, no "multi-hit" will occur for that address on replay.

Figure 5.3d depicts the eviction process, which executes in parallel with the pipeline. Upon receiving the victim address in case ③, the *Flush Controller* immediately reads the *Data* RAM and invalidates the entry in a single cycle. This allows the pipeline to replay the *RED* instruction without waiting for the entire eviction to complete. The *Flush Controller* then forwards the victim data to the *Accumulation Table* for temporary storage. Meanwhile, the *Miss Handler* issues a memory read, which may take on the order of a hundred cycles. Upon receiving the data from memory, the *Miss Handler* sends it to the *Accumulation Table*. Once the *Accumulation Table* has both data lines, it performs word-by-word accumulation and returns the result to the *Flush Controller*, which writes it back to memory to complete the eviction.
The following sequences enumerate all possible cases when the region argument is *IBR*:

- hit in the *DRC*: ①;
- miss in the *DRC*: ②;
- miss+eviction in the *DRC*: ③ + ⑥ → ②;
- miss in the *DRC* with "multi-hit" in the *SRT*: ④ → ⑤;
- miss+eviction in the *DRC* with "multi-hit" in the *SRT*: ④ → ③ + ⑥ → ⑤.



(a) Hit in the *DRC*

(b) Hit in the *SRT*

(c) Miss in the *SRT*

(d) Miss+Eviction in the *SRT*

Figure 5.4 – Pipeline Operations when the Region Argument is *OBR*

We now describe the cases when the region argument is *OBR*, as illustrated in Figures 5.4a to 5.4d. In all cases, we check for hits in both the *DRC* and *SRT* at *st0*. If a hit occurs in the *DRC*, case ⑦, the *RED* instruction is processed in the *DRC* regardless of the region argument. It is then sent to the *PrioRED Table* for replay as a *DRC* hit, corresponding to case ①.

If no hit is found in the *DRC*, we check the *Dir* CAM for hits in the *SRT*. In case of a hit, ⑧, the data is accumulated with the *RED* value and written back to the *Data* RAM. In case of a miss, ⑨, the data is inserted into the *Data* RAM. For an eviction, ⑩, the victim data is read directly between *st0* and *st1*, since the *Dir* CAM responds within one cycle and can initiate the

victim read before *st1*. Thus, the *RED* value is inserted into the *Data* RAM at *st2*, while the victim data is sent to the *RMWQ*. The *RMWQ* independently issues atomic *RMW* transactions to memory as entries become available.
The following sequences enumerate all possible cases when the region argument is *OBR*:

- hit in the *DRC*, ⑦ → ①;
- hit in the *SRT*, ⑧;
- miss in the *SRT*, ⑨;
- miss+eviction in the *SRT*, ⑩.

NOTE

This implementation leverages most of the original pipeline capabilities while keeping modifications to a minimum. We do not claim this is the optimal design for SpDCache. A ground-up redesign or significant modifications to HPDcache could yield better performance.

CONCLUSION

We implemented **SpDCache** in RTL by extending HPDcache with **minimal modifications**, introducing support for *reduction* operations and multiple cache regions without requiring additional memory. This **configurable design** allows a single RTL implementation to function as a generic cache, a traditional *reduction cache*, or the region-based SpDCache variant, facilitating comprehensive performance evaluation.

5.3 Evaluation

IN this section, we aim to evaluate the performance of various cache configurations and their impact on memory traffic and latency during RTL simulations. We will first outline the methodology used for these evaluations, followed by a detailed analysis of the results obtained from the simulations, ending with comparisons with the previous models.

5.3.1 Methodology for RTL Simulations

The starting point is a CVA6 [125] RISC-V core with HPDcache [34] as an L1 data-cache with size 32 KiB ($S = 128$, $W = 4$, $l = 8$ elements). This serves as our baseline for performance comparison which we refer to as the Generic Cache (GC). Next we evaluate a *reduction cache* which we derived from the HPDcache and configured to have two regions: a generic region with 16 KiB, and a *reduction* region with 16 KiB whose behavior is the same as the *DRC* region of SpDCache. We refer to this design as RedC. Finally, we evaluate SpDCache (SpDC) with a reference configuration of: *GRC* = 8 KiB, *DRC* = 16 KiB, *SRT* = 8 KiB, which corresponds to the 25%/50%/25% configuration selected in Chapter 3. The configurations are summarized in Table 5.1.

Table 5.1 – Region Sizing of the Cache Configurations

Configurations	*GRC*	*DRC*	*SRT*
GC	100% ($S = 128$)	0% ($S = 0$)	0% ($S = 0$)
RedC	50% ($S = 64$)	50% ($S = 64$)	0% ($S = 0$)
SpDC	25% ($S = 32$)	50% ($S = 64$)	25% ($S = 32$)

All our simulations are performed using Siemens Questasim™ 2023.3 and the memory accesses beyond the L1 incur a fixed latency of 64 clock cycles. We do not aim to evaluate a full system in this work, as our goal is to identify performance behavior using real matrices. Thus, we use a single-core, single-cache-level environment. The CVA6 core is single-issue and can process at most one instruction per cycle. The fixed memory latency of 64 cycles provides a reasonable approximation of the latency incurred by requests passing through the other memory levels, and a deterministic memory model makes the results easily reproducible.

Our SystemVerilog implementation supports only integer addition up to 64 bits, precluding floating-point accumulations. Extending SpDCache to support floating-point operations remains future work. We employ 64-bit integers throughout and assume this choice does not significantly impact performance results.

A second implementation constraint is that region sizes for *GRC*, *DRC*, and *SRT* must be powers of two in terms of *sets*, unlike the analytical and Python models described in Chapters 3 and 4. This limitation motivates our choice of a 50/50/0% configuration for RedC, which we expect to slightly underperform compared to more flexible configurations. Future optimizations of the implementation could relax this constraint.

CONCLUSION

We simulated three cache configurations, GC, RedC, and SpDC, using a single-core CVA6 processor with a single cache level.

5.3.2 Sparse Formats Used for thr RTL Simulations

By default, matrices are stored in CSC sparse format, and we use this format for most of our simulations. However, in our evaluation with banded matrices, we also simulated SpDC with a modified CSC format tailored for banded matrices. As shown in Figure 5.5, compared to a CSC, we added extra information in the ptr table to identify the beginning and end of the band. Indeed, three pointers instead of one are now needed per column, as shown with the example of column 2 in *yellow*. The column is split in one in-band region in the middle (IB, from ptr 7 to 9) and two out-of-band regions around (OB, from ptr 6 to 7, and 9 to 10). The ptr table size thus becomes $3 \times M + 1$. With this format, we can avoid the computational cost of identifying in which region non-zero elements are, while traversing the matrix, with the drawback of having to preprocess the matrix to store this additional information.

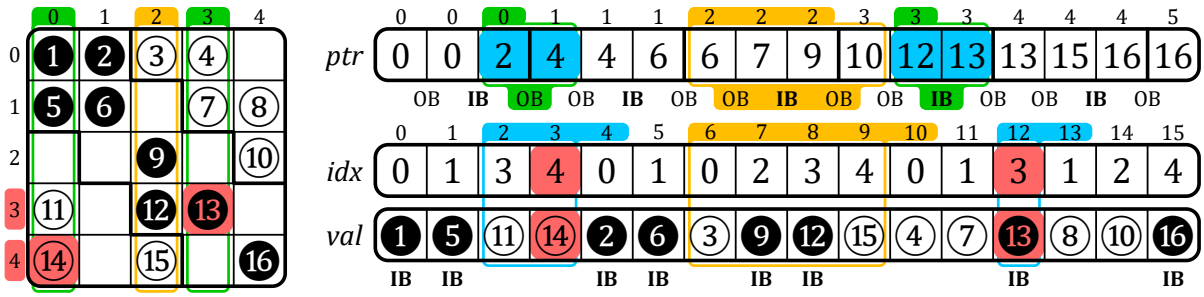


Figure 5.5 – Example of a Sparse Matrix in BaCSC Format

CONCLUSION

We employ the CSC format for most simulations and additionally evaluate SpDC using BaCSC, a specialized format optimized for banded matrices.

5.3.3 Software Implementation of Outer-Product SpMV Kernel

The code running on the CVA6 is written in generic C, compiled with *gcc* 11.2.0 and -O3 optimization. The *RED*-related instructions used in the `OP` SpMV kernel are manually inserted using *asm* directives, as illustrated in Source Code 5.2.

Source Code 5.2 – C Code for the `OP` SpMV with *RED* Operations

```
1  redEngineOn(config);
2
3  int currCol = A.ptr[0];
4  for (int j = 0; j < N; j++) { /* Column first */
5      int nextCol = A.ptr[j+1];
6      int bj      = b[j];
7
8      for (int pos = currCol; pos < nextCol; pos++) {
9          int idx = A.idx[pos];
10         int val = A.val[pos] * bj;
11
12         /* Without RED engine: c[idx] += val; */
13         /* With RED engine: */
14         int *addr    = c + idx;
15         region_t reg = regionSelect();
16
17         asm volatile ( /* RISC-V ISA: Type R */ /* REG(reg); */
18             "reg x0, x0, %[rs2]"
19             :
20             : [rs2] "r" ((int) reg)
21         );
22         asm volatile ( /* RISC-V ISA: Type S */ /* RED(addr, val); */
23             "red.w %[rs2], 0(%[rs1])"
24             :
25             : [rs2] "r" (addr), [rs2] "r" (val)
26         );
27     }
28
29     currCol = nextCol;
30 }
31
32 redFlush();
33 redEngineOff();
```

The code shown is for SpDC. Variants for other designs differ as follows: RedC omits region selection (line 15) and invokes *REG* once before the loops with *IBR* as the argument; GC requires no *REG* instruction and replaces *RED* with standard accumulation: $c[idx] \mathrel{+}= val$.

For SpDC, it is necessary to know the target region for each *RED* instruction based on the size of the band (line 15). The half-band width w_{hB} is derived from Chapter 4 (Equation 4.2) to ensure the in-band region fits within the *DRC* independent of matrix structure. In our reference configuration ($s_C(DRC) = 2048$, $l = 8$), $w_{hB} = 1021$. Region selection follows Equation 5.1 via an inline function, avoiding conditional branches within the inner-loop.

$$\forall (i,j) \in [\![0, M-1]\!] \times [\![0, N-1]\!],\ region = \begin{cases} IBR & \text{if } |i-j| < w_{hB} \\ OBR & \text{else} \end{cases} \tag{5.1}$$

On-the-fly region computation introduces overhead in the critical inner-loop. To mitigate

this, we propose an alternative implementation (Source Code 5.3) that precomputes regions and stores them using the BaCSC format. Each column iteration begins with lookups in the outer-loop (line 5) to retrieve pointers for three sections: top out-of-band (*OB1*), in-band (*IB*), and bottom out-of-band (*OB2*). The column is then processed through three separate inner-loops (lines 13, 21, 29) without region computation overhead.

Source Code 5.3 – Optimized C Code for the `OP` SpMV when Using BaCSC

```
1  redEngineOn(config);
2
3  /* A is stored in BaCSC format with pointers delimiting the band sections */
4  int currCol_OB1      = A.ptr[0];   /* begining of the column - first OB section*/
5  for (int j = 0, int k = 0; j < N; j++, k += 3) {
6      int currCol_IB  = A.ptr[k+1]; /* begining of the band   - IB section */
7      int currCol_OB2 = A.ptr[k+2]; /* end of the band        - second OB section */
8      int nextCol     = A.ptr[k+3]; /* end of the column      - next column */
9      int bj          = b[j];
10
11     /* 1st step: top OB section of the column */
12     REG(OBR);
13     for (int pos = currCol_OB1; pos < currCol_IB; pos++) {
14         int idx = A.idx[pos];
15         int val = A.val[pos] * bj;
16         RED(c + idx, val);
17     }
18
19     /* 2nd step: IB section of the column */
20     REG(IBR);
21     for (int pos = currCol_IB; pos < currCol_OB2; pos++) {
22         int idx = A.idx[pos];
23         int val = A.val[pos] * bj;
24         RED(c + idx, val);
25     }
26
27     /* 3rd step: bottom OB section of the column */
28     REG(OBR);
29     for (int pos = currCol_OB2; pos < nextCol; pos++) {
30         int idx = A.idx[pos];
31         int val = A.val[pos] * bj;
32         RED(c + idx, val);
33     }
34
35     currCol_OB1 = nextCol; /* begining of the next column */
36 }
37
38 redFlush();
39 redEngineOff();
```

Each approach trades off one limitation for another. SpDC(CSC) uses simple CSC format with minimal preprocessing and reduced memory overhead, but incurs additional inner-loop instructions and thus latency per iteration. SpDC(BaCSC) reduces instructions and latency at the cost of matrix preprocessing into BaCSC format and an increased memory footprint for the matrix. Both variants are evaluated in the results section.

These software approaches avoid hardware modifications, providing flexibility. However, hardware-computed regions with CSC format would eliminate the noted drawbacks. A detailed breakdown of each design's instruction characteristics:

- GC: 10 inner-loop instructions; 3 instructions per *RED* with critical load dependency.
- RedC: 8 inner-loop instructions; 1 "fire-and-forget" *RED* instruction; 1 *REG* setup per entire computation.
- SpDC(CSC): 14 inner-loop instructions; 1 "fire-and-forget" *RED* instruction; region computation within inner-loop.

- SpDC(BaCSC): 8 inner-loop instructions; 1 "fire-and-forget" *RED* instruction; outer-loop overhead.

To prevent software from limiting performance, we unroll inner-loops by a factor of 8 (not shown in Source Codes 5.2 and 5.3). Each unrolled iteration executes 8 logical iterations with optimized instruction scheduling. With cache-line size $l = 8$, this unrolling hides read-miss latency through hits on consecutive addresses. "Random" *reduction* accesses do not benefit from this latency hiding. Consequently, GC experiences potential latency from both *reductions* reads and writes, while RedC and SpDC benefit from "fire-and-forget" semantics and consecutive *reduction* scheduling.

SpDC(CSC) is constrained to 4× unrolling due to register pressure on our platform. This limitation, addressable with a higher-register CPU, highlights another advantage of hardware based region computation.

CONCLUSION

We utilize four distinct software implementations of the `OP` SpMV, each customized for our four simulations and fully unrolled to prevent performance constraints.

5.3.4 Matrix Selection for the RTL Evaluation

We have selected an extended set of 62 square matrices from the SuiteSparse Matrix Collection [63] which includes virtually all the matrices used by recent hardware accelerators [87, 103, 120, 126, 127]. We verified that this set is diverse containing banded and non-banded sparse matrices with various sizes ($M \in [1.2 \times 10^2, 2.0 \times 10^6]$), numbers of non-zero elements ($nnz \in [2.2 \times 10^3, 1.4 \times 10^7]$) and a wide range of densities ($d \in [1.4 \times 10^{-6}, 7.8 \times 10^{-1}]$).

Based on our experimental results presented in the following section, we categorize these matrices into two groups according to their performance characteristics. To facilitate this classification, we introduce a new metric, d_{cd}, defined as the average density of cache-line-sized clusters within matrix columns. Specifically, for a cache with line size l, this metric quantifies the average density of non-empty cache-lines (or vertical-stripes as referenced in Chapter 4), ranging between $\frac{1}{l}$ and 1.

This metric provides a fine grain view of matrix density, which aligns more closely with cache characteristics. When d_{cd} approaches $\frac{1}{l}$ for a given matrix, it indicates that most non-zero elements are separated by at least l zeros. Consequently, the cache-lines are often nearly empty, which may render the line-wise *DRC* of a RedC inefficient, while the element-wise *SRT* of SpDC becomes more beneficial. Conversely, as d_{cd} increases, the non-zero elements of the matrix cluster more closely, enhancing the efficiency of the *DRC*.

In our experiments with $l = 8$, we establish two groups of matrices based on a d_{cd} threshold of 20%. This threshold corresponds to either one to two non-zero elements per eight. At this point, this threshold may appear somewhat arbitrary and it, of course, depends on the processor and caches. As we presents the results, this choice, for our CVA6 core, will become clearer.

Using the threshold based on d_{cd}, we categorize 21 matrices in group 1 ($d_{cd} \leqslant 20\%$) and 41 in group 2 ($d_{cd} > 20\%$). We have selected 13 well-known matrices from both groups that are frequently used for performance evaluation, as illustrated in Table 5.2. The selected matrices represent a range of overall matrix densities (d) and d_{cd} values, as depicted by the *red* and *purple* points in the scatter plot in Figure 5.6. In the following evaluation, we report the

performance results of the 13 matrices from group 1 and 2, the geometric means for both groups (covering 21 matrices in group 1 and 41 in group 2), and the geometric mean for the full set of 62 matrices.

Table 5.2 – Characteristics of Selected Group 1 and 2 Matrices

ID	Matrix	M	nnz	Density	d_{cd}
Group 1					
11	poisson3Da	13.5 k	352 k	1.93 e-3	14%
12	cit-HepTh	27.8 k	353 k	4.57 e-4	13%
13	soc-Epinions1	75.8 k	508 k	8.84 e-5	16%
14	sme3Db	29.1 k	2.08 M	2.46 e-3	15%
15	Stanford	282 k	2.31 M	2.91 e-5	12%
16	web-Google	916 k	5.11 M	6.08 e-6	13%
Group 2					
21	msc10848	10.8 k	1.23 M	1.05 e-2	73%
22	opt1	15.4 k	1.93 M	8.09 e-3	86%
23	bcsstk32	44.6 k	2.01 M	1.01 e-3	75%
24	xenon2	15.7 k	2.87 M	1.56 e-4	40%
25	consph	83.3 k	6.01 M	8.66 e-4	57%
26	x104	108 k	10.0 M	8.66 e-4	78%
27	pwtk	218 k	11.6 M	2.45 e-4	75%

CONCLUSION

The benchmark matrices are divided into two groups depending on d_{cd}, their average cache-line-sized clusters density.

5.3.5 Results Analysis

5.3.5.1 Memory Traffic Performance

For all benchmark matrices, we simulated the RTL model executing the full **OP** SpMV and measured output memory traffic, shown in Figure 5.7 for RedC, SpDC with CSC, and SpDC with BaCSC, normalized compared to the output traffic when simulating with GC.

SpDC achieves a 3.9× output memory traffic reduction compared to GC, which is 2.8× better than RedC (1.4×). While SpDC performs rather consistently for both groups of matrices, RedC lacks hardware adaptation for irregular access patterns, making its performance heavily group-dependent. For low d_{cd} (sparse cache-lines), the line-wise cache proves ineffective. Conversely, SpDC maintains dense data in the *DRC* across columns, enabling the rotating pattern from Chapter 4. Outside the *DRC*, the element-wise *SRT* organization efficiently avoids transferring mostly empty cache-lines.

For group 1 matrices, RedC traffic exceeds GC because GC's cache is 2× larger than RedC's *DRC*, reducing performance [1]. For group 2 matrices, higher cache-line density compensates, allowing RedC to leverage in-memory accumulation through increased hit-rates.

1. Recall that for the benchmarking, the total cache size was kept constant at 32 KiB. The RedC has a 16 KiB generic region and a 16 KiB *DRC*, whereas the GC can flexibly use the full 32 KiB.

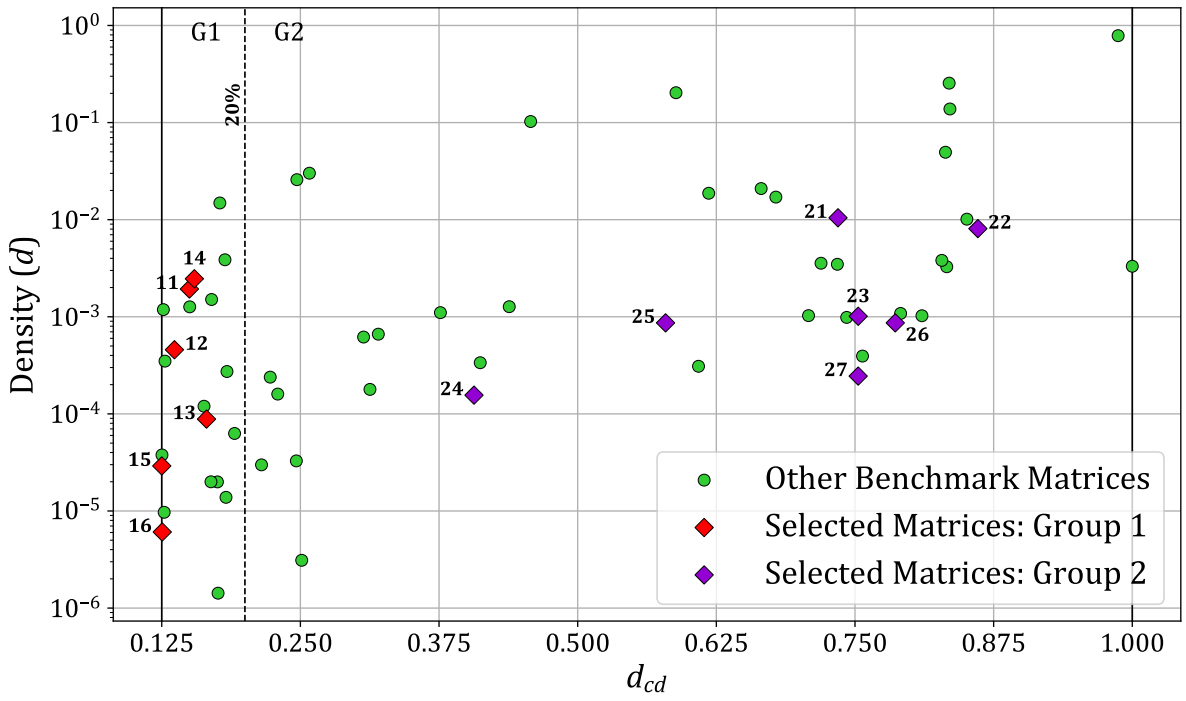


Figure 5.6 – Density Characteristics of Benchmark Matrices: d_{cd} vs Density (d)

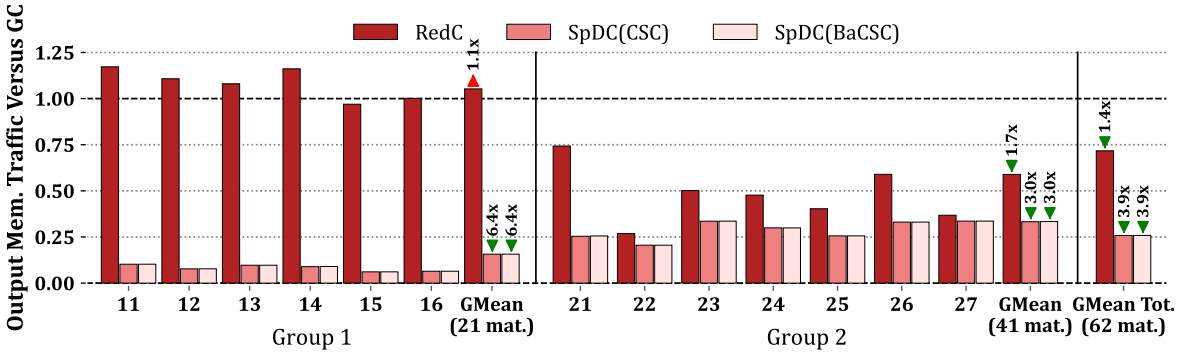


Figure 5.7 – Output Memory Traffic Normalized Compared to GC, Measured from RTL Simulations

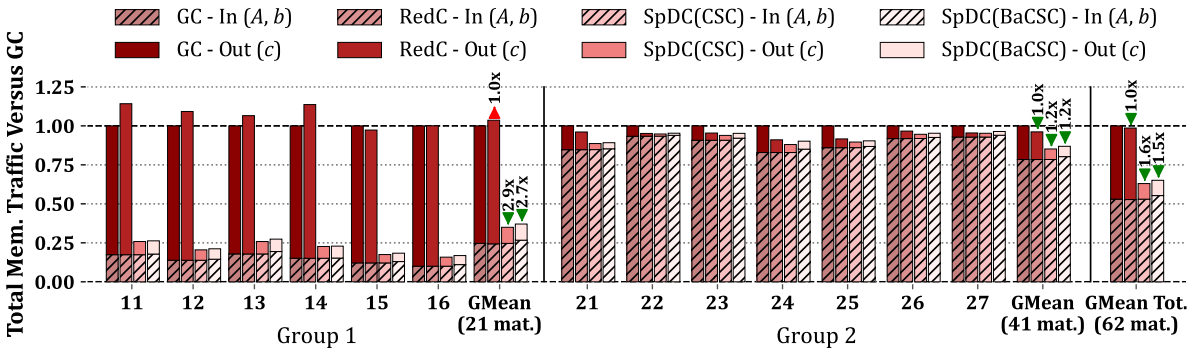


Figure 5.8 – Total Memory Traffic for Input Matrix A, Input Vector b and Output Vector c, Normalized Compared to GC, Measured from RTL Simulations

In Figure 5.8 we show the memory traffic broken down by input data reads and output vector updates. As the two SpDC variants show no difference in output traffic, the total memory traffic shows that the increased matrix storage overhead for SpDC with BaCSC is below 10%. Otherwise, input memory traffic (A and b, *hatched*) is constant on every configurations and both groups of matrices. For GC, input memory traffic normalized per nnz is only 1.1× higher for group 1. Consequently, output traffic dominates GC performance in group 1 (75%) but represents a minority in group 2 (<25%). Thus the output traffic reduction from SpDC has a more significant benefit for the group 1 matrices.

CONCLUSION

SpDC achieves 3.9× output memory traffic reduction versus GC, substantially outperforming RedC's 1.4× reduction. Group 1 matrices (low d_{cd}) benefit most from traffic reduction due to sparse cache-lines, while group 2 (high d_{cd}) shows more modest gains as denser cache-lines are inherently more efficient. The total memory traffic overhead for the BaCSC format is under 10%.

5.3.5.2 Latency Performance

We now analyze total compute time, recalling that all main memory reads incur a fixed 64 cycle latency. Figure 5.9 presents the speedup relative to GC.

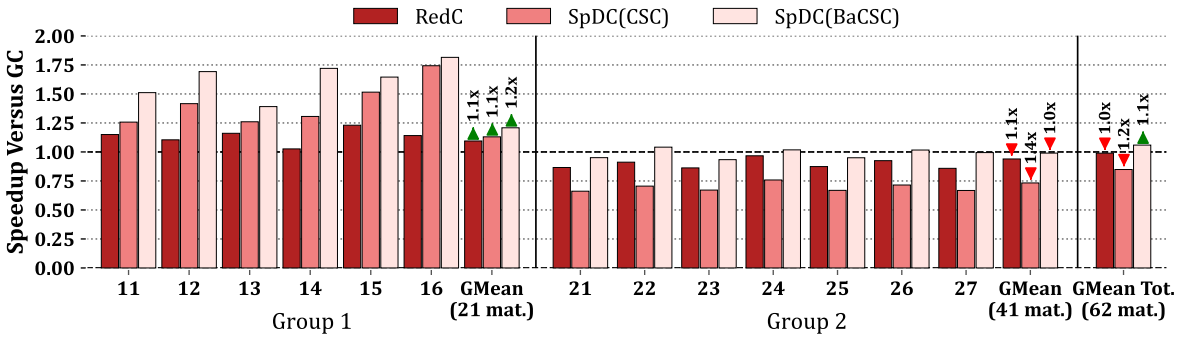


Figure 5.9 – Speedup Compared to GC, Measured from RTL Simulations

Group 1 matrices show speedup across all cache configurations, whereas group 2 shows slowdown. Overall, RedC achieves no speedup over GC, SpDC with CSC incurs 1.2× slowdown, and SpDC with BaCSC achieves 1.1× speedup.

For group 1 matrices, RedC obtains a small speedup (1.1×) because *RED* instructions are "fire-and-forget", replacing multiple read-accumulate-write operations. However, blocking occurs when the *accumulation table* processes prior evictions limiting the benefit. Group 1's sparse cache-lines trigger frequent eviction bursts, dominating the performance and making the system highly memory-bound. Thus, SpDC's substantial memory traffic reduction translates directly to speedup by eliminating costly line-wise evictions. SpDC with BaCSC performs best, with a 1.2× speedup, by eliminating region calculation overhead in the critical loop.

For group 2 matrices, our single-core RISC-V processor becomes compute-bound. Higher cache-line density means evictions are typically followed by non-blocking hits. In the extreme case, 7 hits following an eviction require 56-98 instructions (8-14 instructions per iteration, see Section 5.3.3) before the next potential eviction, sufficient to hide the 64 cycle memory latency.

Consequently, SpDC with BaCSC surpasses GC performance. RedC underperforms by 1.1× due to its 2× smaller *DRC* versus GC. With a larger *DRC*, RedC would yield comparable performance. SpDC with CSC underperforms by 1.4× because, in compute-bound regimes, reducing inner-loop instruction count is critical for performance.

CONCLUSION

For group 1 matrices (low d_{cd}), **SpDC(BaCSC) performs best**, with a 1.2× speedup, by eliminating costly evictions and region computation overhead. For group 2 matrices (high d_{cd}), the workload becomes compute-bound but SpDC(BaCSC) still matches the GC performance. Overall, SpDC(BaCSC) achieves a 1.1× average speedup, while the other configurations underperform on average, with **gains concentrated in memory-bound scenarios**.

5.3.5.3 Further Discussion Regarding the Two Performance Groups

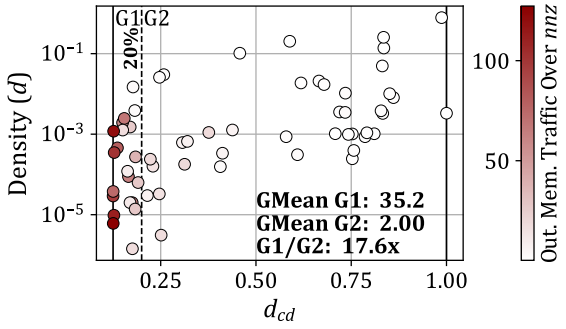
In the two previous sections, we showed performance normalized to GC, highlighting how SpDC affects baseline performance, assuming it was used as an alternative cache. Here, we further analyze performance by normalizing against the number of non-zero elements (nnz), yielding average raw output memory traffic and latency per `OP` SpMV iteration, or equivalently per *reduction* processed. This reveals raw performance variations across matrix groups and configurations (GC, RedC, SpDC(CSC), and SpDC(BaCSC)). Figure 5.10 organizes results as follows: each row represents a configuration; columns show output memory traffic and latency normalized by nnz; dots represent the 62 matrices scattered by density (d) and d_{cd}; red coloring indicates performance data. For both metrics, white indicates better performance, although the scale is not the same across graphs.

We first notice that GC shows group-dependent variation. Indeed, the GC output memory traffic exhibits 17.6× higher traffic on group 1 than group 2. SpDC thus maintains significant relative benefits on both groups of matrices when comparing against GC in previous sections.

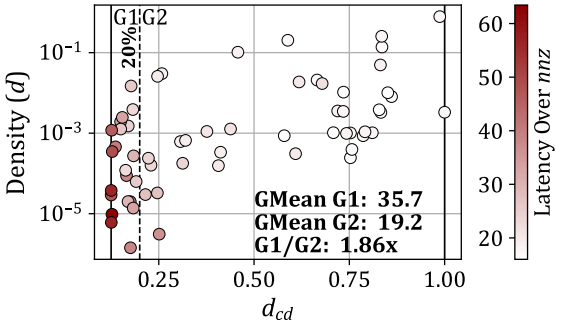
We then observe that group 2 consistently outperforms group 1 across all configurations and metrics, e.g. the *red* points are nearly all group 2 matrices, on the left side. Group 1 exhibits from 8.27× to 31.4× higher memory traffic per nnz and from 1.20× to 1.86× higher latency per nnz than group 2. When d_{cd} is low, cache-lines remain nearly empty, increasing eviction frequency. Additionally, low d_{cd} causes consecutive `OP` SpMV iterations to generate eviction bursts. Consequently, when d_{cd} is low: (1) eviction count increases and (2) evictions cluster temporally. The first effect raises output vector c traffic, potentially triggering critical memory reads that affect latency. The second exacerbates latency: when consecutive iterations process evictions, the intervening ∼10 instructions cannot hide the 64 cycle read latency (insufficient loop unrolling). As visible in Figure 5.10b, worst-case group 1 matrices average 64 cycles latency on GC. Repeatedly waiting for evictions causes processor stalls when d_{cd} is low, making the performance highly memory bound.

As d_{cd} increases, consecutive iterations exhibit cache-line hits. Eviction frequency and burst clustering both decrease. Between successive evictions, hit iterations execute sufficient instructions to overlap with prior read latency. Once the eviction latency falls below the loop iteration time, the workload becomes compute-bound.

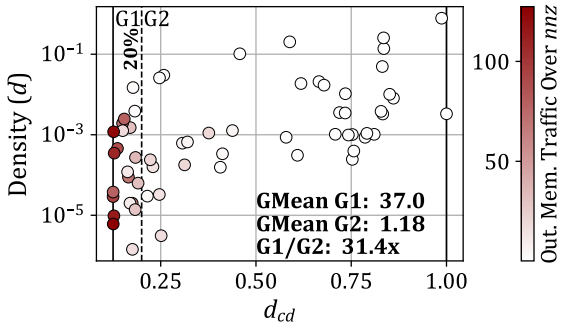
Group 1 benefits significantly from memory traffic reduction, while group 2 is constrained by the latency of the inner-loop in software implementation. Previous sections demonstrated this while taking GC as the performance reference. While GC and RedC remain bottlenecked, SpDC substantially reduces costly evictions and avoids bursting by accumulating with high



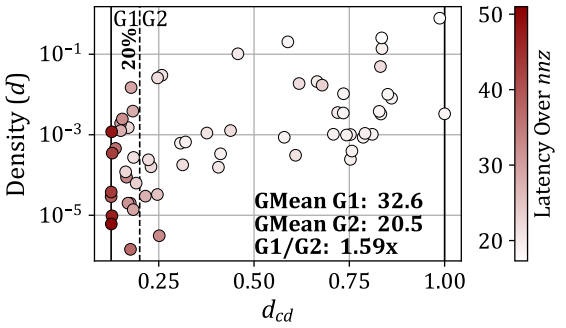
(a) GC Output Memory Traffic Over nnz



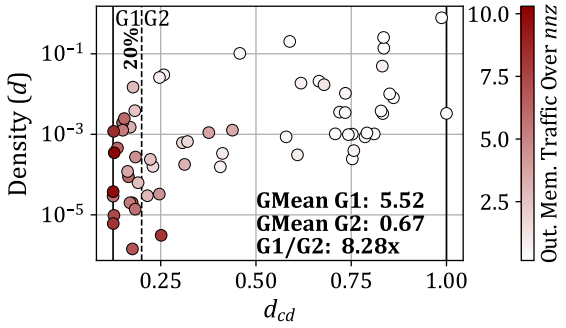
(b) GC Speedup Over nnz



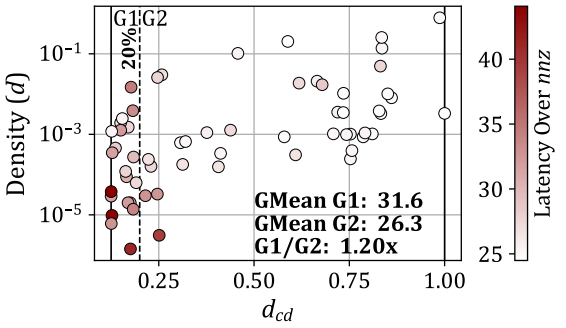
(c) RedC Output Memory Traffic Over nnz



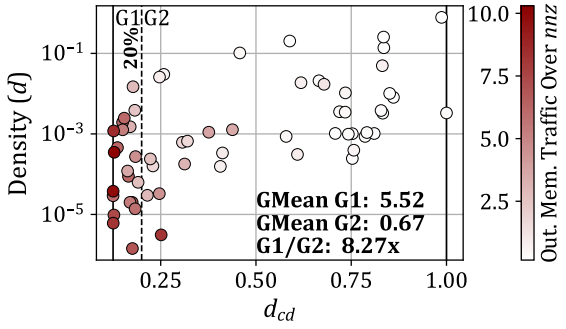
(d) RedC Speedup Over nnz



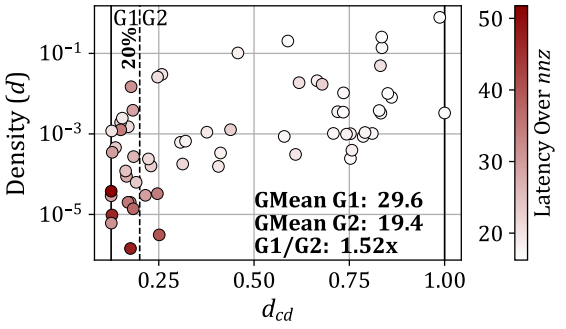
(e) SpDC(CSC) Output Memory Traffic Over nnz



(f) SpDC(CSC) Speedup Over nnz



(g) SpDC(BaCSC) Output Memory Traffic Over nnz



(h) SpDC(BaCSC) Speedup Over nnz

Figure 5.10 – Output Memory Traffic and Speedup Performance, Normalized Over nnz, and Visualized Across Benchmark Matrix Densities and d_{cd}

hit-rates in the *DRC* or evicting via low-cost, element-wise transactions within the *SRT*. Consequently, SpDC delivers significant latency reduction for the matrices in group 1 which are memory bound. For the compute bound matrices in group 2, traffic reduction yields modest latency improvements. Here, the BaCSC format reduces the number of instructions in the inner-loop by avoiding the region calculation.

For GC, loop unrolling allows the *miss handler* to queue up to eight concurrent eviction reads through its multiple entries. However, this proves insufficient for group 1 matrices, as the unrolled inner-loop contains too many instructions to hide the memory latency. Additionally, the *miss handler*'s parallelism cannot be fully exploited if the *accumulation table* lacks sufficient entries to process the corresponding *RED* evictions. Increasing the *accumulation table* to eight entries similarly yields negligible speedup ($< 1\%$) for RedC and SpDC, since the bottleneck remains the critical path length of the inner-loop instructions.

CONCLUSION

Overall, **group 2 always outperforms group 1** when compared with raw memory traffic and latency per nnz. Group 1 matrices (low d_{cd}) are **memory-bound** due to sparse cache-lines causing frequent eviction bursts, while group 2 matrices (high d_{cd}) are **compute-bound** with dense cache-lines hiding eviction latency. Consequently, SpDC achieves speedup in group 1 through memory traffic reduction, but only matches baseline performance in group 2 where the inner-loop latency becomes the bottleneck.

5.3.5.4 Area Overhead

Both HPDcache and SpDCache systems were synthesized in GF22FDX technology using Synopsis DC™. The original area was $0.99\ mm^2$ and the SpDCache version was $1.3\ mm^2$, an increase of 26%. These results are preliminary since the SpDCache RTL implementation has not undergone synthesis optimization. This shows that SpDCache could reasonably be integrated into a processor, providing a large performance improvement ($3.9\times$ in memory traffic), with an area overhead that is much lower than predicted by Borkar's law [15].

5.3.6 Comparison with the Analytical Model

To validate consistency between our models and implementations, we compare the normalized output memory traffic produced by three sources: the analytical model of Chapter 4, the Python model of Chapter 3, and the RTL implementation presented in this chapter. For a fair comparison, the caches for RedC and SpDC are configured to 32 KiB total (see Table 5.1): 50/50/0% for RedC and 25/50/25% for SpDC. The analytical model accepts global parameters and density distributions rather than concrete matrices, so we synthesize banded matrices for the Python and RTL simulations that match those parameters. All synthetic matrices use $M = 10^4$, half-band width $w_{hB} = 1021$, in-band density $d_{IB} = 10^{-2}$, and out-of-band density d_{OB} varying from 10^{-7} to 10^{-2}. Note that at $d_{OB} \simeq 8\times10^{-3}$, approximately one non-zero element exists per column in the out-of-band region, whereas at $d_{OB} \simeq 8\times10^{-7}$, approximately one non-zero element exists in the entire out-of-band region, resulting in a nearly 'perfectly' banded matrix.

Figure 5.11 shows the results: SpDC (*green*) outperforms RedC (*red*) in normalized output memory traffic ($normMT$). Two observations are noteworthy. First, the three approaches agree most closely when d_{OB} is low. Indeed, at $d_{OB} = 10^{-2}$ there is no measurable difference for

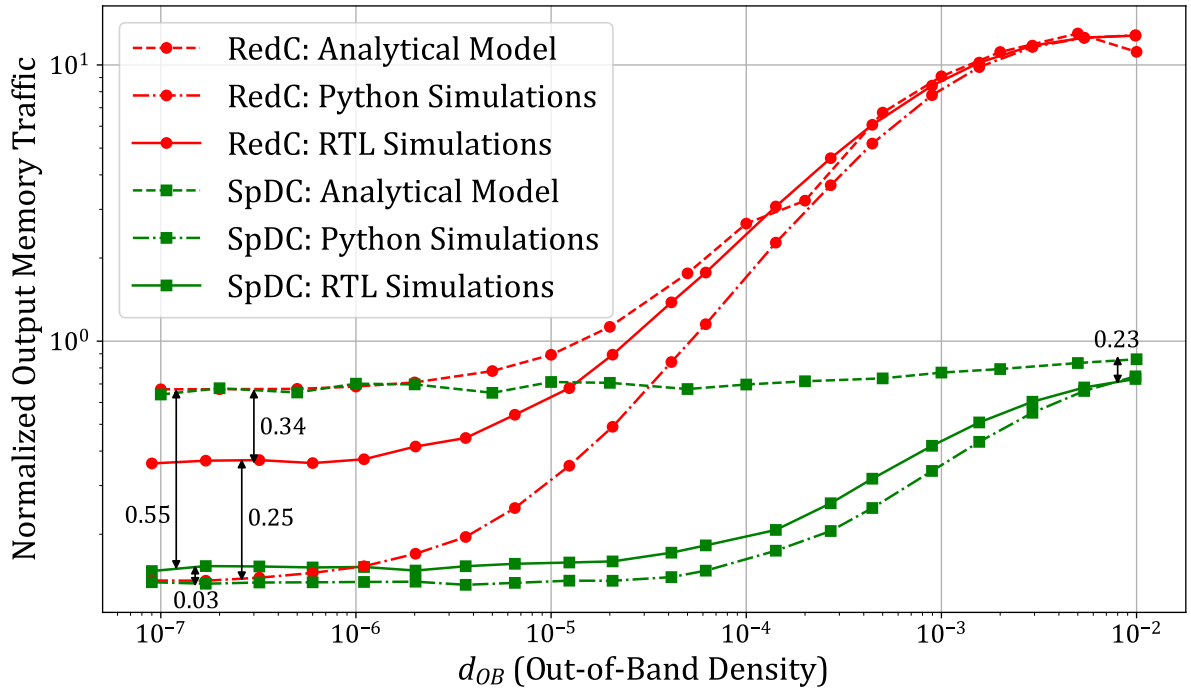


Figure 5.11 – Normalized Output Memory Traffic Depending on the Out-of-Band Density for the RedC and SpDC Configurations, Through Different Simulation Models

RedC, while SpDC exhibits a 0.23 difference in $normMT$. At the smallest d_{OB} the discrepancies reach 0.58 element transferred in memory per *reduction*. Overall trends are hold across models and deviations remain small. Second, the analytical model consistently overestimates traffic, the Python model consistently underestimates it, and the RTL results fall between the two.

CONCLUSION

Across synthetic banded matrices, all three simulation models demonstrate consistent memory traffic results, with only minor divergences as matrices approach the 'perfectly' banded regime.

5.3.7 Comparison with the Python Model

To further validate our implementation over real, not necessarily banded, matrices, we compare the RTL results against the Python simulations from Chapter 3. Table 5.3 presents the Python results for the RTL cache configurations shown in Table 5.1. We include the RedC 50%/50%/0% configuration for comparison, though it is suboptimal according to the Python analysis. Table 5.4 reports RTL results with geometric mean over the same 48 matrices used in Python simulations, comparing output and total memory traffic.

The total memory traffic for RedC and SpDC relative to GC is nearly equivalent across both simulations. However, output memory traffic shows notable differences: RTL achieves better traffic reduction than Python, 1.3× for RedC (vs. 1.0× in Python) and 5.0× for SpDC (vs. 4.1× in Python). Comparing raw traffic values between RTL and Python simulations across all configurations, we observe approximately 25% lower total memory traffic in RTL. Output memory traffic varies by ±15%, with RTL showing lower results for both RedC and SpDC.

Table 5.3 – Results of Main Cache Configurations from Python Simulation

GMeans (48 mat.)		**GC**	**RedC**	**SpDC**
sets	GRC	*128*	*64*	*32*
	DRC	*Ø*	*64*	*64*
	SRT	*Ø*	*Ø*	*32*
Output Memory Traffic (% over GC)		100 ■1.0	103 ■1.0	24.5 ▼4.1
Total Memory Traffic (% over GC)		100 ■1.0	98.5 ■1.0	56.1 ▼1.8

Table 5.4 – RTL Results Comparison with Python Simulations

GMeans (48 mat.)		**GC**	**RedC**	**SpDC**
sets	GRC	*128*	*64*	*32*
	DRC	*Ø*	*64*	*64*
	SRT	*Ø*	*Ø*	*32*
Output Memory Traffic (% over GC)		100 ■1.0	75.5 ▼1.3	20.0 ▼5.0
→ **RTL/Python** (%)		116 ▲0.9	85.0 ▼1.2	94.6 ▼1.1
Total Memory Traffic (% over GC)		100 ■1.0	99.3 ■1.0	54.7 ▼1.8
→ **RTL/Python** (%)		75.5 ▼1.3	76.1 ▼1.3	73.6 ▼1.4

Indeed, the RTL model manages *reductions* out-of-order using the *replay table*, whereas the Python model simulates them sequentially without overlap. For instance, in the Python model, a *RED* triggering an eviction completes entirely before processing the next *RED*. Conversely, in the RTL model, a *RED* triggering an eviction can be queued in the *replay table* if the *miss handler* or *accumulation table* reach capacity, allowing the cache to accept subsequent *RED* operations. If a later *RED* hits the cache-line that would have been evicted by an earlier operation, an unnecessary eviction is avoided, an event that would have been counted in the Python model. This out-of-order execution capability, inherited from HPDcache, thus contributes to reduced memory traffic in the RTL implementation.

CONCLUSION

Over a set of real application matrices, the RTL implementation corroborates the Python simulations, exhibiting slightly better memory traffic performance.

5.4 Conclusion

In this chapter, we presented the microarchitectural implementation of SpDCache and evaluated its performance through RTL simulations. By translating the theoretical insights from previous chapters into an actual hardware design, we demonstrated the effectiveness of SpDCache in addressing the challenges of irregular memory access patterns in sparse matrix computations.

Our RTL implementation of SpDCache, with its three dedicated regions, *GRC*, *DRC*, and *SRT*, successfully optimizes memory transactions for both dense and sparse regions of matrices. The results from RTL simulations highlight significant reductions in memory traffic and improved cache utilization, validating the architectural choices and theoretical models developed earlier. Specifically, SpDCache achieves a substantial decrease in memory traffic compared to generic and *reduction caches*, showcasing its ability to leverage the coarse-grain structure of sparse matrices.

The importance of fine-grain matrix structure is further underscored by our findings. The RTL simulations revealed two distinct groups of matrices, split depending on the d_{cd} metric, which measures the density of non-zero elements within cache-line-sized portions of the matrices. This distinction highlights how the fine-grain distribution of non-zero elements within matrices impacts performance. By effectively handling both coarse-grain and fine-grain structure, SpDCache demonstrates its adaptability and efficiency across a diverse set of sparse matrices.

As a contribution, this chapter detailed the RTL implementation of SpDCache, including the hardware components and pipeline stages that enable efficient data processing and *reduction* operations. Through RTL simulations, we assessed SpDCache's performance, demonstrating its superiority in reducing memory traffic and improving cache utilization. The RTL results validated the theoretical models and insights, confirming the benefits of region-based caching and *reduction* support. Additionally, we identified and analyzed the impact of fine-grain matrix structure on performance, revealing two distinct groups of matrices based on the d_{cd} metric.

However, several limitations must be acknowledged. The current RTL implementation is a first iteration and could be further optimized, as some features are still missing or not fully refined. The use of element-wise, atomic transactions for offloading sparse data carries the risk of overwhelming the rest of the memory hierarchy if it is not designed to handle such transactions efficiently. Finally, the evaluation was conducted on a single-core, single-cache level system, which helps us easily understand the cache behavior but limits raw performance compared to more complex, multi-level, or multi-core architectures. However, we believe that with a more powerful processor (such as multiple-issue), the pressure on the memory system would be higher and the threshold between the two groups of matrices would be shifted, with more matrices benefiting from SpDCache.

Overall, this chapter highlights the critical role of both coarse-grain and fine-grain matrix structure in optimizing sparse matrix computations. The contributions of SpDCache not only advance the state-of-the-art in cache architectures but also provide a robust foundation for future enhancements in scalability and bandwidth efficiency, as discussed in the following chapter. The ability to adapt to varying matrix structure ensures that SpDCache remains effective across a wide range of applications and workloads.

TAKEAWAYS

Contributions:

- **Microarchitectural Design:** We detailed the RTL implementation of SpDCache, including the hardware components and pipeline stages that enable efficient data processing and *reduction* operations.
- **Performance Evaluation:** Through RTL simulations, we assessed SpDCache's performance, demonstrating its ability to reduce memory traffic and improve cache utilization.
- **Validation of Theoretical Models:** The RTL results validated the theoretical models and insights, confirming the benefits of region-based caching and *reduction* support.
- **Fine-Grain Structure Analysis:** We identified and analyzed the impact of fine-grain matrix structure on performance, revealing two distinct groups of matrices based on the d_{cd} metric.

Limitations:

- **RTL Implementation Optimization:** The current RTL implementation is a first iteration and could be further optimized, as some performance feature is still missing or not fully refined.
- **Offloading Risk:** The use of element-wise, atomic transactions for offloading sparse data carries the risk of overwhelming the rest of the memory hierarchy if it is not designed to handle such transactions efficiently.
- **Single-Core, Single-Cache Level System:** The evaluation was conducted on a single-core, single-cache level system, which is ideal for detailed analysis but limits raw performance compared to more recent architectures with multi-level caches and multi-core processors with multi-issue cores.

Chapter 6

SpDCache Optimizations

Contents

6.1 Introduction

THE increasing complexity and scale of sparse matrix computations demand not only efficient single-core architectures but also scalable solutions that can leverage multi-core and multi-level memory hierarchies. While the previous chapters have focused on the design, theoretical analysis, and microarchitectural implementation of SpDCache, this chapter explores opportunities to further enhance its performance and scalability.

In Chapter 5, we demonstrated the effectiveness of SpDCache in reducing memory traffic and improving cache utilization for Sparse Matrix-Vector multiplication (SpMV). However, as applications grow in size and complexity, the need for scalable architectures that can efficiently distribute workloads across multiple cores and memory levels becomes critical. This chapter addresses these challenges by proposing optimizations and extensions to SpDCache that target both single-core and multi-core systems.

We begin by discussing RTL-level optimizations for SpDCache, focusing on fine-tuning the hardware implementation to maximize performance and resource utilization. Next, we explore the application of SpDCache to Sparse Matrix-Sparse Matrix multiplication (SpMSpM), extending its benefits beyond SpMV and demonstrating its adaptability to more complex operations.

Finally, we investigate the integration of SpDCache into multi-level, multi-core architectures. This includes designing a scalable version of SpDCache that can operate efficiently

across multiple cores, as well as optimizing memory bandwidth usage to ensure high throughput in distributed environments. By addressing these aspects, we aim to provide a comprehensive roadmap for scaling SpDCache to meet the demands of modern, data-intensive applications.

Through these optimizations and extensions, this chapter underscores the versatility and scalability of SpDCache, reinforcing its potential as a foundational component for future high-performance sparse matrix processing systems.

6.2 RTL Optimizations for SpDCache

THE RTL implementation of SpDCache provided in Chapter 5 has been enough to get performance results in accordance with all the models, and allowed us to do a precise analysis of the cache performance depending on the matrix structure. However, as already noted several times in the previous chapter, the RTL implementation is missing some features and could be optimized for better practicality and performance. In this section, we detail the major features and optimizations that are still to be implemented.

6.2.1 Floating-Point Support Implementation

The evaluated matrices (Appendix D) include binary, 4-byte integer, and 8-byte floating-point data types. While 8-byte floating-point matrices dominate our dataset and present the worst-case memory traffic due to their word size, we used 8-byte integers as a practical substitute for evaluation. However, floating-point accumulation presents greater hardware complexity than integer accumulation: integer operations guarantee single-cycle latency, whereas floating-point operations can require multiple cycles [48].

To support accumulations between *st1* and *st2* in the pipeline (Section 5.2.3), we can either deploy a single-cycle high-performance FPU with area trade-offs, or extend the pipeline to accommodate two-cycle, for example, floating-point accumulation. The latter approach better suits our *accumulation table*, which does not stall the pipeline and can parallelize element-wise accumulations across stored 64-byte lines. Additionally, full floating-point support requires extending the AXI port to handle element-wise *Read-Modify-Write* (*RMW*) operations beyond its native integer capabilities [2].

Floating-point support is critical for SpDCache deployment. We anticipate that RTL extensions, while time-intensive, should not present fundamental technical obstacles.

CONCLUSION

Floating-point support is achievable through pipeline extension and requires extending the AXI port for element-wise operations, presenting no fundamental obstacles to implementation.

6.2.2 Region-Sizes Configuration

One strength of SpDCache's RTL implementation is its highly configurable design, inherited from the HPDcache implementation used as a foundation. This allowed us to develop a single RTL codebase that could be compiled and simulated for multiple designs, GC, RedC, and SpDC, each with different region size configurations. However, these configurations are static, requiring RTL recompilation for each design variant.

Currently, SpDC's *reduction* mode can be dynamically enabled or disabled via an instruction issued before and after the SpMV kernel. This switches between a generic cache and a hard-coded RTL configuration. To enhance software flexibility, SpDC should support dynamic configuration settings through the initial instruction.

A fundamental limitation is that region sizes are restricted to powers of two, which severely constrains configuration options and led to the 25/50/25% configuration used in the previous chapter. This restriction stems from address decoding that uses bit shifts to extract the *set* index, an approach inherited from HPDcache. Supporting arbitrary region sizes would require

integer division and modulo operations, which would incur significant performance and frequency costs. However, our Python simulations in Chapter 3 demonstrate that precise region sizing is unnecessary for achieving good performance, making this optimization impractical relative to its implementation complexity.

A practical compromise would be to provide one or two predefined configurations for RedC and SpDC, allowing dynamic design selection from software while maintaining a simple user interface.

CONCLUSION

Dynamic region-size configuration through software, while desirable, is constrained by hardware limitations and adds complexity without significant performance gains. A practical approach is to provide **predefined configurations for dynamic selection**, balancing flexibility with implementation simplicity.

6.2.3 Band-Width Calculus in Hardware

SpDCache evaluation shows that depending on matrix characteristics, the `OP` SpMV kernel can be compute-bound, making region calculation a critical software bottleneck. We addressed this by using a custom sparse format that converts latency costs into memory traffic costs. To eliminate any latency or memory traffic overhead, we can compute the region argument directly in hardware. This approach sacrifices software flexibility but enables CSC format compatibility and simplifies the `OP` SpMV kernel to the version shown in Algorithm 5.2. The required SpDCache modifications are:

- Configuration: The initial instruction enabling SpDCache's *reduction* mode must accept a configuration argument specifying the word size in bytes (s_w), the output vector address (c), and the *DRC* size in bytes (s_{DRC}). The hardware could fetch the last parameter independently. Upon issuance with each SpMV, the half-band-width is computed and retained until reconfigured, where s_l denotes cache-line size in bytes:

$$w_{hB} = \frac{s_{DRC} - s_l}{2s_w} + 1$$

- Column indexing: Each outer-loop iteration requires issuing a new instruction containing the current column index $j \in [\![0; M-1]\!]$, denoted $CCOL(j)$. This value persists until the next $CCOL$ instruction arrives, consuming one cycle per outer-loop iteration. Note that the *REG* instruction becomes unnecessary.
- Region determination: For each $RED(addr, val)$ instruction, SpDC computes the current row $i \in [\![0; M-1]\!]$ from the address: $i = \frac{addr - c}{s_w}$, then determines the region as *IBR* if $|i - j| \leq w_{hB}$, otherwise *OBR*.

These modifications impose minimal hardware overhead, with per-*reduction* computations executed at *st0* in the SpDCache pipeline. This feature becomes essential when extending SpDC to multi-level, multi-core architectures (Section 6.4).

CONCLUSION

Hardware-based band-width computation eliminates software bottlenecks, enabling CSC format compatibility and simplifying the SpMV kernel with minimal hardware overhead.

6.2.4 Complete Inter-Region Cache Coherency

When executing an `OP` SpMV kernel with SpDCache, we assume that the processor issues *RED* instructions strictly on addresses of the output vector c, and that loads and stores are issued only on non-c addresses. SpDCache's current implementation provides no hardware safeguards if this assumption is violated. While we provide local coherency between the two *reduction* regions (*DRC* and *SRT*) to prevent c address duplication, we lack coherency with the *GRC*. Implementing global inter-region coherency would require the following modifications:

- Configuration: When enabling *reduction* mode, the configuration parameter must specify the address range for *RED* instructions (e.g., output vector c and its extent).
- Region Assignment: To prevent address duplication between *reduction* regions and the *GRC*, loads and stores on c addresses target *reduction* regions, while *REDs* on non-c addresses target the *GRC*. The following cases apply only when *reduction* mode is enabled.
- Load on c address:
  - Misses in *DRC* and *SRT*: Issue memory read and return data to processor, bypassing cache.
  - Hit in *DRC* or *SRT*: Issue memory read, accumulate with cached data, and return result to processor.

  We do not update *reduction* regions on load to avoid complexity. Updating would require writing zeros to memory to prevent double accumulation upon eviction or flush.
- Store on c address:
  - Misses in *DRC* and *SRT*: Issue memory write with store value, bypassing cache.
  - Hit in *DRC* or *SRT*: Overwrite cached data with store value and issue memory write of zeros to maintain consistency.

  Cache policy (write-through and write-back) can influence this design choice. Crucially, data must not exist in both memory and cache simultaneously.
- *RED* on non-c address:
  - Miss in *GRC*: Issue memory read, accumulate with *RED* value, insert into *GRC* (with eviction if needed), and mark dirty (write-back) or write to memory (write-through).
  - Hit in *GRC*: Accumulate cached data with *RED* value, update *GRC* accordingly, and mark dirty (write-back) or write to memory (write-through).

It is to expect that incorrect region usage increases memory traffic and latency, degrading performance. Implementing this coherency system would ensure deterministic behavior, a critical feature for SpDCache deployment.

CONCLUSION

Complete inter-region cache coherency prevents address duplication across *reduction* and generic regions, ensuring deterministic behavior at minimal hardware cost through address-range configuration and region-specific load/store/reduction handling.

6.3 SpDCache on Matrix-Matrix Multiplication Kernels

To demonstrate SpDCache's applicability beyond the SpMV kernel, we present its use for Sparse Matrix-Sparse Matrix multiplication (SpMSpM, $A \times B = C$), a widely used kernel essential for AI applications.

As discussed in Chapter 1, the *Gustavson* algorithm (`Gust`) offers the best performance

potential for SpMSpM. However, it requires handling irregular accesses to both input matrix B and output matrix C. Since SpDCache targets *reductions* on the output, additional hardware for fetching B is necessary to achieve peak performance, as demonstrated by the Gamma accelerator [127].

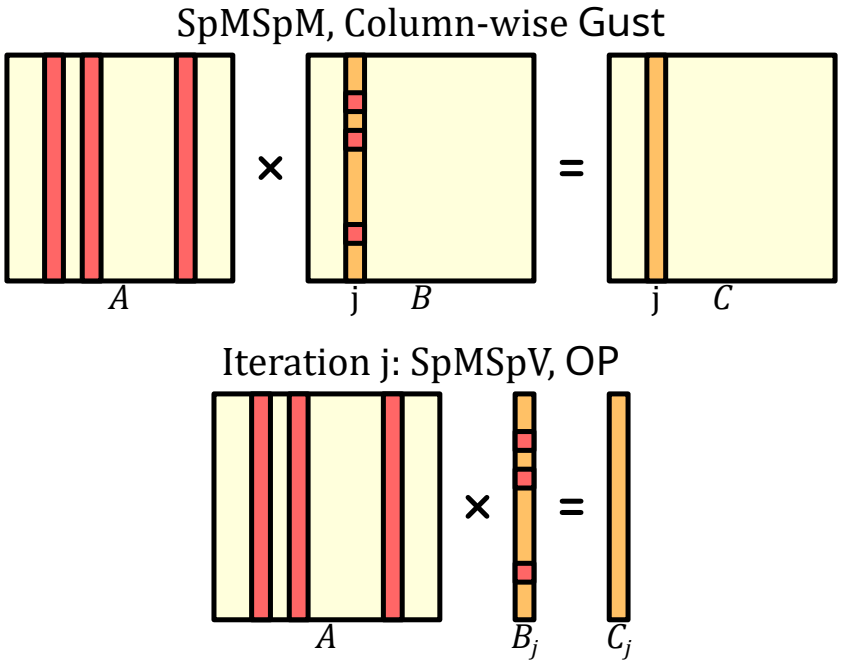


Figure 6.1 – *Gustavson* SpMSpM Kernel with an *Outer-Product* SpMSpV Iteration

A SpMSpM kernel can be decomposed into a series of SpMSpV (Sparse Matrix-Sparse Vector multiplication) operations. As shown in Figure 6.1, each column C_j is computed from column B_j and matrix A, yielding M independent matrix-vector multiplications: $A \times B_j = C_j$. SpDCache readily extends to this scenario.

Since matrix B is sparse, so are its columns. When computing the *Outer-Product* SpMSpV, only columns of A corresponding to non-zero entries in B_j contribute to the result. While SpMV (dense vector) processes the entire matrix structure of A, SpMSpV accesses only a subset of columns, which are not necessarily consecutive. As the band in A is disrupted, SpDCache's hit rate in the *DRC* decreases with sparse vectors, but retains its benefits. When B is banded, this limitation is alleviated as columns of A are consecutive in a subset. This issue vanishes entirely for Sparse Matrix-Dense Matrix multiplication (SpMM).

Explicit flushes are unnecessary between successive SpMSpV operations except the last one, since each iteration updates a distinct address range in C. The next iteration implicitly flushes previous values through replacements and evictions.

This approach suits multicore architectures, where multiple SpMSpV iterations can execute in parallel. The primary hardware challenge is managing shared access to matrix A across cores.

CONCLUSION

SpDCache effectively extends to SpMSpM kernels through decomposition into **SpMSpV operations**, maintaining performance benefits while introducing manageable trade-offs in cache hit rates for sparse vectors. This demonstrates SpDCache's **versatility** across different

sparse linear algebra kernels.

6.4 SpDCache on Multi-Level, Multi-Core System

THROUGHOUT this manuscript, we studied SpDCache in isolation to clearly understand its performance characteristics across different matrices. While we validated SpDCache's individual performance and provided thorough analysis, we must now address how to scale SpDCache to a multi-level, multi-core system. This scaling is non-trivial and requires careful consideration. In this section, we describe a complete memory hierarchy for SpDCache that we believe to be the best, though we do not evaluate it as the full system remains unimplemented.

As a concrete example, we consider a simple three-level, two-core system where each core has two private consecutive caches (L1 and L2) and shares a last-level cache (LLC). As shown in Figure 6.2, each core computes half of the matrix in parallel, divided vertically. This approach favors the CSC format, which stores matrices column-wise, minimizing required software changes. Consequently, each core reads a different portion of matrix A and vector b, while both cores perform updates to the entire output vector c.

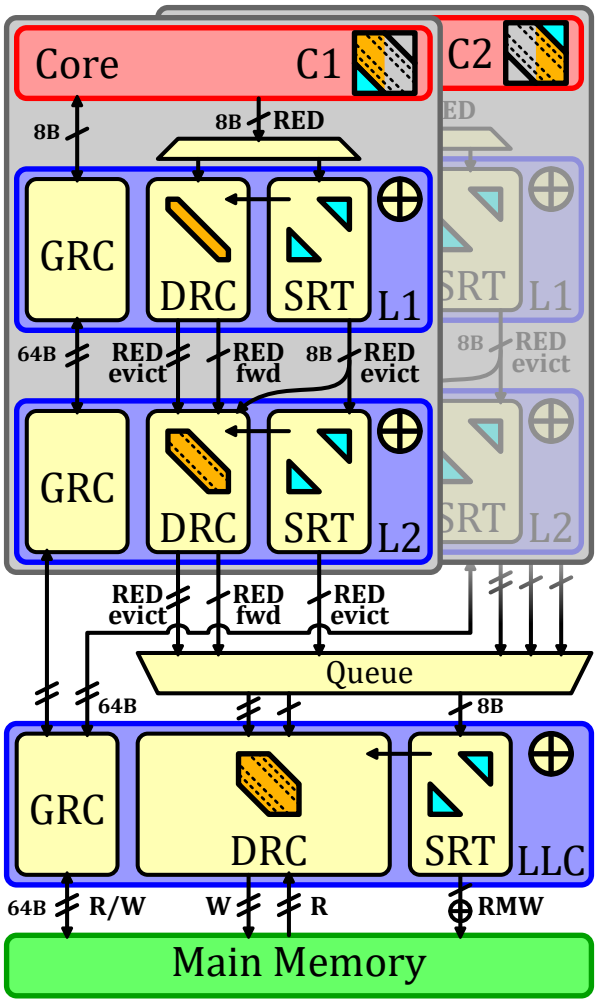


Figure 6.2 – An Example of SpDCache Usage on a Three-Level, Two-Core System

Two key requirements emerge: first, all core updates must eventually be merged into main memory; second, the *reduction* process must remain "fire-and-forget" across cache levels to

prevent stalling the entire memory hierarchy through inter-level dependencies.

Our proposed architecture replaces each cache level with adapted SpDCache versions, where the *DRC* band width scales with cache size. As shown in the figure, the memory hierarchy separates into two parts: *GRCs* function as a conventional memory hierarchy with coherency management, while *DRCs* and *SRTs* manage *reductions* independently. *DRC* band width grows from L1 to LLC, with the LLC defining the matrix band, portions of which cascade to L1 and L2. The out-of-band region is fixed by the LLC and remains constant across levels.

When L1 receives a RED instruction from its core, the SpDCache procedure routes it to the *DRC* or *SRT* (checking the *DRC* first, then the region argument, with *SRT*-to-*DRC* transfers preventing duplication, as seen Chapter 3). If the *reduction* targets in-band data not in L1's sub-band, it forwards to L2. The L1 accumulator processes hits but not *DRC* evictions. Upon *DRC* eviction in L1, the cache-line transfers to L2's *DRC* with a line-wise RED instruction. Upon *SRT* eviction, the element transfers to L2 with an element-wise RED instruction.

L2 behaves similarly to L1 but receives both line-wise and element-wise RED instructions and maintains a wider band. L1 *DRC* evictions thus populate L2's *DRC* alongside forwarded in-band elements, with further forwarding to the LLC if elements exceed L2's band.

The LLC merges core contributions by receiving element-wise and line-wise RED instructions, queuing them in arrival order. It then operates as an isolated SpDCache connected to main memory, managing *DRC* evictions through line-wise reads and writes, and *SRT* evictions through element-wise *RMW* transactions.

RMW transactions can be processed at the LLC level. Since *RMWs* between L1 and the LLC are replaced by element-wise *reductions* with equivalent memory traffic, implementing *RMWs* between the LLC and main memory has minimal impact and can be realized as standard read-accumulate-write operations.

This architecture demonstrates how SpDCache must adapt to its position in the memory hierarchy. With this design, the *reduction* path requires no coherency management, provided software only issues *reduction* on the output vector c. Vertical matrix partitioning enables each core to generate *partial outputs* (*POs*), which the LLC merges in real-time.

This system requires detecting four matrix regions rather than two, from L1's smallest band to the LLC's largest band, plus out-of-band. Software-based region calculation would significantly increase per-iteration latency, and extending the BaCSC format risks substantial memory overhead. Therefore, hardware-based region calculation (Section 6.2) is necessary for scalability.

Note that LLC *DRC* band width calculation differs slightly from other levels: two matrix band portions are simultaneously computed by both cores. The LLC must allocate sufficient memory space for both portions to prevent one core's *reductions* from evicting the other's. Thus, LLC *DRC* band width should be calculated as: *DRC* size divided by the number of cores.

This system carries congestion risk when merging LLC requests, as many element-wise and line-wise *reductions* compete for the accumulator. Further investigation with proper evaluation is warranted.

Alternative configurations merit exploration: for example, SpDCache at L1 with a single band and *reduction*-only caches at L2 and LLC would be simpler but sacrifices the benefit of region separation to minimize unnecessary evictions. Similarly, adjusting band sizing policies involves trade-offs: sizing L1 bands identically across L2 and LLC simplifies region calculation and eliminates RED forwarding but underutilizes L2 and LLC *DRC* space; conversely, sizing by LLC band would cause unwanted L1 and L2 evictions.

Our example vertically partitions the matrix into two parts for simplicity and CSC alignment. However, alternatives exist: for instance, odd and even columns could be assigned to separate cores, remaining CSC-compliant while allowing both cores to do *reductions* on the same addresses at the same time and thus using the entire LLC *DRC* size for the band width. The drawback is reduced L1 and L2 hit rates, concentrating most hits (and accumulations) in the LLC, potentially causing congestion. Determining the optimal approach requires empirical evaluation. Many other matrix partitioning and system configurations are conceivable and warrant investigation, but remain beyond this manuscript's scope.

CONCLUSION

The proposed multi-level, multi-core architecture for SpDCache **effectively addresses the challenges of scaling across cache levels and cores** while optimizing memory usage. Future work should focus on empirical evaluations of various configurations to validate performance and efficiency.

6.5 Conclusion

THIS chapter explored optimizations and extensions to SpDCache, focusing on enhancing its performance and scalability for sparse matrix computations. We refined the RTL-level implementation to maximize efficiency and resource utilization, ensuring SpDCache remains robust and adaptable to modern computational demands.

We extended SpDCache to matrix-matrix multiplication kernels, demonstrating its versatility beyond SpMV. This extension highlights its adaptability to complex operations, reinforcing its potential as a foundational component for diverse sparse matrix computations.

A key focus was integrating SpDCache into multi-level, multi-core architectures, addressing the need for scalable, distributed processing. By optimizing memory traffic and parallel processing capabilities, SpDCache becomes a viable solution for large-scale, high-performance applications.

While this chapter presents theoretical insights and proposed optimizations, no evaluation has been conducted yet. The work aims to guide future research and implementation, with proper evaluation and analysis to conduct in subsequent studies. These advancements position SpDCache as a versatile and scalable solution for modern sparse matrix processing systems, paving the way for future research in high-performance architectures.

TAKEAWAYS

SpDCache Optimizations:

- **Floating-Point Support:** Handle floating-point arithmetic within SpDCache;
- **Region-Size Configuration:** Methods to dynamically adjust region sizes in SpDCache;
- **Band-Width Calculs in Hardware:** Hardware-based calculations for *DRC* band-width;
- **Complete Inter-Region Cache Coherency:** Comprehensive cache coherency protocol for SpDCache, ensuring data integrity and consistency across regions.

SpDCache on Matrix-Matrix Kernels:

- SpDCache effectively **extends to matrix-matrix kernels** through decomposition into matrix-vector operations;
- This demonstrates SpDCache's versatility across different sparse linear algebra kernels.

Multi-Level, Multi-Core SpDCache Architecture:

- SpDCache can be **scaled to multi-level, multi-core systems** by adapting slightly its architecture to each cache level and core;
- Vertical matrix partitioning enables efficient parallel processing across cores;
- The proposed architecture effectively addresses the challenges of scaling across cache levels and cores while optimizing memory usage.

Conclusion

Contents

1 TODO

TODO

Synthèse en Français

Sommaire

1 TODO

TODO

Appendices

Contents

A Sparse Formats

TODO

COO The simplest *sparse format* is called *COO* (*COOrdinate*) [22, 93, 110]. It consists in three separate tables, one filled only with the non-zero elements of the matrix, a row index table containing the row of the corresponding elements, and similarly a column index table (see Figure **??**). The two latter are *metadata* tables and all three tables are of size *nnz*. In this format, the elements are not necessarily sorted, the memory footprint does not depend on the structure of the matrix and all elements are stored independently of each other.

CSR & CSC The most commonly used *sparse format* is called *CSR* (*Compressed Sparse Row*) [12, 22, 93, 110]. It is similar to the *COO* format except that the row index table is replaced by a set of indices that point to the position of the start of each row in the column index table (see Figure **??**). The size of this table is the number of rows plus one, and, in practical cases, the memory footprint is smaller than with *COO*. However, the non-zero elements need to be sorted in *row-wise* order. This format has a *column-wise* variant called *CSC* (*Compressed Sparse Column*) where the column index table stores the positions of columns instead of rows. These two formats are symmetrical but with *CSR*, it is the traversal of rows that is efficient because they are sequential whereas in *CSC*, column traversal is efficient.

BCSR Even in sparse matrices, there can be regions, or blocks, that have a large fraction of non-zero values. In this case, the *BCSR* format [12], which is an extension of the *CSR* format with *tiling*, can be attractive. We use the term *tiling* to describe a level of hierarchy in a storage format. That is, a *tile* refers to a sub-matrix, which itself may be dense, as in the case of *BCSR*, or which could be sparse. With the *BCSR* format, we simply replace the non-zero elements of the *CSR* format with dense, fixed-size sub-matrices (see Figure **??**). Within these sub-matrices, it is possible to have some zero elements. The advantage of the *BCSR* format is that the dense sub-matrices can be processed using techniques optimized for dense matrices, particularly using vector units or GPUs [52]. Compared to the *CSR* format, *BCSR* makes it possible to accelerate the computation within the dense *tiles*, but this comes with the overhead of storing some zero values. Not all matrices are well suited to the *BCSR* format.

DIA For diagonally structured matrices ($\forall (i,j) \in [\![0,M]\!] \times [\![0,N]\!],\ a_{i,j} = 0\ for\ |i-j| > K$), a format called *DIA* (*DIAgonal*) is particularly well suited [12, 22, 93]. It consists in storing the values along the $2 \times K + 1$ diagonals in a table and keeping information about the position of the diagonals in the matrix (see Figure **??**). The organization of the non-zero data and selection of the diagonals can be tailored to the matrices and access patterns. This format is well suited to diagonal matrices, has a low overhead for storing indices and can be combined with other formats to better fit more complex structures.

ELL Another format called *ELL* (*ELLPACK*) [12, 22, 60, 93] is well suited when the number of non-zero elements per row can be bounded by K. In this case, the non-zero values in a $M \times N$ matrix can be stored as a dense $M \times K$ matrix, augmented with an index table, of the same dimensions, which gives the column position of each non-zero value (see Figure **??**). This format is not efficient if the number of non-zero elements per row is highly variable, as

many entries in the $M \times K$ table are unused. This format is well suited to GPUs [20, 112] since it transforms a sparse matrix into a dense one in memory.

HYB The *HYB* format (HYBrid) [14] is an example of a format which combines *ELL* and *COO*. The *ELL* format can be used, but with a value of K (the width of the value table) that is smaller than the maximum non-zero entries per row in the original matrix (see Figure **??**). For the small number of rows where the table lacks a sufficient number of columns, the *COO* format can be used to store the extra values. In this way, the storage efficiency of *ELL* is improved, and it can be applied to a broader class of matrices.

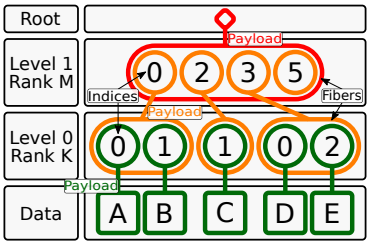


Figure B.1 – Example of the *FiberTree* Representation with CSR Format

B A Generic Representation for *Sparse Formats*

A visual representation of compressed matrices, called *FiberTree* [108], is helpful to understand these sparse formats. Each *rank* of a matrix represents a level of a tree where the nodes are filled with either the metadata of the child nodes or the matrix data which can only appear in the leaves (see Figure B.1). With this representation, we clearly identify the number of *ranks*, which define the *fibers*. A *fiber* is the generic term to describe a one-dimensional stream of data or metadata, which corresponds to a row or column in the case of a two-dimensional matrix. A stream of indices is a *fiber* of metadata.Figure B.1 presents, as an example, a *FiberTree* representation of the matrix of Figure **??**, matching the CSR format: each *rank* in the tree corresponds to a *fiber* stored in the table *Row PTR* or *Col IDX* of Figure 1.3b.

In a recent work [118], a systematic nomenclature for *sparse matrix (and tensor) formats* was proposed, based on *FiberTree* representation. In fact, in the example of Figure B.1, the *fibers* are compressed with different methods. Thus each *rank* can be characterized by a label indicating how it is compressed: *Uncompressed* (*U*), *Coordinate Payload* (*CP*), *Uncompressed Offset Pairs* (*UOP*). *U* defines *ranks* where all data values are stored contiguously in memory. *CP* defines *ranks* that store the indices of each payload, a payload being either a non-zero element or a sub-matrix[1]. *UOP* defines *ranks* that store pointers (offsets) to delimit sub-matrices of the next *rank*[2]. These are the three main ones as they are used in the most common sparse formats but many others exist or can be created.

With this nomenclature, the sparse formats can be classified according to their *rank* compression methods. For example, the CSR format is denoted as *(UOP, CP) row-major* since its *Row PTR* table stores pointers (offsets) to rows. The CSR and CSC formats are both of type *(UOP, CP)*, one being *row-major*, the other *column-major*. The COO format is denoted as CP^2 because there are two coordinates (row and column) for each non-zero element, as it is a *CP fiber* having two indices per data element instead of one. User defined formats can also be easily described and named. For example, a user could define a storage format with a nested CSR. That is, each non-zero element in the outer CSR, corresponds to a sub-matrix, itself stored in CSR format. Such a representation would be labeled *(UOP, CP, UOP, CP)*. A similar nomenclature is proposed by TACO [22]. It is more adapted to code generation applications, where the *FiberTree*-based nomenclature better describes the memory storage of the sparse format. For example, *U* corresponds to *Dense*, *CP* to *Singleton*, the pair *(UOP, CP)* is translated

1. In this section, we use the term sub-matrix but this is extended to the notion of *tiling* in Section 1.4.1.
2. If some sub-matrices are empty, pointers will be repeated, hence 'uncompressed'.

as *Compressed* while *UOP* alone corresponds to *Offset*. We chose to adopt the *FiberTree*-based nomenclature in this article since it is more suited to a hardware context.

C Extended Overview of the State-of-the-Art Accelerators for Sparse Computing

TODO

D Sparse Matrices Used for the Evaluations

FOR transparency, we gathered in Table D.1 all matrices used in our simulations. This includes matrices from the SuiteSparse Matrix Collection [63] and 20 generated matrices, totaling 87. We provide their dimensions, density, density within in- and out-of-band regions (with half-band width 1021), and d_{cd}. We also indicate which evaluations use each matrix (Python or RTL; the analytical model does not require real matrices).

Table D.1 – Matrices Used in Every Evaluations and Their Characteristics

Name	**Src**	***M***	***nnz***	***d***	d_{IB} (with $w_{hB} = 1021$)	d_{OB} (with $w_{hB} = 1021$)	d_{cd}	**Eval.**	
								Py.	**RTL**
west0381	SS	381	2.16 k	1.49 e-2	1.49 e-2	0.00 e-0	18%		✓
mhda416	SS	416	8.56 k	4.95 e-2	4.95 e-2	0.00 e-0	83%		✓
rotor2	SS	791	10.7 k	1.71 e-2	1.71 e-2	0.00 e-0	68%		✓
filter2D	SS	1.67 k	10.8 k	3.86 e-3	3.86 e-3	0.00 e-0	18%		✓
Journals	SS	124	12.1 k	7.85 e-1	7.85 e-1	0.00 e-0	99%		✓
Trefethen_700	SS	700	12.7 k	2.58 e-2	2.58 e-2	0.00 e-0	25%		✓
CAG_mat364	SS	364	13.6 k	1.03 e-1	1.03 e-1	0.00 e-0	46%		✓
Si2	SS	769	17.8 k	3.01 e-2	3.01 e-2	0.00 e-0	26%		✓
bcsstk10	SS	1.09 k	22.1 k	1.87 e-2	1.87 e-2	0.00 e-0	62%		✓
qc324	SS	324	26.7 k	2.55 e-1	2.55 e-1	0.00 e-0	83%		✓
ex2	SS	441	26.8 k	1.38 e-1	1.38 e-1	0.00 e-0	84%		✓
LeGresley_4908	SS	4.91 k	30.5 k	1.27 e-3	1.39 e-3	1.19 e-3	15%	✓	✓
mbeacxc	SS	496	49.9 k	2.03 e-1	2.03 e-1	0.00 e-0	59%		✓
bcsstk13	SS	2.00 k	83.9 k	2.09 e-2	2.73 e-2	7.21 e-4	67%		✓
wiki-Vote	SS	8.30 k	104 k	1.51 e-3	3.55 e-3	8.92 e-4	17%	✓	✓
p2p-Gnutella31	SS	62.6 k	148 k	3.78 e-5	2.60 e-4	3.03 e-5	13%	✓	✓
ca-CondMat	SS	23.1 k	187 k	3.49 e-4	3.96 e-4	3.45 e-4	13%	✓	✓
Alice_1021	Gen	10.0 k	193 k	1.93 e-3	9.98 e-3	9.00 e-8	14%	✓	✓
Bob_1021	Gen	10.0 k	193 k	1.93 e-3	9.98 e-3	1.70 e-7	14%	✓	✓
Charlie_1021	Gen	10.0 k	193 k	1.93 e-3	9.98 e-3	3.20 e-7	14%	✓	✓
Diana_1021	Gen	10.0 k	193 k	1.93 e-3	9.98 e-3	6.10 e-7	14%	✓	✓
Ethan_1021	Gen	10.0 k	193 k	1.93 e-3	9.98 e-3	1.12 e-6	14%	✓	✓
Fiona_1021	Gen	10.0 k	193 k	1.93 e-3	9.98 e-3	2.06 e-6	13%	✓	✓
George_1021	Gen	10.0 k	194 k	1.94 e-3	9.98 e-3	3.76 e-6	14%	✓	✓
Hannah_1021	Gen	10.0 k	194 k	1.94 e-3	9.98 e-3	6.88 e-6	14%	✓	✓
Isaac_1021	Gen	10.0 k	194 k	1.94 e-3	9.98 e-3	1.24 e-5	14%	✓	✓

Continued on next page

Table D.1 – Matrices Used in Every Evaluations and Their Characteristics (Continued)

Name	Src	M	nnz	d	d_{IB} (with $w_{hB} = 1021$)	d_{OB} (with $w_{hB} = 1021$)	d_{cd}	Eval. Py.	Eval. RTL
Julia_1021	Gen	10.0 k	195 k	1.95 e-3	9.98 e-3	2.07 e-5	13%	✓	✓
Kevin_1021	Gen	10.0 k	197 k	1.97 e-3	9.98 e-3	4.13 e-5	13%	✓	✓
Laura_1021	Gen	10.0 k	198 k	1.98 e-3	9.98 e-3	6.20 e-5	13%	✓	✓
Michael_1021	Gen	10.0 k	205 k	2.05 e-3	9.98 e-3	1.42 e-4	13%	✓	✓
Nina_1021	Gen	10.0 k	215 k	2.15 e-3	9.98 e-3	2.64 e-4	13%	✓	✓
Oliver_1021	Gen	10.0 k	228 k	2.28 e-3	9.98 e-3	4.34 e-4	13%	✓	✓
TSOPF_RS_b39_c7	SS	14.1 k	252 k	1.27 e-3	1.03 e-3	1.31 e-3	44%	✓	✓
Paula_1021	Gen	10.0 k	265 k	2.65 e-3	9.98 e-3	8.86 e-4	13%	✓	✓
Quentin_1021	Gen	10.0 k	324 k	3.24 e-3	9.98 e-3	1.62 e-3	13%	✓	✓
poisson3Da	SS	13.5 k	353 k	1.93 e-3	2.71 e-3	1.80 e-3	15%	✓	✓
cit-HepTh	SS	27.8 k	353 k	4.57 e-4	1.05 e-3	4.11 e-4	14%	✓	✓
email-Enron	SS	36.7 k	368 k	2.73 e-4	1.69 e-3	1.91 e-4	18%	✓	✓
Rachel_1021	Gen	10.0 k	428 k	4.28 e-3	9.98 e-3	2.91 e-3	13%	✓	✓
bcsstk17	SS	11.0 k	429 k	3.56 e-3	2.01 e-2	0.00 e-0	72%	✓	✓
soc-Epinions1	SS	75.9 k	509 k	8.84 e-5	7.00 e-4	7.16 e-5	17%	✓	✓
patents_main	SS	241 k	561 k	9.69 e-6	2.00 e-8	9.78 e-6	13%	✓	✓
Samuel_1021	Gen	10.0 k	628 k	6.28 e-3	9.98 e-3	5.39 e-3	13%	✓	✓
m133-b3	SS	200 k	801 k	2.00 e-5	9.97 e-6	2.01 e-5	17%	✓	✓
scircuit	SS	171 k	959 k	3.28 e-5	1.83 e-3	1.11 e-5	25%	✓	✓
Tina_1021	Gen	10.0 k	998 k	9.98 e-3	9.98 e-3	9.98 e-3	14%	✓	✓
EternityII_Etilde *	SS	10.1 k	1.17 M	5.70 e-4	2.22 e-5	2.81 e-5	13%	✓	
Maragal_7 *	SS	46.8 k	1.20 M	9.65 e-4	8.63 e-4	1.77 e-3	24%	✓	
msc10848	SS	10.8 k	1.23 M	1.05 e-2	2.10 e-2	8.15 e-3	73%	✓	✓
mac_econ_fwd500	SS	207 k	1.27 M	2.99 e-5	2.99 e-3	4.10 e-7	21%	✓	✓
NotreDame_actors *	SS	392 k	1.47 M	2.93 e-5	1.68 e-4	8.87 e-5	16%	✓	
raefsky3	SS	21.2 k	1.49 M	3.31 e-3	3.51 e-2	1.41 e-5	100%	✓	✓
lhr71	SS	70.3 k	1.53 M	3.09 e-4	2.02 e-3	2.58 e-4	61%		✓
2cubes_sphere	SS	101 k	1.65 M	1.60 e-4	6.16 e-3	3.73 e-5	23%	✓	✓
opt1	SS	15.4 k	1.93 M	8.09 e-3	5.44 e-2	1.31 e-3	86%	✓	✓
bcsstk32	SS	44.6 k	2.01 M	1.01 e-3	1.95 e-2	1.35 e-4	75%	✓	✓
cage12	SS	130 k	2.03 M	1.20 e-4	4.23 e-3	5.46 e-5	16%	✓	✓
sme3Db	SS	29.1 k	2.08 M	2.46 e-3	3.08 e-3	2.42 e-3	15%	✓	✓

Continued on next page

Table D.1 – Matrices Used in Every Evaluations and Their Characteristics (Continued)

Name	Src	M	nnz	d	d_{IB} (with $w_{hB} = 1021$)	d_{OB} (with $w_{hB} = 1021$)	d_{cd}	Eval. Py.	Eval. RTL
mario002	SS	390 k	2.10 M	1.38 e-5	8.98 e-4	9.17 e-6	18%	✓	✓
Stanford	SS	282 k	2.31 M	2.91 e-5	2.61 e-5	2.91 e-5	13%		✓
rma10	SS	46.8 k	2.37 M	1.08 e-3	1.74 e-2	3.47 e-4	79%	✓	✓
cop20k_A	SS	121 k	2.62 M	1.79 e-4	2.80 e-3	1.34 e-4	31%	✓	✓
ct20stif	SS	52.3 k	2.70 M	9.85 e-4	1.96 e-2	2.38 e-4	74%		✓
filter3D	SS	106 k	2.71 M	2.39 e-4	8.15 e-3	8.50 e-5	22%	✓	✓
ramage02	SS	16.8 k	2.87 M	1.01 e-2	3.77 e-2	6.44 e-3	85%	✓	✓
webbase-1M	SS	1.00 M	3.11 M	3.11 e-6	6.55 e-4	1.77 e-6	25%	✓	✓
amazon0312	SS	401 k	3.20 M	1.99 e-5	6.88 e-4	1.65 e-5	18%	✓	✓
xenon2	SS	157 k	3.87 M	1.56 e-4	1.06 e-2	1.88 e-5	41%		✓
cant	SS	62.5 k	4.01 M	1.03 e-3	3.17 e-2	0.00 e-0	71%	✓	✓
vsp_bcsstk30_[...]	SS	58.3 k	4.03 M	1.18 e-3	1.18 e-3	1.18 e-3	13%	✓	✓
offshore	SS	260 k	4.24 M	6.29 e-5	5.15 e-3	2.26 e-5	19%	✓	✓
gupta2	SS	62.1 k	4.25 M	1.10 e-3	3.88 e-3	1.01 e-3	38%	✓	✓
pdb1HYS	SS	36.4 k	4.34 M	3.28 e-3	4.97 e-2	5.61 e-4	83%	✓	✓
ship_001	SS	34.9 k	4.64 M	3.81 e-3	3.35 e-2	1.99 e-3	83%	✓	✓
web-Google	SS	916 k	5.11 M	6.08 e-6	6.06 e-6	6.08 e-6	13%	✓	✓
roadNet-CA	SS	1.97 M	5.53 M	1.42 e-6	1.16 e-3	2.20 e-7	18%	✓	✓
consph	SS	83.3 k	6.01 M	8.65 e-4	2.27 e-2	3.21 e-4	58%	✓	✓
shipsec1	SS	141 k	7.81 M	3.94 e-4	1.12 e-2	2.35 e-4	76%	✓	✓
Ge87H76	SS	113 k	7.89 M	6.18 e-4	1.38 e-2	3.77 e-4	31%	✓	✓
degme *	SS	186 k	8.13 M	6.64 e-5	1.37 e-5	1.87 e-5	23%	✓	
Ge99H100	SS	113 k	8.45 M	6.62 e-4	1.42 e-2	4.14 e-4	32%	✓	✓
m_t1	SS	97.6 k	9.75 M	1.02 e-3	2.62 e-2	4.89 e-4	81%	✓	✓
x104	SS	108 k	10.2 M	8.66 e-4	2.60 e-2	3.86 e-4	79%	✓	✓
ohne2	SS	181 k	11.1 M	3.36 e-4	8.91 e-3	2.39 e-4	41%		✓
pwtk	SS	218 k	11.6 M	2.45 e-4	2.52 e-2	1.01 e-5	75%	✓	✓
crankseg_2	SS	63.8 k	14.1 M	3.47 e-3	3.70 e-2	2.37 e-3	73%		✓
cit_Patents	SS	3.77 M	16.5 M	1.16 e-6	0.00 e-0	1.16 e-6	13%	✓	

SS: SuiteSparse Matrix Collection [63]; Gen: Generated; *not square

E Analytical Model of a *Reduction Cache* Computing an `OP` SpMV Kernel

TODO

F Python Model of a Data-Cache with Support for *Reductions*

TODO

G Statistics

TODO (get infos before end of contract)
(commits, nb. of lines, bugs, timeline -> into figures?)

List of Figures

List of Tables

List of Algorithms

List of Source Code

Bibliography

[1] James Alfred Ang, Brian W. Barrett, Kyle Bruce Wheeler, and Richard C Murphy. 2010. *Introducing the graph 500.* Technical Report. United States.

[2] ARM IHI 0022. 2023. ARM – AMBA AXI Protocol Specification. (Sept. 2023). https://developer.arm.com/documentation/ihi0022/latest https://developer.arm.com/documentation/ihi0022/latest.

[3] Bahar Asgari, Ramyad Hadidi, Tushar Krishna, Hyesoon Kim, and Sudhakar Yalamanchili. 2020. ALRESCHA: A Lightweight Reconfigurable Sparse-Computation Accelerator. In *2020 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. 249–260. https://doi.org/10.1109/HPCA47549.2020.00029

[4] Venkitesh Ayyar, Evan Weinberg, Richard C. Brower, M.A. Clark, and Mathias Wagner. 2023. Optimizing Staggered Multigrid for Exascale performance. In *Proceedings of The 39th International Symposium on Lattice Field Theory — PoS(LATTICE2022)*, Vol. 430. 335. https://doi.org/10.22323/1.430.0335

[5] Ariful Azad, Aydin Buluç, and John Gilbert. 2015. Parallel Triangle Counting and Enumeration Using Matrix Algebra. In *2015 IEEE International Parallel and Distributed Processing Symposium Workshop*. 804–811. https://doi.org/10.1109/IPDPSW.2015.75

[6] Ariful Azad, Georgios A Pavlopoulos, Christos A Ouzounis, Nikos C Kyrpides, and Aydin Buluç. 2018. HipMCL: a high-performance parallel implementation of the Markov clustering algorithm for large-scale networks. *Nucleic Acids Research* 46, 6 (01 2018), e33–e33. https://doi.org/10.1093/nar/gkx1313 arXiv:https://academic.oup.com/nar/article-pdf/46/6/e33/24525991/gkx1313.pdf

[7] Daehyeon Baek, Soojin Hwang, Taekyung Heo, Daehoon Kim, and Jaehyuk Huh. 2021. InnerSP: A Memory Efficient Sparse Matrix Multiplication Accelerator with Locality-Aware Inner Product Processing. In *2021 30th International Conference on Parallel Architectures and Compilation Techniques (PACT)*. 116–128. https://doi.org/10.1109/PACT52795.2021.00016

[8] Ricardo Baeza-Yates and Alejandro Salinger. 2010. *Fast Intersection Algorithms for Sorted Sequences*. Springer Berlin Heidelberg, Berlin, Heidelberg, 45–61. https://doi.org/10.1007/978-3-642-12476-1_3

[9] Vignesh Balaji, Neal Crago, Aamer Jaleel, and Brandon Lucia. 2021. P-OPT: Practical Optimal Cache Replacement for Graph Analytics. In *2021 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. 668–681. https://doi.org/10.1109/HPCA51647.2021.00062

[10] Rajeev Balasubramonian, Andrew B. Kahng, Naveen Muralimanohar, Ali Shafiee, and Vaishnav Srinivas. 2017. CACTI 7: New Tools for Interconnect Exploration in Innovative Off-Chip Memories. *ACM Trans. Archit. Code Optim.* 14, 2, Article 14 (jun 2017), 25 pages. https://doi.org/10.1145/3085572

[11] Adrián Barredo, Jonathan C. Beard, and Miquel Moretó. 2019. POSTER: SPiDRE: Accelerating Sparse Memory Access Patterns. In *2019 28th International Conference on Parallel Architectures and Compilation Techniques (PACT)*. Institute of Electrical and Electronics Engineers, New York, NY, USA, 483–484. https://doi.org/10.1109/PACT.2019.00056

[12] Richard Barrett, Michael Berry, Tony F. Chan, James Demmel, June Donato, Jack Dongarra, Victor Eijkhout, Roldan Pozo, Charles Romine, and Henk van der Vorst. 1994. *Templates for the Solution of Linear Systems: Building Blocks for Iterative Methods*. Society for Industrial and Applied Mathematics. https://doi.org/10.1137/1.9781611971538

[13] Scott Beamer, Krste Asanović, and David Patterson. 2017. The GAP Benchmark Suite. (2017). https://doi.org/10.48550/arXiv.1508.03619 arXiv:1508.03619 [cs.DC]

[14] Nathan Bell and Michael Garland. 2009. Implementing Sparse Matrix-Vector Multiplication on Throughput-Oriented Processors. In *Proceedings of the Conference on High Performance Computing Networking, Storage and Analysis* (Portland, Oregon) *(SC '09)*. Association for Computing Machinery, New York, NY, USA, Article 18, 11 pages. https://doi.org/10.1145/1654059.1654078

[15] Shekhar Borkar. 2007. Thousand core chips: a technology perspective. In *Proceedings of the 44th Annual Design Automation Conference* (San Diego, California) *(DAC '07)*. Association for Computing Machinery, New York, NY, USA, 746–749. https://doi.org/10.1145/1278480.1278667

[16] Achi Brandt and Dorit Ron. 2003. *Multigrid Solvers and Multilevel Optimization Strategies*. Springer US, Boston, MA, 1–69. https://doi.org/10.1007/978-1-4757-3748-6_1

[17] Sergey Brin and Lawrence Page. 1998. The anatomy of a large-scale hypertextual Web search engine. *Computer Networks and ISDN Systems* 30, 1 (1998), 107–117. https://doi.org/10.1016/S0169-7552(98)00110-X Proceedings of the Seventh International World Wide Web Conference.

[18] Benjamin Brock, Aydın Buluç, Timothy Mattson, Scott McMillan, and José Moreira. 2021. *The GraphBLAS C API Specification*. Technical Report. https://graphblas.org/

[19] Aydın Buluç, John Gilbert, and Viral B. Shah. 2011. *13. Implementing Sparse Matrices for Graph Algorithms*. Society for Industrial and Applied Mathematics, Chapter 13, 287–313. https://doi.org/10.1137/1.9780898719918.ch13

[20] Wei Cao, Lu Yao, Zongzhe Li, Yongxian Wang, and Zhenghua Wang. 2010. Implementing Sparse Matrix-Vector multiplication using CUDA based on a hybrid sparse matrix format. In *2010 International Conference on Computer Application and System Modeling (ICCASM 2010)*, Vol. 11. V11–161–V11–165. https://doi.org/10.1109/ICCASM.2010.5623237

[21] Timothy M. Chan. 2007. More Algorithms for All-Pairs Shortest Paths in Weighted Graphs. In *Proceedings of the Thirty-Ninth Annual ACM Symposium on Theory of Computing* (San Diego, California, USA) *(STOC '07)*. Association for Computing Machinery, New York, NY, USA, 590–598. https://doi.org/10.1145/1250790.1250877

[22] Stephen Chou, Fredrik Kjolstad, and Saman Amarasinghe. 2018. Format Abstraction for Sparse Tensor Algebra Compilers. *Proc. ACM Program. Lang.* 2, OOPSLA, Article 123 (oct 2018), 30 pages. https://doi.org/10.1145/3276493

[23] Xilinx Corporation. 2021. *Vitis Sparse Library*. Technical Report. https://xilinx.github.io/Vitis_Libraries/sparse/2021.1/index.html

[24] Vidushi Dadu, Jian Weng, Sihao Liu, and Tony Nowatzki. 2019. Towards General Purpose Acceleration by Exploiting Common Data-Dependence Forms. In *Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture* (Columbus, OH, USA) *(MICRO '52)*. Association for Computing Machinery, New York, NY, USA, 924–939. https://doi.org/10.1145/3352460.3358276

[25] Steven Dalton, Nathan Bell, Luke Olson, and Michael Garland. 2014. Cusp: Generic Parallel Algorithms for Sparse Matrix and Graph Computations. Retrieved May 2022 from http://cusplibrary.github.io/ Version 0.5.1.

[26] Mehmet Deveci, Simon David Hammond, Michael M. Wolf, and Sivasankaran Rajamanickam. 2018. *Sparse Matrix-Matrix Multiplication on Multilevel Memory Architectures: Algorithms and Experiments*. Technical Report. United States. https://doi.org/10.2172/1435688

[27] Stijn Dongen. 2000. Graph Clustering by Flow Simulation. *PhD thesis, Center for Math and Computer Science (CWI)* (05 2000).

[28] Yixiao Du, Yuwei Hu, Zhongchun Zhou, and Zhiru Zhang. 2022. High-Performance Sparse Linear Algebra on HBM-Equipped FPGAs Using HLS: A Case Study on SpMV. In *Proceedings of the 2022 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays* (Virtual Event, USA) *(FPGA '22)*. Association for Computing Machinery, New York, NY, USA, 54–64. https://doi.org/10.1145/3490422.3502368

[29] Iain S. Duff, Michael A. Heroux, and Roldan Pozo. 2002. An Overview of the Sparse Basic Linear Algebra Subprograms: The New Standard from the BLAS Technical Forum. *ACM Trans. Math. Softw.* 28, 2 (jun 2002), 239–267. https://doi.org/10.1145/567806.567810

[30] D. K. Faddeev and V. N. Faddeeva. 1981. Computational methods of linear algebra. *Journal of Soviet Mathematics* 15, 5 (1981), 531–650. https://doi.org/10.1007/BF01086544

[31] Mirko Farina, Usman Ahmad, Ahmad Taha, Hussein Younes, Yusuf Mesbah, Xiao Yu, and Witold Pedrycz. 2024. Sparsity in transformers: A systematic literature review. *Neurocomputing* 582 (2024), 127468. https://doi.org/10.1016/j.neucom.2024.127468

[32] Siying Feng, Jiawen Sun, Subhankar Pal, Xin He, Kuba Kaszyk, Dong-hyeon Park, Magnus Morton, Trevor Mudge, Murray Cole, Michael O'Boyle, Chaitali Chakrabarti, and Ronald Dreslinski. 2021. CoSPARSE: A Software and Hardware Reconfigurable SpMV Framework for Graph Analytics. In *2021 58th ACM/IEEE Design Automation Conference (DAC)*. 949–954. https://doi.org/10.1109/DAC18074.2021.9586114

[33] Sanford Friedenthal, Alan Moore, and Rick Steiner. 2012. Chapter 18 - Integrating SysML into a Systems Development Environment. In *A Practical Guide to SysML (Second Edition)* (second edition ed.), Sanford Friedenthal, Moore Alan, and Steiner Rick (Eds.). Morgan Kaufmann, Boston, 523–556. https://doi.org/10.1016/B978-0-12-385206-9.00018-1

[34] César Fuguet. 2023. HPDcache: Open-Source High-Performance L1 Data Cache for RISC-V Cores. In *Proceedings of the 20th ACM International Conference on Computing Frontiers* (, Bologna, Italy,) *(CF '23)*. Association for Computing Machinery, New York, NY, USA, 377–378. https://doi.org/10.1145/3587135.3591413

[35] Noble G, Nalesh S, and Kala S. 2023. MOSCON: Modified Outer Product based Sparse Matrix-Matrix Multiplication Accelerator with Configurable Tiles. In *2023 36th International Conference on VLSI Design and 2023 22nd International Conference on Embedded Systems (VLSID)*. 264–269. https://doi.org/10.1109/VLSID57277.2023.00061

[36] Trevor Gale, Matei Zaharia, Cliff Young, and Erich Elsen. 2020. Sparse GPU Kernels for Deep Learning. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis* (Atlanta, Georgia) *(SC '20)*. IEEE Press, Article 17, 14 pages. https://dl.acm.org/doi/10.5555/3433701.3433723

[37] Jianhua Gao, Weixing Ji, Fangli Chang, Shiyu Han, Bingxin Wei, Zeming Liu, and Yizhuo Wang. 2023. A Systematic Survey of General Sparse Matrix-Matrix Multiplication. *ACM Comput. Surv.* 55, 12, Article 244 (mar 2023), 36 pages. https://doi.org/10.1145/3571157

[38] Yingxue Gao, Lei Gong, Chao Wang, Teng Wang, Xi Li, and Xuehai Zhou. 2023. Algorithm/Hardware Co-Optimization for Sparsity-Aware SpMM Acceleration of GNNs. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 42, 12 (2023), 4763–4776. https://doi.org/10.1109/TCAD.2023.3281714

[39] John R. Gilbert, Steve Reinhardt, and Viral B. Shah. 2006. High-Performance Graph Algorithms from Parallel Sparse Matrices. In *Proceedings of the 8th International Conference on Applied Parallel Computing: State of the Art in Scientific Computing* (Umeå, Sweden) *(PARA'06)*. Springer-Verlag, Berlin, Heidelberg, 260–269. https://link.springer.com/chapter/10.1007/978-3-540-75755-9_32

[40] John R. Gilbert, Steve Reinhardt, and Viral B. Shah. 2008. A Unified Framework for Numerical and Combinatorial Computing. *Computing in Science & Engineering* 10, 2 (March 2008), 20–25. https://doi.org/10.1109/MCSE.2008.45

[41] Branko Grünbaum and Geoffrey C. Shephard. 1987. *Tilings and Patterns*. W. H. Freeman & Co., New York.

[42] Sumanth Gudaparthi, Sarabjeet Singh, Surya Narayanan, Rajeev Balasubramonian, and Visvesh Sathe. 2022. CANDLES: Channel-Aware Novel Dataflow-Microarchitecture Co-Design for Low Energy Sparse Neural Network Acceleration. In *2022 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. 876–891. https://doi.org/10.1109/HPCA53966.2022.00069

[43] Fred G. Gustavson. 1978. Two Fast Algorithms for Sparse Matrices: Multiplication and Permuted Transposition. *ACM Trans. Math. Softw.* 4, 3 (sep 1978), 250–269. https://doi.org/10.1145/355791.355796

[44] Song Han, Xingyu Liu, Huizi Mao, Jing Pu, Ardavan Pedram, Mark A. Horowitz, and William J. Dally. 2016. EIE: Efficient Inference Engine on Compressed Deep Neural Network. *SIGARCH Comput. Archit. News* 44, 3 (jun 2016), 243–254. https://doi.org/10.1145/3007787.3001163

[45] Václav Hapla, David Horák, and Michal Merta. 2012. Use of Direct Solvers in TFETI Massively Parallel Implementation. In *Proceedings of the 11th International Conference on Applied Parallel and Scientific Computing* (Helsinki, Finland) *(PARA'12)*. Springer-Verlag, Berlin, Heidelberg, 192–205. https://doi.org/10.1007/978-3-642-36803-5_14

[46] Kartik Hegde, Hadi Asghari-Moghaddam, Michael Pellauer, Neal Crago, Aamer Jaleel, Edgar Solomonik, Joel Emer, and Christopher W. Fletcher. 2019. ExTensor: An Accelerator for Sparse Tensor Algebra. In *Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture* (Columbus, OH, USA) *(MICRO '52)*. Association for Computing Machinery, New York, NY, USA, 319–333. https://doi.org/10.1145/3352460.3358275

[47] John L. Hennessy and David A. Patterson. 2011. *Computer Architecture, Fifth Edition: A Quantitative Approach* (5th ed.). Morgan Kaufmann Publishers Inc., San Francisco, CA, USA.

[48] R. Hossain, J.C. Herbert, J.F. Gouger, and R. Bechade. 1998. A 5.2 nS cycle time floating point unit macrocell. In *Proceedings of the 24th European Solid-State Circuits Conference*. 136–139. https://doi.org/10.1109/ESSCIR.1998.186227

[49] Mohammad Hosseinabady and Jose Luis Nunez-Yanez. 2020. A Streaming Dataflow Engine for Sparse Matrix-Vector Multiplication Using High-Level Synthesis. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 39, 6 (June 2020), 1272–1285. https://doi.org/10.1109/TCAD.2019.2912923

[50] Yuwei Hu, Yixiao Du, Ecenur Ustun, and Zhiru Zhang. 2021. GraphLily: Accelerating Graph Linear Algebra on HBM-Equipped FPGAs. In *2021 IEEE/ACM International Conference On Computer Aided Design (ICCAD)*. 1–9. https://doi.org/10.1109/ICCAD51958.2021.9643582

[51] Paul Xuanyuanliang Huang, Yannis Tsividis, and Mingoo Seok. 2024. SPADES: A 0.54-GFLOPS/W Sparse Matrix Vector Multiplication Accelerator Featuring On-the-Fly GZIP

Decompression for 3.36X Reduction in Off-Chip Data Movement. In *Proceedings of the 29th ACM/IEEE International Symposium on Low Power Electronics and Design* (Newport Beach, CA, USA) *(ISLPED '24)*. Association for Computing Machinery, New York, NY, USA, 1–6. https://doi.org/10.1145/3665314.3670811

[52] Eun-Jin Im, Katherine Yelick, and Richard Vuduc. 2004. Sparsity: Optimization Framework for Sparse Matrix Kernels. *The International Journal of High Performance Computing Applications* 18, 1 (2004), 135–158. https://doi.org/10.1177/1094342004041296 arXiv:https://doi.org/10.1177/1094342004041296

[53] Intel. 2003. Intel Math Kernel Library. Retrieved May 2022 from https://www.intel.com/content/www/us/en/developer/tools/oneapi/onemkl.html#gs.ylo2j7

[54] Valentin Isaac--Chassande, Adrian Evans, Yves Durand, and Frédéric Rousseau. 2024. Dedicated Hardware Accelerators for Processing of Sparse Matrices and Vectors: A Survey. *ACM Trans. Archit. Code Optim.* 21, 2, Article 27 (Feb. 2024), 26 pages. https://doi.org/10.1145/3640542

[55] Valentin Isaac--Chassande, Adrian Evans, Yves Durand, and Frédéric Rousseau. 2024. SpDCache: Region-Based Reduction Cache for Outer-Product Sparse Matrix Kernels. In *2024 IEEE 35th International Conference on Application-specific Systems, Architectures and Processors (ASAP)*. IEEE Computer Society, Los Alamitos, CA, USA, 3–7. https://doi.org/10.1109/ASAP61560.2024.00012

[56] Satoshi Itoh, Pablo Ordejón, and Richard M. Martin. 1995. Order-N tight-binding molecular dynamics on parallel computers. *Computer Physics Communications* 88, 2 (1995), 173–185. https://doi.org/10.1016/0010-4655(95)00031-A

[57] JESD235A 2016. *JEDEC Updates Groundbreaking High Bandwidth Memory (HBM) Standard*. Technical Report. JEDEC. https://www.jedec.org/news/pressreleases/jedec-updates-groundbreaking-high-bandwidth-memory-hbm-standard

[58] Hanchen Jin, Zichao Yue, Zhongyuan Zhao, Yixiao Du, Chenhui Deng, Nitish Srivastava, and Zhiru Zhang. 2025. Vesper: A Versatile Sparse Linear Algebra Accelerator With Configurable Compute Patterns. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 44, 5 (May 2025), 1731–1744. https://doi.org/10.1109/TCAD.2024.3496882

[59] Jeremy Kepner, David Bader, Aydın Buluç, John Gilbert, Timothy Mattson, and Henning Meyerhenke. 2015. Graphs, Matrices, and the GraphBLAS: Seven Good Reasons. *Procedia Computer Science* 51, International Conference On Computational Science, ICCS 2015 (2015), 2453–2462. https://doi.org/10.1016/j.procs.2015.05.353

[60] David R. Kincaid, Thomas C. Oppe, and David M. Young. 1989. *ITPACKV 2D user's guide*. Technical Report. United States. https://doi.org/10.2172/7093021

[61] Vladimir Kiriansky, Yunming Zhang, and Saman Amarasinghe. 2016. Optimizing Indirect Memory References with Milk. In *Proceedings of the 2016 International Conference on Parallel Architectures and Compilation* (Haifa, Israel) *(PACT '16)*. Association for Computing Machinery, New York, NY, USA, 299–312. https://doi.org/10.1145/2967938.2967948

[62] Fredrik Kjolstad, Shoaib Kamil, Stephen Chou, David Lugato, and Saman Amarasinghe. 2017. The Tensor Algebra Compiler. *Proc. ACM Program. Lang.* 1, OOPSLA, Article 77 (oct 2017), 29 pages. https://doi.org/10.1145/3133901

[63] Scott P. Kolodziej, Mohsen Aznaveh, Matthew Bullock, Jarrett David, Timothy A. Davis, Matthew Henderson, Yifan Hu, and Read Sandstrom. 2019. The SuiteSparse Matrix Collection Website Interface. *Journal of Open Source Software* 4, 35 (2019), 1244. https://doi.org/10.21105/joss.01244

[64] Monica D. Lam, Edward E. Rothberg, and Michael E. Wolf. 1991. The Cache Performance and Optimizations of Blocked Algorithms. *SIGPLAN Not.* 26, 4 (apr 1991), 63–74. https://doi.org/10.1145/106973.106981

[65] Jure Leskovec and Andrej Krevl. 2014. *SNAP Datasets: Stanford Large Network Dataset Collection*. Retrieved May 2022 from http://snap.stanford.edu/data

[66] Ruipeng Li, Björn Sjögreen, and Ulrike Meier Yang. 2021. A New Class of AMG Interpolation Methods Based on Matrix-Matrix Multiplications. *SIAM Journal on Scientific Computing* 43, 5 (2021), S540–S564. https://doi.org/10.1137/20M134931X

[67] Sheng Li, Jung Ho Ahn, Richard D. Strong, Jay B. Brockman, Dean M. Tullsen, and Norman P. Jouppi. 2009. McPAT: An integrated power, area, and timing modeling framework for multicore and manycore architectures. In *2009 42nd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. Institute of Electrical and Electronics Engineers and Association for Computing Machinery, New York, NY, USA, 469–480.

[68] Shiqing Li, Shuo Huai, and Weichen Liu. 2023. An Efficient Gustavson-Based Sparse Matrix–Matrix Multiplication Accelerator on Embedded FPGAs. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 42, 12 (Dec 2023), 4671–4680. https://doi.org/10.1109/TCAD.2023.3281719

[69] Shiqing Li, Shuo Huai, and Weichen Liu. 2023. An Efficient Gustavson-Based Sparse Matrix–Matrix Multiplication Accelerator on Embedded FPGAs. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 42, 12 (Dec 2023), 4671–4680. https://doi.org/10.1109/TCAD.2023.3281719

[70] Zhiyao Li, Jiaxiang Li, Taijie Chen, Dimin Niu, Hongzhong Zheng, Yuan Xie, and Mingyu Gao. 2023. Spada: Accelerating Sparse Matrix Multiplication with Adaptive Dataflow. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2* (Vancouver, BC, Canada) *(ASPLOS 2023)*. Association for Computing Machinery, New York, NY, USA, 747–761. https://doi.org/10.1145/3575693.3575706

[71] Weifeng Liu and Brian Vinter. 2014. An Efficient GPU General Sparse Matrix-Matrix Multiplication for Irregular Data. In *2014 IEEE 28th International Parallel and Distributed Processing Symposium*. 370–381. https://doi.org/10.1109/IPDPS.2014.47

[72] Jason Lowe-Power, Abdul Mutaal Ahmad, Ayaz Akram, Mohammad Alian, Rico Amslinger, Matteo Andreozzi, Adrià Armejach, Nils Asmussen, Brad Beckmann, Srikant

Bharadwaj, Gabe Black, Gedare Bloom, Bobby R. Bruce, Daniel Rodrigues Carvalho, Jeronimo Castrillon, Lizhong Chen, Nicolas Derumigny, Stephan Diestelhorst, Wendy Elsasser, Carlos Escuin, Marjan Fariborz, Amin Farmahini-Farahani, Pouya Fotouhi, Ryan Gambord, Jayneel Gandhi, Dibakar Gope, Thomas Grass, Anthony Gutierrez, Bagus Hanindhito, Andreas Hansson, Swapnil Haria, Austin Harris, Timothy Hayes, Adrian Herrera, Matthew Horsnell, Syed Ali Raza Jafri, Radhika Jagtap, Hanhwi Jang, Reiley Jeyapaul, Timothy M. Jones, Matthias Jung, Subash Kannoth, Hamidreza Khaleghzadeh, Yuetsu Kodama, Tushar Krishna, Tommaso Marinelli, Christian Menard, Andrea Mondelli, Miquel Moreto, Tiago Mück, Omar Naji, Krishnendra Nathella, Hoa Nguyen, Nikos Nikoleris, Lena E. Olson, Marc Orr, Binh Pham, Pablo Prieto, Trivikram Reddy, Alec Roelke, Mahyar Samani, Andreas Sandberg, Javier Setoain, Boris Shingarov, Matthew D. Sinclair, Tuan Ta, Rahul Thakur, Giacomo Travaglini, Michael Upton, Nilay Vaish, Ilias Vougioukas, William Wang, Zhengrong Wang, Norbert Wehn, Christian Weis, David A. Wood, Hongil Yoon, and Éder F. Zulian. 2020. The gem5 Simulator: Version 20.0+. https://doi.org/10.48550/ARXIV.2007.03152

[73] Xiaobo Lu, Jianbin Fang, Lin Peng, Chun Huang, Zidong Du, Yongwei Zhao, and Zheng Wang. 2024. Mentor: A Memory-Efficient Sparse-dense Matrix Multiplication Accelerator Based on Column-Wise Product. *ACM Trans. Archit. Code Optim.* 21, 4, Article 79 (Nov. 2024), 25 pages. https://doi.org/10.1145/3688612

[74] Xiaoyang Lu, Boyu Long, Xiaoming Chen, Yinhe Han, and Xian-He Sun. 2024. ACES: Accelerating Sparse Matrix Multiplication with Adaptive Execution Flow and Concurrency-Aware Cache Optimizations. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3* (La Jolla, CA, USA) *(ASPLOS '24)*. Association for Computing Machinery, New York, NY, USA, 71–85. https://doi.org/10.1145/3620666.3651381

[75] J.-C. Luo. 1992. Algorithms for Reducing the Bandwidth and Profile of a Sparse Matrix. *Computers & Structures* 44, 3 (1992), 535–548. https://doi.org/10.1016/0045-7949(92)90386-E

[76] Shengbai Luo, Bo Wang, Yihao Shi, Xueyi Zhang, Qingshan Xue, and Sheng Ma. 2024. Sparm: A Sparse Matrix Multiplication Accelerator Supporting Multiple Dataflows. In *2024 IEEE 35th International Conference on Application-specific Systems, Architectures and Processors (ASAP)*. 122–130. https://doi.org/10.1109/ASAP61560.2024.00034

[77] Shenghong Ma, Jinwei Xu, Jingfei Jiang, Yaohua Wang, and Dongsheng Li. 2024. Funnel: An Efficient Sparse Attention Accelerator with Multi-Dataflow Fusion. In *2024 IEEE International Symposium on Parallel and Distributed Processing with Applications (ISPA)*. 1311–1318. https://doi.org/10.1109/ISPA63168.2024.00176

[78] Liviu Mafteiu-Scai. 2014. The Bandwidths of a Matrix. A Survey of Algorithms. *West University of Timisoara Annals, Timisoara, Romania* LII (12 2014), 183– 223. https://doi.org/10.2478/awutm-2014-0019

[79] Kiran Matam, Siva Rama Krishna Bharadwaj Indarapu, and Kishore Kothapalli. 2012. Sparse matrix-matrix multiplication on modern architectures. In *2012 19th Interna-*

tional Conference on High Performance Computing. 1–10. https://doi.org/10.1109/HiPC.2012.6507483

[80] Thaha Mohammed and Rashid Mehmood. 2022. Performance Enhancement Strategies for Sparse Matrix-Vector Multiplication (SpMV) and Iterative Linear Solvers. (2022). https://doi.org/10.48550/ARXIV.2212.07490 arXiv:2212.07490 [cs.DS]

[81] Francisco Muñoz Martínez, Raveesh Garg, Michael Pellauer, José L. Abellán, Manuel E. Acacio, and Tushar Krishna. 2023. Flexagon: A Multi-Dataflow Sparse-Sparse Matrix Multiplication Accelerator for Efficient DNN Processing. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3* (Vancouver, BC, Canada) *(ASPLOS 2023)*. Association for Computing Machinery, New York, NY, USA, 252–265. https://doi.org/10.1145/3582016.3582069

[82] Anurag Mukkara, Nathan Beckmann, and Daniel Sanchez. 2019. PHI: Architectural Support for Synchronization- and Bandwidth-Efficient Commutative Scatter Updates. In *Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture* (Columbus, OH, USA) *(MICRO '52)*. Association for Computing Machinery, New York, NY, USA, 1009–1022. https://doi.org/10.1145/3352460.3358254

[83] Francisco Muñoz-Martínez, José L. Abellán, Manuel E. Acacio, and Tushar Krishna. 2021. STONNE: Enabling Cycle-Level Microarchitectural Simulation for DNN Inference Accelerators. In *2021 IEEE International Symposium on Workload Characterization (IISWC)*. 201–213. https://doi.org/10.1109/IISWC53511.2021.00028

[84] Yuta Nagahara, Jiale Yan, Kazushi Kawamura, Daichi Fujiki, Masato Motomura, and Thiem Van Chu. 2025. DMSA: An Efficient Architecture for Sparse–Sparse Matrix Multiplication Based on Distribute-Merge Product Dataflow. *IEEE Transactions on Very Large Scale Integration (VLSI) Systems* 33, 7 (July 2025), 1858–1871. https://doi.org/10.1109/TVLSI.2025.3558895

[85] Yusuke Nagasaka, Satoshi Matsuoka, Ariful Azad, and Aydın Buluç. 2018. High-Performance Sparse Matrix-Matrix Products on Intel KNL and Multicore Architectures. In *Workshop Proceedings of the 47th International Conference on Parallel Processing* (Eugene, OR, USA) *(ICPP Workshops '18)*. Association for Computing Machinery, New York, NY, USA, Article 34, 10 pages. https://doi.org/10.1145/3229710.3229720

[86] NVIDIA. 2014. cuSPARSE Library. Retrieved May 2022 from https://developer.nvidia.com/cusparse

[87] Subhankar Pal, Jonathan Beaumont, Dong-Hyeon Park, Aporva Amarnath, Siying Feng, Chaitali Chakrabarti, Hun-Seok Kim, David Blaauw, Trevor Mudge, and Ronald Dreslinski. 2018. OuterSPACE: An Outer Product Based Sparse Matrix Multiplication Accelerator. In *2018 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. Institute of Electrical and Electronics Engineers, New York, NY, USA, 724–736. https://doi.org/10.1109/HPCA.2018.00067

[88] A. Parashar, M. Rhu, A. Mukkara, A. Puglielli, R. Venkatesan, B. Khailany, J. Emer, S. W. Keckler, and W. J. Dally. 2017. SCNN: An accelerator for compressed-sparse convolutional neural networks. In *2017 ACM/IEEE 44th Annual International Symposium*

on Computer Architecture (ISCA). IEEE Computer Society, Los Alamitos, CA, USA, 27–40. https://doi.org/10.1145/3079856.3080254

[89] Gerald Penn. 2006. Efficient transitive closure of sparse matrices over closed semirings. *Theoretical Computer Science* 354, 1 (2006), 72–81. https://doi.org/10.1016/j.tcs.2005.11.008 Algebraic Methods in Language Processing.

[90] Michael O Rabin and Vijay V Vazirani. 1989. Maximum matchings in general graphs through randomization. *Journal of Algorithms* 10, 4 (1989), 557–567. https://doi.org/10.1016/0196-6774(89)90005-9

[91] Vijay Janapa Reddi, Christine Cheng, David Kanter, Peter Mattson, Guenther Schmuelling, Carole-Jean Wu, Brian Anderson, Maximilien Breughe, Mark Charlebois, William Chou, Ramesh Chukka, Cody Coleman, Sam Davis, Pan Deng, Greg Diamos, Jared Duke, Dave Fick, J. Scott Gardner, Itay Hubara, Sachin Idgunji, Thomas B. Jablin, Jeff Jiao, Tom St. John, Pankaj Kanwar, David Lee, Jeffery Liao, Anton Lokhmotov, Francisco Massa, Peng Meng, Paulius Micikevicius, Colin Osborne, Gennady Pekhimenko, Arun Tejusve Raghunath Rajan, Dilip Sequeira, Ashish Sirasao, Fei Sun, Hanlin Tang, Michael Thomson, Frank Wei, Ephrem Wu, Lingjie Xu, Koichi Yamada, Bing Yu, George Yuan, Aaron Zhong, Peizhao Zhang, and Yuchen Zhou. 2020. MLPerf Inference Benchmark. In *2020 ACM/IEEE 47th Annual International Symposium on Computer Architecture (ISCA)*. 446–459. https://doi.org/10.1109/ISCA45697.2020.00045

[92] Oleg I. Ryabkov. 2022. Implementation of the Algebraic Multigrid Solver Designed for Graphics Processing Units Based on the AMGCL Framework. In *Parallel Computational Technologies*, Leonid Sokolinsky and Mikhail Zymbler (Eds.), Vol. 1618. Springer International Publishing, Cham, 131–142. https://doi.org/10.1007/978-3-031-11623-0_10

[93] Yousef Saad. 2003. *Iterative Methods for Sparse Linear Systems* (second ed.). Society for Industrial and Applied Mathematics. https://doi.org/10.1137/1.9780898718003

[94] Fazle Sadi, Joe Sweeney, Tze Meng Low, James C. Hoe, Larry Pileggi, and Franz Franchetti. 2019. Efficient SpMV Operation for Large and Highly Sparse Matrices Using Scalable Multi-Way Merge Parallelization. In *Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture* (Columbus, OH, USA) *(MICRO '52)*. Association for Computing Machinery, New York, NY, USA, 347–358. https://doi.org/10.1145/3352460.3358330

[95] Daniel Sanchez and Christos Kozyrakis. 2013. ZSim: Fast and Accurate Microarchitectural Simulation of Thousand-Core Systems. In *Proceedings of the 40th Annual International Symposium on Computer Architecture* (Tel-Aviv, Israel) *(ISCA '13)*. Association for Computing Machinery, New York, NY, USA, 475–486. https://doi.org/10.1145/2485922.2485963

[96] Conrad Sanderson and Ryan Curtin. 2016. Armadillo: a template-based C++ library for linear algebra. *Journal of Open Source Software* 1, 2 (2016), 26. https://doi.org/10.21105/joss.00026

[97] P. Scheffler, F. Zaruba, F. Schuiki, T. Hoefler, and L. Benini. 2023. Sparse Stream Semantic Registers: A Lightweight ISA Extension Accelerating General Sparse Linear

Algebra. *IEEE Transactions on Parallel and Distributed Systems* 34, 12 (dec 2023), 3147–3161. https://doi.org/10.1109/TPDS.2023.3322029

[98] Alan Jay Smith. 1982. Cache Memories. *ACM Comput. Surv.* 14, 3 (Sept. 1982), 473–530. https://doi.org/10.1145/356887.356892

[99] Shaden Smith, Jee W. Choi, Jiajia Li, Richard Vuduc, Jongsoo Park, Xing Liu, and George Karypis. 2017. *FROSTT: The Formidable Repository of Open Sparse Tensors and Tools*. Retrieved May 2022 from http://frostt.io/

[100] Linghao Song, Yuze Chi, Atefeh Sohrabizadeh, Young-kyu Choi, Jason Lau, and Jason Cong. 2022. Sextans: A Streaming Accelerator for General-Purpose Sparse-Matrix Dense-Matrix Multiplication. In *Proceedings of the 2022 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays* (Virtual Event, USA) *(FPGA '22)*. Association for Computing Machinery, New York, NY, USA, 65–77. https://doi.org/10.1145/3490422.3502357

[101] Linghao Song, Youwei Zhuo, Xuehai Qian, Hai Li, and Yiran Chen. 2018. GraphR: Accelerating Graph Processing Using ReRAM. In *2018 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. 531–543. https://doi.org/10.1109/HPCA.2018.00052

[102] Sriseshan Srikanth, Thomas M. Conte, Erik P. DeBenedictis, and Jeanine Cook. 2017. The superstrider architecture: Integrating logic and memory towards non-von Neumann computing. *2017 IEEE International Conference on Rebooting Computing, ICRC 2017 - Proceedings* 2017-January (2017), 1 – 8. https://doi.org/10.1109/ICRC.2017.8123669

[103] Sriseshan Srikanth, Anirudh Jain, Thomas M. Conte, Erik P. Debenedictis, and Jeanine Cook. 2021. SortCache: Intelligent Cache Management for Accelerating Sparse Data Workloads. *ACM Trans. Archit. Code Optim.* 18, 4, Article 56 (sep 2021), 24 pages. https://doi.org/10.1145/3473332

[104] Sriseshan Srikanth, Anirudh Jain, Joseph M. Lennon, Thomas M. Conte, Erik Debenedictis, and Jeanine Cook. 2019. MetaStrider: Architectures for Scalable Memory-centric Reduction of Sparse Data Streams. *ACM Trans. Archit. Code Optim.* 16, 4, Article 35 (oct 2019), 26 pages. https://doi.org/10.1145/3355396

[105] Nitish Srivastava, Hanchen Jin, Jie Liu, David Albonesi, and Zhiru Zhang. 2020. MatRaptor: A Sparse-Sparse Matrix Multiplication Accelerator Based on Row-Wise Product. In *2020 53rd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. Institute of Electrical and Electronics Engineers, New York, NY, USA, 766–780. https://doi.org/10.1109/MICRO50266.2020.00068

[106] Nitish Srivastava, Hanchen Jin, Shaden Smith, Hongbo Rong, David Albonesi, and Zhiru Zhang. 2020. Tensaurus: A Versatile Accelerator for Mixed Sparse-Dense Tensor Computations. In *2020 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. 689–702. https://doi.org/10.1109/HPCA47549.2020.00062

[107] Wenhao Sun, Wendi Sun, Song Chen, and Yi Kang. 2025. IOPS: A Unified SpMM Accelerator Based on Inner-Outer-Hybrid Product. *IEEE Trans. Comput.* 74, 7 (July 2025), 2210–2222. https://doi.org/10.1109/TC.2025.3558013

[108] Vivienne Sze, Yu-Hsin Chen, Tien-Ju Yang, and Joel Emer. 2020. *Efficient Processing of Deep Neural Networks* (1 ed.). Springer Cham. https://doi.org/10.1007/978-3-031-01766-7

[109] O. Temam and W. Jalby. 1992. Characterizing the behavior of sparse algorithms on caches. In *Supercomputing '92:Proceedings of the 1992 ACM/IEEE Conference on Supercomputing*. 578–587. https://doi.org/10.1109/SUPERC.1992.236646

[110] Reginald P. Tewarson. 1973. *Sparse matrices*. Vol. 69. Academic press New York, State University of New York, Stony Brook, NY.

[111] Richard Vuduc, James W Demmel, and Katherine A Yelick. 2005. OSKI: A library of automatically tuned sparse matrix kernels. *Journal of Physics: Conference Series* 16, 1 (jan 2005), 521. https://doi.org/10.1088/1742-6596/16/1/071

[112] F. Vázquez, E M Garzon, Jose Martinez, and J Fernández. 2009. The sparse matrix vector product on GPUs. *Proceedings of the 2009 International Conference on Computational and Mathematical Methods in Science and Engineering* 2 (01 2009).

[113] Chen Wang, Chuan Xu, and Abdel Lisser. 2014. Bandwidth Minimization Problem. In *10th International Conference on MOdeling, Optimization and SIMulation - MOSIM14*. Nancy, France. https://hal.science/hal-01166658

[114] Hongyi Wang, Kai Zhong, Haoyu Zhang, Shulin Zeng, Zhenhua Zhu, Xinhao Yang, Shuang Wang, Guohao Dai, Huazhong Yang, and Yu Wang. 2024. DySpMM: From Fix to Dynamic for Sparse Matrix-Matrix Multiplication Accelerators. In *Proceedings of the 61st ACM/IEEE Design Automation Conference* (San Francisco, CA, USA) *(DAC '24)*. Association for Computing Machinery, New York, NY, USA, Article 96, 6 pages. https://doi.org/10.1145/3649329.3657362

[115] Andrew Waterman, Yunsup Lee, David Patterson, and Krste Asanović. 2016. The RISC-V Instruction Set Manual, Volume I: User-Level ISA Version 2.1. (May 2016). https://riscv.org/specifications/ratified/ https://www2.eecs.berkeley.edu/Pubs/TechRpts/2016/EECS-2016-118.pdf.

[116] Lucas Wilkinson, Kazem Cheshmi, and Maryam Mehri Dehnavi. 2023. Register Tiling for Unstructured Sparsity in Neural Network Inference. *Proc. ACM Program. Lang.* 7, PLDI, Article 188 (June 2023), 26 pages. https://doi.org/10.1145/3591302

[117] Samuel Williams, Leonid Oliker, Richard Vuduc, John Shalf, Katherine Yelick, and James Demmel. 2007. Optimization of sparse matrix-vector multiplication on emerging multicore platforms. In *SC '07: Proceedings of the 2007 ACM/IEEE Conference on Supercomputing*. 1–12. https://doi.org/10.1145/1362622.1362674

[118] Yannan Nellie Wu, Po-An Tsai, Angshuman Parashar, Vivienne Sze, and Joel S. Emer. 2022. Sparseloop: An Analytical Approach To Sparse Tensor Accelerator Modeling. In *2022 55th IEEE/ACM International Symposium on Microarchitecture (MICRO)*. Institute of Electrical and Electronics Engineers and Association for Computing Machinery, New York, NY, USA, 1377–1395. https://doi.org/10.1109/MICRO56248.2022.00096

[119] Wm. A. Wulf and Sally A. McKee. 1995. Hitting the Memory Wall: Implications of the Obvious. *SIGARCH Comput. Archit. News* 23, 1 (mar 1995), 20–24. https://doi.org/10.1145/216585.216588

[120] Xinfeng Xie, Zheng Liang, Peng Gu, Abanti Basak, Lei Deng, Ling Liang, Xing Hu, and Yuan Xie. 2021. SpaceA: Sparse Matrix Vector Multiplication on Processing-in-Memory Accelerator. In *2021 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. 570–583. https://doi.org/10.1109/HPCA51647.2021.00055

[121] Ichitaro Yamazaki and Xiaoye S. Li. 2011. On Techniques to Improve Robustness and Scalability of a Parallel Hybrid Linear Solver. In *High Performance Computing for Computational Science – VECPAR 2010*, José M. Laginha M. Palma, Michel Daydé, Osni Marques, and João Correia Lopes (Eds.). Springer Berlin Heidelberg, Berlin, Heidelberg, 421–434.

[122] Enxin Yi, Jiarui Bai, Yijie Nie, Dan Niu, Zhou Jin, and Weifeng Liu. 2025. Leda: Leveraging Tiling Dataflow to Accelerate SpMM on HBM-Equipped FPGAs for GNNs. In *Proceedings of the 43rd IEEE/ACM International Conference on Computer-Aided Design* (Newark Liberty International Airport Marriott, New York, NY, USA) *(ICCAD '24)*. Association for Computing Machinery, New York, NY, USA, Article 215, 9 pages. https://doi.org/10.1145/3676536.3676773

[123] Jiangnan Yu, Yang Fan, Hanfei Wang, Yuxuan Qiao, Zheng Wu, Xiankui Xiong, Xiao Yao, Haidong Yao, and Yecheng Zhang. 2024. FullSparse: A Sparse-Aware GEMM Accelerator with Online Sparsity Prediction. In *Proceedings of the 21st ACM International Conference on Computing Frontiers* (Ischia, Italy) *(CF '24)*. Association for Computing Machinery, New York, NY, USA, 298–301. https://doi.org/10.1145/3649153.3649180

[124] Orestis Zachariadis, Nitin Satpute, Juan Gómez-Luna, and Joaquín Olivares. 2020. Accelerating sparse matrix–matrix multiplication with GPU Tensor Cores. *Computers & Electrical Engineering* 88 (2020), 106848. https://doi.org/10.1016/j.compeleceng.2020.106848

[125] Florian Zaruba and Luca Benini. 2019. The Cost of Application-Class Processing: Energy and Performance Analysis of a Linux-Ready 1.7-GHz 64-Bit RISC-V Core in 22-nm FDSOI Technology. *IEEE Transactions on Very Large Scale Integration (VLSI) Systems* 27, 11 (Nov 2019), 2629–2640. https://doi.org/10.1109/TVLSI.2019.2926114

[126] Chao Zhang, Maximilian Bremer, Cy Chan, John Shalf, and Xiaochen Guo. 2022. ASA: Accelerating Sparse Accumulation in Column-Wise SpGEMM. *ACM Trans. Archit. Code Optim.* 19, 4, Article 49 (sep 2022), 24 pages. https://doi.org/10.1145/3543068

[127] Guowei Zhang, Nithya Attaluri, Joel S. Emer, and Daniel Sanchez. 2021. Gamma: Leveraging Gustavson's Algorithm to Accelerate Sparse Matrix Multiplication. In *Proceedings of the 26th ACM International Conference on Architectural Support for Programming Languages and Operating Systems* (Virtual, USA) *(ASPLOS '21)*. Association for Computing Machinery, New York, NY, USA, 687–701. https://doi.org/10.1145/3445814.3446702

[128] Guowei Zhang, Webb Horn, and Daniel Sanchez. 2015. Exploiting commutativity to reduce the cost of updates to shared data in cache-coherent systems. In *2015 48th*

Annual IEEE/ACM International Symposium on Microarchitecture (MICRO). Institute of Electrical and Electronics Engineers and Association for Computing Machinery, New York, NY, USA, 13–25. https://doi.org/10.1145/2830772.2830774

[129] Guowei Zhang and Daniel Sanchez. 2019. Leveraging Caches to Accelerate Hash Tables and Memoization. In *Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture* (Columbus, OH, USA) *(MICRO '52)*. Association for Computing Machinery, New York, NY, USA, 440–452. https://doi.org/10.1145/3352460.3358272

[130] Xiaoyu Zhang, Zerun Li, Rui Liu, Xiaoming Chen, and Yinhe Han. 2024. GAS: General-Purpose In-Memory-Computing Accelerator for Sparse Matrix Multiplication. *IEEE Trans. Comput.* 73, 6 (June 2024), 1427–1441. https://doi.org/10.1109/TC.2024.3371790

[131] Zhekai Zhang, Hanrui Wang, Song Han, and William J. Dally. 2020. SpArch: Efficient Architecture for Sparse Matrix Multiplication. In *2020 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE Computer Society, Los Alamitos, CA, USA, 261–274. https://doi.org/10.1109/HPCA47549.2020.00030

Contents